

North Echo Agent Security Lab - Complete Field Manual v1.0

This release manual combines the twelve validated module chapters. The cold capstone contract is in the repository and deliberately contains no guided solution.

North Echo Agent Security Lab

v0.1 field manual - Modules 01-03

This is the self-contained teaching manual for the first three playable modules. It assumes no prior systems-programming expertise. The goal is not to memorize commands. The goal is to build a reliable mental model of what authority a Linux process carries and to verify every containment claim from observable kernel state.

The learning rhythm is: predict, run, observe, explain, break, repair, and verify.

Safety boundary: Use only a disposable Linux VM with synthetic fixtures. Do not load personal or employer credentials. Do not target external systems. Run as an ordinary user unless an exercise explicitly creates namespace-local root.

What this manual contains

- A command-reading primer so punctuation such as `|`, `>`, `$()`, and quotes is never magic.
- Detailed mental models for processes, `execve`, environment inheritance, file descriptors, namespaces, credentials, capabilities, and `no_new_privs`.
- Every guided command for Modules 01-03 followed by a line-by-line explanation.
- Complete guided source listings with explanations of each meaningful C or Python line.
- Expected output patterns, including values that legitimately vary by machine.
- Intentional failures, why they fail, and how to verify the correction.
- Independent lab contracts and verification plans without complete solutions.
- Troubleshooting notes, a glossary, and a compact command reference.

Starting, resetting, and replaying

From the repository root:

```
./scripts/attest-course
./lab-start 01.01
cd .student/01.01
less README.md
```

What each line does

- `./scripts/attest-course` recomputes hashes for canonical course material. A pass means it matches the recorded manifest.
- `./lab-start 01.01` creates or resumes a disposable workspace for Module 01, Lesson 01.
- `cd .student/01.01` enters the generated editing area. Edit here, not under `course/`.
- `less README.md` opens the workspace instructions. Press `q` to leave.

Return to the repository root before lifecycle commands:

```
./lab-status  
./lab-reset 01.01 --dry-run  
./lab-reset 01.01 --yes  
./lab-start 01.01
```

What each line does

- `lab-status` reads attempt counts, workspace state, and pass state without changing anything.
- `--dry-run` checks reset boundaries and prints intended actions without deleting the workspace.
- `--yes` confirms the scoped reset after the same containment checks pass.
- Starting again produces new synthetic IDs, names, paths, ports, and canaries while retaining only progress metadata.

Reset is part of the pedagogy. After reset, the old solution is gone. A new attempt tests understanding instead of recognition.

Reading command and code blocks

Shell command anatomy

A shell expands text, constructs an argument vector, applies redirections, and launches programs. Small punctuation marks can change data flow and authority.

```
grep -E '^CapEff:' /proc/self/status
```

What each token means

- `grep` is the program selected through `PATH`.
- `-E` enables extended regular expressions.
- `'^CapEff:'` is one quoted argument. The shell removes the quotes before launch.
- `^` anchors the expression at the beginning of a line.
- `/proc/self/status` is the input pathname. For the running `grep`, `/proc/self` identifies that process.

Variables and command substitution

```
CURRENT_UID=$(id -u)  
printf 'uid=%s\n' "$CURRENT_UID"
```

- `$(id -u)` launches `id`, captures stdout without its trailing newline, and substitutes the result.
- `CURRENT_UID=...` creates a shell variable. It is not inherited unless exported.
- `printf` receives a format string and a value as separate arguments.
- Double quotes permit expansion while preserving the result as one argument.
- `\n` tells `printf` to emit a newline.

Quoting changes which shell performs expansion

```
sh -c 'echo "child pid=$$"'  
sh -c "echo child pid=$$"
```

- Single quotes protect `$$` from the outer shell. The child shell expands it to the child's PID.
- Double quotes let the outer shell expand `$$` before `sh` starts. The child receives the parent's number as ordinary text.
- Both outputs can look plausible, so quoting must be reasoned about explicitly.

Pipes and redirections

```
strace -e trace=write ./program 2>&1 | grep write
```

- `strace ... ./program` starts the observed process.
- `2>&1` makes descriptor 2 use the destination currently used by descriptor 1.

- `|` connects the left command's stdout to the right command's stdin.
- `grep write` filters display. It does not change the traced process.
- Pipeline stages are separate processes with separate descriptors and exit statuses.

Exit status

```
command && echo passed
command || echo failed
```

- Processes return a small integer status. Zero conventionally means success.
- `&&` runs the right side only after status zero.
- `||` runs the right side only after nonzero status.
- `;` sequences commands regardless of status.

C source anatomy

- `#include` supplies declarations from a header.
- `main` is the user-space entry point after the program image is initialized.
- `argc` counts argument pointers; `argv` is a NUL-terminated pointer array.
- Successful `exec` does not create a process and does not return. It replaces the current program image.
- Most syscall wrappers report failure as `-1` and set `errno`; `perror` formats that error.
- File descriptors are process-local integers referring to kernel-managed open objects.
- Bitwise OR (`|`) combines flag bits; bitwise AND (`&`) tests bits. They differ from logical `||` and `&&`.

Reading rule: Identify the authority-transfer point in every launcher. Hygiene and policy that must govern the executed program must be established before `exec`.

Module 01 - Processes, syscalls, and inherited authority

Module outcome and mental model

You will trace shell text into a process, an execve transition, and syscalls. Then you will remove two quiet authority channels: inherited environment values and open file descriptors.

```
parent process
-> optional fork or clone
-> prepare arguments, environment, and descriptors
-> execve(new program, argv, envp)
-> new program continues with surviving process authority
```

fork creates a process by copying state. execve replaces a process image while preserving selected process attributes. A syscall crosses from user space into the kernel. The new executable is not automatically a clean slate.

01.01 - Processes are the boundary you actually launched

Goals

- Identify a shell and its parent from kernel state.
- Separate library calls from kernel-visible syscalls.
- Read a focused strace.
- Observe why one source call need not equal one syscall.

Exercise 1 - Establish the process facts

```
pwd
printf 'shell pid=%s parent=%s\n' "$$" "$PPID"
ps -o pid,ppid,user,stat,comm,args -p $$ -p $PPID
```

What each line does

- `pwd` confirms the disposable workspace directory.
- `printf` formats two shell variables; `%s` accepts a string and `\n` ends the line.
- `$$` is this shell's PID and `$PPID` is its recorded parent PID.
- Quoting preserves each expanded value as one argument.
- `ps -o ...` selects explicit columns instead of distribution defaults.
- `pid` and `ppid` expose the relationship; `user` is the effective account; `stat` is process state; `comm` is executable name; `args` is the argument vector.
- The repeated `-p` options restrict output to the shell and its parent.

Expected shape:

```
/path/to/repository/.student/01.01
shell pid=1842 parent=1790
  PID    PPID USER   STAT  COMMAND  COMMAND
  1790    1789 student  Ss   terminal  ...
  1842    1790 student  S    bash     bash
```

Numbers and names vary. The relationship is the observation: the shell's PPID identifies the other row. A process is a kernel object with identity, memory, descriptors, credentials, and namespace memberships. Command text is only input used to launch it.

```
readlink /proc/$$/exe
ls -l /proc/$$/ns
```

What each line does

- `/proc/$$` is procfs state for the current shell.
- `exe` links to the executable object backing the process.
- `readlink` prints the target without executing it.
- `ns` contains one symbolic link per namespace type.

- `ls -l` exposes targets such as `mnt : [4026531841]`; the bracketed identity is useful when comparing processes.

Exercise 2 - Build and trace a small process

Complete source:

```
#include <stdio.h>
#include <unistd.h>

int main(void) {
    setvbuf(stdout, NULL, _IONBF, 0);
    puts("userspace: about to write");
    const char message[] = "kernel-visible write\n";
    if (write(STDOUT_FILENO, message, sizeof(message) - 1) < 0) {
        perror("write");
        return 1;
    }
    return 0;
}
```

Source, line by line

- `<stdio.h>` declares `stdout`, `setvbuf`, `puts`, and `perror`.
- `<unistd.h>` declares `write` and `STDOUT_FILENO`.
- `int main(void)` declares an integer result and no command-line parameters.
- `setvbuf(..., _IONBF, 0)` disables `stdout` buffering for a clearer observation.
- `puts` is a library call. It appends a newline and eventually needs a syscall to make bytes visible.
- The `const char` array contains visible bytes, `\n`, and a terminating NUL.
- Descriptor 1 is `stdout` by convention.
- `sizeof(message) - 1` excludes the terminating NUL.
- A negative `write` result indicates failure. `perror` explains `errno`.
- Return 1 reports failure; return 0 reports success.

Build and run:

```
sed -n '1,200p' hello-syscall.c
cc -std=c11 -Wall -Wextra -O2 hello-syscall.c -o hello-syscall
./hello-syscall
```

What each line does

- `sed -n` suppresses default output and `'1, 200p'` prints source lines 1 through 200.
- `cc` invokes the compiler driver.
- `-std=c11` selects C11; `-Wall -Wextra` enable useful diagnostics; `-O2` enables normal optimization.
- `-o hello-syscall` names the output binary.
- `./` executes the local file instead of searching `PATH`.

Expected:

```
userspace: about to write
kernel-visible write
```

Trace it:

```
strace -f -e trace=execve,write,exit_group ./hello-syscall 2>&1
```

What each part does

- `strace` observes syscalls made by its target.
- `-f` follows descendants if any are created.
- `-e trace=...` restricts output to execution, writes, and process exit.
- `2>&1` merges trace stderr with stdout for one transcript.

Expected pattern:

```
execve("./hello-syscall", ["./hello-syscall"], ...) = 0
write(1, "userspace: about to write\n", 26) = 26
write(1, "kernel-visible write\n", 21) = 21
exit_group(0) = ?
```

Lengths can vary. Read each call as arguments followed by a return value.

Exercise 3 - Break a naive prediction

Predict that one library output call means one `write`, then run:

```
strace -e trace=write ./hello-syscall 2>&1 | grep write
```

- The trace filter records only `write` syscalls.
- Redirection exposes trace output to the pipeline.
- `grep` changes display only.

Temporarily change `_IONBF` to `_IOFBF`, rebuild, and trace again. Full buffering may combine or delay writes depending on the destination and C library. Restore `_IONBF` afterward.

The security lesson is not buffering trivia. Policies written in terms of syscalls must be validated against syscalls that actually occur, not those guessed from source function names.

Troubleshooting and checkpoint

- If `strace` is missing, install it inside the disposable VM.
- If output order surprises you, remember stdout and stderr have different buffering and become one stream only after redirection.
- If namespace links are absent, confirm the environment is Linux and `/proc` is mounted.
- You should now be able to name the process, the `execve`, and the syscall producing visible output.

01.02 - Environment inheritance is authority

Mental model

execve conceptually receives path, argv, and envp. envp is an array of KEY=VALUE strings supplied by the launcher. The new program need not request individual values. Environment entries can influence command lookup, language runtimes, proxy use, locale, library loading, and application configuration.

Exercise 1 - Observe the inherited value

```
export DEMO_AGENT_TOKEN="fake-$(awk -F= '/fixture_id/{print $2}' FIXTURE.txt)"
python3 show-env.py
```

What each part does

- `export` marks a variable for inheritance by later children.
- The value is deliberately synthetic.
- `$(...)` runs `awk` and substitutes its output.
- `-F=` makes `=` the field separator; the pattern selects the fixture line; `{print $2}` emits its value.
- The outer quotes preserve the result as one argument.
- Python inherits exported variables because no replacement environment is supplied.

Expected:

```
DEMO_AGENT_TOKEN=fake-<random fixture id>
PATH=<current path>
```

The child did not open a credential store. The parent supplied the string across exec.

```
strace -f -e trace=execve python3 show-env.py 2>&1 | head -20
```

- `-f` follows descendants; the trace selector keeps execution transitions.
- `2>&1` exposes trace diagnostics to the pipe.
- `head -20` limits display without changing execution.
- Do not use verbose environment-dumping trace options around real secrets.

Exercise 2 - Inspect the intentional mistake

Launcher source:

```
#!/usr/bin/env python3
import os
import subprocess

# Intentional mistake: copying the full parent environment copies its authority.
child_env = os.environ.copy()
subprocess.run(["python3", "show-env.py"], env=child_env, check=True)
```

Launcher source, line by line

- The shebang locates Python through PATH when the script is directly executable.
- `os` exposes the process environment; `subprocess` exposes child execution.
- `os.environ.copy()` creates a separate dictionary containing every inherited key and value. Memory independence is not authority reduction.
- The list form preserves program and argument boundaries and avoids a shell parser.
- `env=child_env` supplies the full copied environment.
- `check=True` treats a nonzero child status as an exception. It does not make the environment safe.

Observer source:

```
#!/usr/bin/env python3
import os

print("DEMO_AGENT_TOKEN=" + os.environ.get("DEMO_AGENT_TOKEN", "<absent>"))
print("PATH=" + os.environ.get("PATH", "<absent>"))
```

Observer source, line by line

- `os.environ.get` returns the value or the literal default when the key is absent.
- `String +` joins the label and observation.
- The second line confirms that an intentionally retained PATH still reaches the child.

Run the vulnerable launcher:

```
sed -n '1,160p' launch-insecure.py
python3 launch-insecure.py
```

- Inspect before executing.
- Expected: the token remains. Supplying an explicit `env` argument is not equivalent to restricting that environment.

Exercise 3 - Construct an allowlist

Replace the copied dictionary with:

```
child_env = {"PATH": "/usr/bin:/bin", "LANG": "C.UTF-8"}
```

- This is a new dictionary, not a derivative of the parent environment.
- PATH permits lookup only in two named directories.
- LANG supplies predictable locale behavior.
- Absence is enforced at the authority-transfer point. The child cannot ignore a value it never receives.

Verify:

```
python3 launch-insecure.py | grep -F 'DEMO_AGENT_TOKEN=<absent>'
printf 'verification status=%s\n' "$?"
```

- `grep -F` performs a literal match.

- `$?` is the preceding pipeline's status in this shell; zero means the expected line was found.

An environment allowlist is narrow. It does not close descriptors, restrict files, remove credentials, or stop a program from fetching data from another service.

Troubleshooting and checkpoint

- If Python is not in the allowlisted directories, locate it deliberately with command `-v python3` inside the disposable VM.
- If the token remains, ensure the copied dictionary was replaced rather than followed by a later reassignment.
- Run without `grep` when diagnosing whether the launcher failed or the match was absent.
- You should be able to explain why copying a dictionary changes object identity but preserves authority.

01.03 - File descriptors cross exec

Mental model

Opening a path performs resolution and access checks, creates or references an open file description, and installs a process-local descriptor. Later reads use the descriptor; they do not repeat the original path lookup. A later pathname restriction does not revoke an already-open object.

Exercise 1 - Observe descriptor authority

Parent source:

```
#define _GNU_SOURCE
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char **argv) {
    if (argc != 2) {
        fprintf(stderr, "usage: %s PATH\n", argv[0]);
        return 2;
    }
    int fd = open(argv[1], O_RDONLY); /* Intentional mistake: no O_CLOEXEC. */
    if (fd < 0) {
        perror("open");
        return 1;
    }
    char *child[] = {"python3", "fd-child.py", NULL};
    execvp(child[0], child);
    perror("execvp");
    return 1;
}
```

Parent source, line by line

- `_GNU_SOURCE` exposes GNU/Linux declarations such as `O_CLOEXEC` before headers are read.
- `<fcntl.h>` declares open flags; `<stdio.h>` diagnostics; `<unistd.h>` POSIX execution interfaces.
- The argument check requires exactly one pathname.
- `open(..., O_RDONLY)` resolves the path and returns read authority as a descriptor.
- A negative descriptor indicates failure.
- `child` is a NUL-terminated argument vector.
- `execvp` searches `PATH`, replaces the program image, and retains descriptors without close-on-exec.
- Code after a successful exec never runs.

Child source:

```
#!/usr/bin/env python3
import os

for fd in range(0, 32):
    try:
        target = os.readlink(f"/proc/self/fd/{fd}")
    except OSError:
        continue
    print(f"fd={fd} target={target}")
    if fd >= 3:
        try:
            os.lseek(fd, 0, os.SEEK_SET)
            print(os.read(fd, 4096).decode(errors="replace"))
        except OSError:
            pass
```

Child source, line by line

- The bounded loop probes descriptor numbers 0 through 31. Production cleanup must not assume this upper bound.
- `/proc/self/fd/N` identifies the object behind descriptor N.
- `OSError` is expected for unopened numbers.
- Descriptors 3 and above are rewound when seekable and read directly.
- No protected pathname is opened by the child.
- Decode replacement keeps display robust; ignored errors do not prove a descriptor lacks other authority.

Build and run:

```
cc -std=c11 -Wall -Wextra -O2 fd-parent.c -o fd-parent
MANIFEST=$(python3 -c 'import json; print(json.load(open(".north-echo.json"))["fixture_manifest"])' )
./fd-parent "$MANIFEST"
```

What each line does

- The first line compiles the parent with warnings.
- `python3 -c` executes one quoted expression.
- `open` reads workspace metadata, `json.load` parses it, indexing selects the manifest path, and `print` emits it.
- Command substitution stores the printed path in `MANIFEST`.
- Quoting passes the entire path as one argument.

Expected: the child lists a descriptor above 2 pointing to `manifest.json` and can read synthetic fixture data. Descriptor 3 is common, not guaranteed.

Exercise 2 - Fix at creation time

```
int fd = open(argv[1], O_RDONLY | O_CLOEXEC);
```

- `O_RDONLY` requests read-only access.
- Bitwise OR combines independent flag bits.
- `O_CLOEXEC` asks the kernel to create the descriptor with `FD_CLOEXEC` atomically.
- Atomic creation avoids a multithreaded race between `open` and a later `fcntl`.

Rebuild and rerun. The protected descriptor should be absent after exec; descriptors 0, 1, and 2 may remain.

Exercise 3 - Verify before and after exec

Add after open and before execvp:

```
int flags = fcntl(fd, F_GETFD);
if (flags < 0) {
    perror("fcntl(F_GETFD)");
    return 1;
}
fprintf(stderr, "FD_CLOEXEC=%s\n", (flags & FD_CLOEXEC) ? "yes" : "no");
```

- F_GETFD reads descriptor flags, distinct from file status flags.
- Failure must be handled before treating the result as a mask.
- Bitwise AND tests only the close-on-exec bit.
- The conditional operator selects yes or no.
- The parent-side diagnostic and child-side absence are complementary evidence.

Troubleshooting and checkpoint

- If O_CLOEXEC is undeclared, confirm _GNU_SOURCE precedes all headers and the build is on Linux.
- Distinguish Python's own descriptors from the protected inherited object by target and controlled comparison.
- Never hard-code descriptor 3; the grader randomizes it.
- You should be able to explain why later path confinement cannot revoke a descriptor already handed to the child.

Module 01 independent practical - Hygienic launcher

Edit `launcher.c`. The grader invokes `./launcher COMMAND [ARG ...]` and requires normal command behavior, absence of synthetic `NE_LAB_TOKEN`, closure of every inherited descriptor above 2, and nonzero status on launch failure.

Starter source:

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <unistd.h>

int main(int argc, char **argv) {
    if (argc < 2) {
        fprintf(stderr, "usage: %s COMMAND [ARG ...]\n", argv[0]);
        return 2;
    }
    /* Intentional starter flaw: authority is transferred without hygiene. */
    execvp(argv[1], &argv[1]);
    perror("execvp");
    return 1;
}
```

The starter preserves arbitrary arguments but performs no authority hygiene.

```
cc -std=c11 -Wall -Wextra -O2 launcher.c -o launcher
./launcher /usr/bin/printf 'child works\n'
../../lab-grade module-01
../../lab-grade module-01 --mode exam
```

What each line does

- The compile line matches the grader interface and exposes warnings.
- The harmless probe checks execution and argument preservation.
- The relative grader path reaches the repository root from `. student/01. lab`; it copies no solution.
- Practice mode names properties and lesson references. Exam mode reduces hints but tests the same properties.

Plan and verify without a revealed solution

- Identify where the new environment is selected.
- Construct a small environment instead of subtracting unknown entries from a large one.
- Avoid assuming the randomized descriptor is 3.
- Preserve descriptors 0, 1, and 2.
- Remember that code after `execvp` represents failure only.
- Test functional, environment, descriptor, and failure properties separately.

Module 02 - Linux namespaces

Module outcome and mental model

Namespaces change what a process sees when it uses particular kernel interfaces. They do not form one all-or-nothing container switch. Each namespace type isolates a resource view, and useful containment requires deliberate composition.

- A **user namespace** changes how user and group IDs and capabilities are interpreted.
- A **UTS namespace** isolates hostname and domain-name state.
- A **PID namespace** changes visible process IDs and the process tree.
- A **mount namespace** isolates mount-table changes.
- A **network namespace** isolates interfaces, routes, ports, and network stacks.
- An **IPC namespace** isolates selected System V IPC and POSIX message queues.
- A **cgroup namespace** changes the cgroup hierarchy view; it does not impose resource limits by itself.

Two verification questions apply to every namespace exercise:

- Did the target process receive a namespace handle different from the observer's handle?
- Does the resource view actually match the intended namespace?

The first question checks membership. The second catches stale mounts, incorrect ordering, and a command that mentions `unshare` without achieving the required state.

Safety boundary: These exercises depend on unprivileged user namespaces. If the VM disables them, enable them in the disposable VM. Do not switch to a work host or run the course as host root to force progress.

02.01 - Read namespace identity

Goals

- Record baseline namespace handles.
- Create a mapped user namespace as an ordinary user.
- Interpret namespace-local UID 0 without confusing it with host root.
- Read the UID mapping that connects inside and outside identities.

Exercise 1 - Capture the baseline

```
printf '%-8s %s\n' TYPE HANDLE
for ns in user uts pid mnt net ipc cgroup; do
    printf '%-8s %s\n' "$ns" "$(readlink /proc/self/ns/$ns)"
done
```

What each line does

- The first `printf` emits headings. `%-8s` left-aligns a string in an eight-character field.
- `for ns in ...; do` begins a loop and assigns each listed namespace name to `ns` in turn.
- `readlink /proc/self/ns/$ns` asks `procfs` for this process's handle for the current type.
- `$(...)` substitutes that handle into `printf`.
- Quotes around both expansions preserve one argument per value.
- `done` ends the loop.

Expected shape:

TYPE	HANDLE
user	user:[4026531837]
uts	uts:[4026531838]
pid	pid:[4026531836]
mnt	mnt:[4026531841]
net	net:[4026531992]
ipc	ipc:[4026531839]
cgroup	cgroup:[4026531835]

Exact numbers vary. Equal handles mean two processes refer to the same namespace object for that type; unequal handles mean different membership.

Exercise 2 - Create a mapped user namespace

```
unshare --user --map-root-user sh -c '
    echo "inside uid=$(id -u)";
    readlink /proc/self/ns/user;
    cat /proc/self/uid_map
'
```

What each line does

- `unshare` creates requested namespaces for the command that follows.

- `--user` requests a new user namespace.
- `--map-root-user` maps the invoking ordinary identity to UID and GID 0 inside it.
- `sh -c` starts a shell whose program is the following quoted string.
- Single quotes keep the outer shell from expanding `$(id -u)` or interpreting inner newlines.
- Inside, `id -u` prints the namespace-visible effective UID.
- `readlink` records the new user namespace handle.
- `uid_map` describes how ranges of inside UIDs translate to parent-namespace UIDs.

Typical mapping shape:

```
inside uid=0
user:[4026532777]
      0          1000          1
```

The mapping line means one ID beginning at inside UID 0 maps to parent UID 1000. The parent value will match your ordinary VM account, not necessarily 1000.

Exercise 3 - Disprove “UID 0 means global root”

```
unshare --user --map-root-user sh -c 'id; touch /root/north-echo-test'
```

What each part does

- The namespace flags reproduce the mapped identity.
- `;` runs the second inner command whether or not `id` succeeds.
- `id` reports namespace-relative UID 0.
- `touch` attempts to create a file through the host-mounted `/root` path.
- The expected permission denial shows that credentials are interpreted relative to namespace ownership.

User namespaces can grant a process capabilities over resources owned by the new namespace. They do not grant corresponding power over parent-owned resources. A complete identity statement therefore includes the numeric IDs, the user namespace handle, and the ID mappings.

Troubleshooting and checkpoint

- `Operation not permitted` usually means unprivileged user namespaces are disabled by VM policy.
- An empty or surprising map means the mapping helper failed; inspect `stderr` instead of assuming root was created.
- Never interpret `id -u` alone as a privilege proof.
- You should be able to compare two namespace handles and explain the three columns in `uid_map`.

02.02 - Change UTS state without changing the host

Goals

- Isolate hostname state in a UTS namespace.
- Understand why namespace creation and authority to modify its state are separate concerns.
- Compare host and child state before and after the change.
- Inspect a live process from another terminal.

Exercise 1 - Baseline, change, and proof

```
HOST_BEFORE=$(hostname)
echo "host before=$HOST_BEFORE"
unshare --user --map-root-user --uts sh -c '
    echo "inside before=$(hostname)";
    hostname north-echo-lab;
    echo "inside after=$(hostname)";
    readlink /proc/self/ns/uts
'
echo "host after=$(hostname)"
```

What each line does

- The first command substitution captures the parent-visible hostname.
- The variable is local to this shell unless exported.
- `--user --map-root-user` creates namespace-local capability; `--uts` creates an isolated UTS view.
- Single quotes defer inner hostname substitutions until the child shell runs.
- The first inner line observes the initial copied hostname.
- `hostname north-echo-lab` changes only the new UTS namespace.
- The second inner observation proves the new value.
- `readlink` records the namespace identity associated with that state.
- The final line runs back in the parent after `unshare` exits and proves the host value is unchanged.

Expected relationship:

```
host before=<vm hostname>
inside before=<vm hostname>
inside after=north-echo-lab
uts:[different handle]
host after=<same vm hostname>
```

The mapped user namespace gives the process the relevant capability inside the newly owned UTS namespace. It does not create host authority.

Exercise 2 - Make the incomplete attempt

```
unshare --uts hostname broken-attempt
```

- `--uts` requests only a UTS namespace.
- `hostname broken-attempt` is the command to run inside it.
- An ordinary user generally lacks the capability needed for this setup without the mapped user namespace.
- Expected: `Operation not permitted` and no host change.

The failure is useful. Creating a namespace type does not automatically supply every credential needed to configure it.

Exercise 3 - Inspect a live namespace

In terminal one:

```
unshare --user --map-root-user --uts sh -c 'hostname north-echo-hold; echo $$; sleep 60'
```

- The flags create the temporary user and UTS namespaces.
- `hostname` assigns a recognizable value.
- `echo $$` prints the shell's PID as seen from the parent namespace running the terminal.
- `sleep 60` keeps the process and namespace alive. `Ctrl-C` ends it early.

In terminal two, substitute the printed decimal PID:

```
readlink /proc/PRINTED_PID/ns/uts
readlink /proc/self/ns/uts
```

- The first line observes the held process's UTS namespace.
- `/proc/self` in the second terminal refers to the inspecting process.
- Different handles prove different membership even if both hostnames happened to contain the same text.

Verification should inspect effective state, not merely search a script for the word `unshare`.

Troubleshooting and checkpoint

- If the first terminal exits before inspection, rerun with a longer harmless sleep.
- If `/proc/PRINTED_PID` is absent, confirm you used the host-visible PID and that the process is still alive.
- If the handles match, inspect the exact flags and quoting rather than changing the expected result.
- You should be able to state why the parent hostname remains unchanged.

02.03 - PID namespaces and the procfs mistake

Goals

- Start a shell as PID 1 inside a new PID namespace.
- Observe why an inherited procfs can show the wrong process view.
- Compose PID and mount namespaces with a matching procfs.
- Recognize the responsibilities assigned to namespace PID 1.

Exercise 1 - Create the incomplete setup

```
unshare --user --map-root-user --pid --fork sh -c '  
  echo "shell says pid=$$";  
  ps -o pid,ppid,comm | head  
,
```

What each line does

- The mapped user namespace provides safe namespace-local authority.
- `--pid` creates a PID namespace for children, not retroactively for the calling `unshare` process.
- `--fork` creates the child that can enter the new PID namespace.
- The quoted shell runs as the namespace's first process and expands its own `$$`.
- `ps` obtains process data through the currently mounted `/proc`.
- `head` limits display but cannot correct the underlying view.

Expected surprise: the shell reports PID 1 while `ps` may show many host processes or numbers that do not agree. The PID namespace changed, but the inherited procfs mount still represents the parent PID namespace.

Exercise 2 - Mount a matching procfs

```
unshare --user --map-root-user --pid --fork --mount --mount-proc sh -c '  
  echo "shell says pid=$$";  
  ps -o pid,ppid,comm;  
  echo "pid namespace=$(readlink /proc/self/ns/pid)";  
  echo "mount namespace=$(readlink /proc/self/ns/mnt)"  
,
```

What each line does

- `--mount` isolates mount-table changes from the parent.
- `--mount-proc` mounts a fresh procfs associated with the new PID namespace.
- `--pid --fork` creates and enters the PID namespace in the required child.
- The shell sees itself as PID 1.
- `ps` now reads the fresh procfs, so its process list should agree with the namespace-local IDs.
- The two `readlink` calls record both namespace identities.

Expected: a small process list containing the shell and observation tools, plus new PID and mount handles. Process counts can vary because ps, shells, and pipelines briefly create processes.

Exercise 3 - Observe PID 1 responsibility

```
unshare --user --map-root-user --pid --fork --mount --mount-proc sh -c '  
  (exit 0) &  
  sleep 0.2  
  ps -o pid,ppid,stat,comm  
,
```

What each line does

- Parentheses create a subshell; `exit 0` makes it terminate successfully.
- `&` runs that subshell asynchronously.
- `sleep 0.2` gives exit processing time to occur.
- The `stat` column exposes a zombie as `Z` if a dead child has not been reaped.

Namespace PID 1 has special signal behavior and becomes the adopter for orphaned descendants. A production runtime normally supplies a small init/reaper instead of casually making an arbitrary workload PID 1.

Why namespace composition matters

A PID namespace without a matching `procfs` produces contradictory observations. A UTS namespace without suitable namespace-local authority may not be configurable. A mount namespace does not automatically create a private filesystem tree. Namespaces isolate selected views; they are components, not a complete sandbox policy.

Troubleshooting and checkpoint

- If `--mount-proc` fails, confirm the mapped user namespace was created and the util-linux `unshare` version supports it.
- If `ps` shows the host, compare the mount namespace handle and inspect the active `/proc mount`.
- Do not grade by exact process count; grade by namespace relationships and the `procfs` view.
- You should be able to explain why `$$` and `ps` disagreed in the incomplete setup.

Module 02 independent practical - Namespace launcher

Implement `sandbox.sh`. The grader invokes:

```
NE_EXPECTED_HOSTNAME=randomized-value ./sandbox.sh COMMAND [ARG ...]
```

The command must receive fresh user, UTS, PID, and mount namespaces; the randomized hostname; namespace PID 1; a `procfs` matching the new PID namespace; and unchanged argument, stream, and exit behavior. The host hostname and mount table must remain unchanged.

Starter source:

```
#!/bin/sh
set -eu

if [ "$#" -eq 0 ]; then
    echo "usage: $0 COMMAND [ARG ...]" >&2
    exit 2
fi

# Intentional starter flaw: this does not create any isolation.
exec "$@"
```

Starter source, line by line

- `#!/bin/sh` selects a POSIX shell interpreter.
- `set -e` exits after unhandled command failure; `-u` rejects unset-variable expansion.
- `$#` is the number of supplied arguments.
- `[ ... ]` is a test command; `-eq` performs integer equality.
- `$0` is the script name. `>&2` sends usage text to `stderr`.
- `exec "$@"` replaces the script with the requested command and preserves each original argument.
- The starter has good forwarding behavior but no namespace isolation.

Probe and grade:

```
chmod +x sandbox.sh
./sandbox.sh sh -c 'echo "pid=$$ host=$(hostname)"; ps -o pid,ppid,comm'
../../lab-grade module-02
```

What each line does

- `chmod +x` adds executable permission.
- The probe supplies `sh` as the arbitrary command under test.
- Single quotes defer `$$` and `$(hostname)` until the sandboxed shell runs.
- `ps` tests whether `procfs` agrees with namespace-local PID state.
- The grader compares actual namespace handles and process state with its own; the word `unshare` alone cannot pass.

Plan and verify without a revealed solution

- Identify which namespace provides authority to configure the others as an ordinary user.
- Preserve `COMMAND [ARG . . .]` as separate arguments.
- Ensure the `hostname` value comes from the randomized environment variable, not a constant.
- Ensure mount changes occur in a distinct mount namespace.
- Verify PID membership and the `procfs` view independently.
- Check that the launched command's exit status reaches the caller.

The independent lab withholds exact flag ordering and implementation. Every required interface has appeared in the guided exercises.

Module 03 - Privilege, capabilities, and no_new_privs

Module outcome and mental model

Linux privilege is not a Boolean “root or not root” value. A process's authority can depend on:

- real, effective, saved, and filesystem user and group IDs;
- supplementary groups;
- the effective, permitted, inheritable, bounding, and ambient capability sets;
- the owning user namespace and its ID mappings;
- file capabilities and set-user-ID or set-group-ID transitions;
- the `no_new_privs` bit;
- open descriptors and other authority channels from earlier modules.

Capabilities split many traditional root powers into named bits. That improves precision, but it creates several interacting sets that must be observed separately.

- **Effective:** bits currently considered for capability checks.
- **Permitted:** capabilities the thread may make effective.
- **Inheritable:** input to capability calculations across exec when file capability rules participate.
- **Bounding:** a ceiling that limits capabilities obtained during exec.
- **Ambient:** selected capabilities that can survive exec of ordinary non-privileged programs.

`no_new_privs` adds a one-way rule: an exec transition may not grant privileges the process did not already possess. It does not remove current authority by itself.

03.01 - UIDs are not the whole privilege story

Goals

- Read complete UID, GID, group, capability, and `no_new_privs` state from `procs`.
- Decode a hexadecimal capability mask.
- Compare ordinary identity with namespace-local root.
- Find file capabilities that complicate simplistic UID reasoning.

Exercise 1 - Inspect current credentials

```
id
grep -E '^ (Uid|Gid|Groups|Cap(Inh|Prm|Eff|Bnd|Amb)|NoNewPrivs):' /proc/self/status
```

What each line does

- `id` summarizes the invoking process's user, primary group, and supplementary groups.
- `grep -E` enables extended regular-expression syntax.
- `^` requires a match at the start of a line.
- The outer alternatives select identity, groups, capabilities, or `NoNewPrivs`.
- `Cap(Inh|Prm|Eff|Bnd|Amb)` matches the five capability fields without repeating the prefix.
- `/proc/self/status` is generated by the kernel for the process opening it. The `grep` child inherits the shell's credentials for this observation.

Typical ordinary-user shape:

```
Uid:      1000      1000      1000      1000
Gid:      1000      1000      1000      1000
Groups: 1000 ...
CapInh: 0000000000000000
CapPrm: 0000000000000000
CapEff: 0000000000000000
CapBnd: 000001ffffffffffff
CapAmb: 0000000000000000
NoNewPrivs: 0
```

Values vary by VM and distribution. A nonzero bounding set does not mean those bits are currently effective; it records an upper bound for future transitions.

Decode the effective mask:

```
CAP_EFF=$(awk '/^CapEff:/{print $2}' /proc/self/status)
capsh --decode="$CAP_EFF"
```

What each line does

- The `awk` pattern selects the line beginning `CapEff:`.
- `{print $2}` emits the second whitespace-delimited field, the hexadecimal mask.

- Command substitution stores it in CAP_EFF.
- `capsh --decode=` translates set bits into capability names.
- Quoting ensures even an empty or unusual expansion remains one argument.

For a zero mask, expect an empty decoded set. For nonzero masks, record each capability name instead of describing the process merely as “privileged.”

Exercise 2 - Compare namespace-local root

```
unshare --user --map-root-user sh -c '
id;
grep -E "(Uid|Cap(Inh|Prm|Eff|Bnd|Amb)):" /proc/self/status;
capsh --decode=$(awk "/^CapEff:/{print \"\\$2\"}" /proc/self/status)
'
```

What each part does

- `unshare` creates a new user namespace and maps the ordinary caller to UID 0 inside it.
- The child `id` displays namespace-relative identity.
- The child `grep` prints UID and all capability fields.
- Nested `awk` extracts `CapEff` for decoding.
- The backslash before `$2` prevents the outer shell from consuming it before inner `awk` runs.
- Capability bits are meaningful relative to the user namespace that owns the target resource.

Expected: UID 0 and a set of capabilities inside the new user namespace, but no corresponding host-root authority.

Exercise 3 - Disprove the nonzero-UID simplification

```
getcap -r /usr/bin 2>/dev/null | head
```

What each part does

- `getcap` reads file capability extended attributes.
- `-r /usr/bin` walks that directory recursively.
- `2>/dev/null` discards expected permission or unsupported-file diagnostics. Do not suppress `stderr` while diagnosing an unexpected failure.
- `head` keeps the display short.

Many systems contain programs with file capabilities such as a narrowly scoped network capability. Executing one can change capability state even when the caller's effective UID is not zero. A review must record IDs, groups, namespace mappings, capability sets, executable attributes, and inherited resources.

Troubleshooting and checkpoint

- If `capsh` or `getcap` is missing, install the distribution's capability utilities package in the VM.
- Treat an all-zero `CapEff` as one observation, not proof that every authority channel is empty.
- Keep the namespace handle alongside the decoded bits when comparing processes.

- You should be able to explain why CapBnd may be nonzero while CapEff is zero.

03.02 - Drop capability sets deliberately

Goals

- Decode all five capability sets rather than reading only CapEff.
- Use setpriv to establish a capability-empty child.
- Observe why clearing only one set is an incomplete fix.
- Verify the resulting process rather than trusting requested flags.

Exercise 1 - Decode every set

```
for field in CapInh CapPrm CapEff CapBnd CapAmb; do
    value=$(awk -v key="$field:" ' $1==key{print $2}' /proc/self/status)
    printf '%-7s %s -> ' "$field" "$value"
    capsh --decode="$value"
done
```

What each line does

- The loop assigns each procfs field name to field.
- `awk -v key="$field:"` passes the current field plus its colon into awk.
- `$1==key` performs exact first-field equality; `{print $2}` emits the mask.
- Command substitution stores the mask in value.
- `printf` aligns the name, prints the raw mask, and deliberately omits a newline.
- `capsh --decode` prints capability names and completes the record.
- `done` closes the loop.

Do not collapse the five results into one adjective. Ask which set controls present checks and which sets constrain or enable future exec transitions.

Exercise 2 - Launch with empty sets

```
unshare --user --map-root-user setpriv \
    --bounding-set=-all \
    --inh-caps=-all \
    --ambient-caps=-all \
    sh -c 'grep -E "^(Groups|Cap(Inh|Prm|Eff|Bnd|Amb)):" /proc/self/status'
```

What each line does

- The mapped user namespace supplies a safe environment for practicing privilege transitions.
- A trailing backslash tells the shell the same command continues on the next physical line.
- `setpriv` changes process privilege attributes before executing the final command.
- `--bounding-set=-all` subtracts every named capability from the bounding set.

- `--inh-caps=-all` clears the inheritable set.
- `--ambient-caps=-all` clears the ambient set.
- The final shell runs the `procfs` observation after the transitions and reports supplementary groups separately from capability state.

Expected:

```
CapInh: 0000000000000000
CapPrm: 0000000000000000
CapEff: 0000000000000000
CapBnd: 0000000000000000
CapAmb: 0000000000000000
```

`setpriv` and the kernel perform linked capability calculations during execution, which is why the resulting permitted and effective sets are also observed rather than assumed.

The Groups line need not be empty. `--map-root-user` uses the kernel's safe single-ID mapping path, which disables later `setgroups(2)` calls before writing the GID map. Adding `setpriv --clear-groups` after that transition therefore fails with `Operation not permitted` on current Ubuntu kernels. Group reduction is a separate launcher responsibility that must happen at a boundary where the caller has authority to change its supplementary groups; this exercise isolates capability-set behavior instead of pretending the two controls are interchangeable.

Exercise 3 - Make the partial fix

```
unshare --user --map-root-user setpriv --inh-caps=-all sh -c 'grep -E "^Cap(Inh|Prm|Eff|Bnd|Amb):" /proc/self/status'
```

- This changes only the inheritable set request.
- The final `grep` deliberately prints all sets.
- Expected: `CapInh` is zero, while other sets need not be empty.

The phrase “drop capabilities” is underspecified. Name every relevant set, state the intended postcondition, and verify the kernel-reported result.

Capability caveats

- Clearing current sets does not close descriptors opened while capabilities were present.
- Dropping the bounding set is a one-way restriction for the process tree, but executable attributes and namespace context still need analysis.
- Supplementary groups are a separate authority channel.
- A zero capability mask does not restrict ordinary permissions granted by UID, GID, ACLs, or open resources.

Troubleshooting and checkpoint

- If `setpriv` is missing, install `util-linux` in the disposable VM.
- If an option spelling differs, check the VM's `util-linux` version rather than silently omitting the property.
- If a transition fails, capture `stderr` and current sets before changing the exercise.
- You should be able to state which set is used for current checks and which set caps future acquisition across `exec`.

03.03 - Make privilege non-gainable

Goals

- Set the one-way no_new_privs bit before exec.
- Observe that bit in the executed child and a grandchild.
- Understand why ordering around exec is a security property.
- Distinguish non-gainability from removal of existing authority.

Exercise 1 - Build the launcher

Complete source:

```
#define _GNU_SOURCE
#include <stdio.h>
#include <sys/prctl.h>
#include <unistd.h>

int main(int argc, char **argv) {
    if (argc < 2) {
        fprintf(stderr, "usage: %s COMMAND [ARG ...]\n", argv[0]);
        return 2;
    }
    if (prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0) != 0) {
        perror("prctl(PR_SET_NO_NEW_PRIVS)");
        return 1;
    }
    execvp(argv[1], &argv[1]);
    perror("execvp");
    return 1;
}
```

Source, line by line

- `_GNU_SOURCE` requests GNU/Linux declarations before headers are included.
- `<stdio.h>` declares diagnostics; `<sys/prctl.h>` declares `prctl` and its operation constants; `<unistd.h>` declares `execvp`.
- `argc < 2` rejects a missing child command.
- The usage message describes the preserved `COMMAND [ARG ...]` interface.
- `prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0)` asks the kernel to set the calling thread's one-way bit. The remaining zero arguments are unused for this operation.
- A nonzero return indicates failure, which must stop execution of untrusted code.
- `execvp(argv[1], &argv[1])` searches `PATH`, uses the requested command, and passes the command plus all remaining arguments as the new `argv`.
- `&argv[1]` points into the original pointer array instead of rebuilding it.
- The final diagnostic runs only when `exec` fails.

Build and observe:

```
sed -n '1,220p' nnp-launch.c
cc -std=c11 -Wall -Wextra -O2 nnp-launch.c -o nnp-launch
./nnp-launch sh -c 'grep -E "^(NoNewPrivs|CapEff):" /proc/self/status'
```

What each line does

- sed displays the full small launcher before execution.
- The compile line selects C11, warnings, normal optimization, and the output name.
- Every token after ./nnp-launch becomes the child command and arguments.
- sh -c executes the quoted observation after the launcher sets policy.
- The anchored expression selects only NoNewPrivs and CapEff.

Expected:

```
NoNewPrivs: 1
CapEff:      0000000000000000
```

NoNewPrivs: 1 is the required property. A zero effective set is an additional observation, not proof that every authority channel is empty.

Exercise 2 - Make the ordering mistake

Temporarily move the prctl block after execvp, rebuild, and rerun.

- A successful exec replaces the launcher, so code after it is unreachable on the success path.
- The executed child therefore observes the old NoNewPrivs value, normally 0.
- Restore the call before exec.

Security controls governing a transition must be established before that transition. Source presence is not enough; control-flow ordering is part of the property.

Exercise 3 - Verify inheritance in a descendant

```
./nnp-launch sh -c 'sh -c "grep ^NoNewPrivs: /proc/self/status"'
```

What each part does

- The launcher sets the bit and executes the first shell.
- The outer single quotes protect the inner command from the invoking shell.
- The first child shell launches another shell.
- The inner shell reads its own procfs status.
- Seeing 1 proves inheritance across more than one exec transition.

no_new_privs cannot be unset and is inherited across fork and exec. It prevents privilege gain from exec mechanisms such as set-user-ID and file capabilities. It does not close descriptors, scrub the environment, empty current capability sets, isolate files, restrict the network, or filter syscalls.

Troubleshooting and checkpoint

- If `prctl` is undeclared, verify Linux headers and `__GNU_SOURCE` placement.
- If the child reports 0, inspect control-flow ordering before changing the probe.
- If the command does not launch, distinguish `prctl` failure from `execvp` failure using `stderr`.
- You should be able to explain both what `no_new_privs` guarantees and what it deliberately does not guarantee.

Module 03 independent practical - Privilege floor

Implement `secure-launch.c`. The grader compiles it and invokes:

```
./secure-launch COMMAND [ARG ...]
```

The executed command must observe `NoNewPrivs: 1`, empty effective/permited/inheritable/ambient capability sets, the invoking ordinary user's real and effective UID, normal arguments and streams, and normal exit behavior. Do not add set-user-ID bits or file capabilities. Do not transform the command through an external shell.

Starter source:

```
#define _GNU_SOURCE
#include <stdio.h>
#include <unistd.h>

int main(int argc, char **argv) {
    if (argc < 2) {
        fprintf(stderr, "usage: %s COMMAND [ARG ...]\n", argv[0]);
        return 2;
    }
    /* Intentional starter flaw: no privilege floor is established. */
    execvp(argv[1], &argv[1]);
    perror("execvp");
    return 1;
}
```

Probe and grade:

```
cc -std=c11 -Wall -Wextra -O2 secure-launch.c -o secure-launch
./secure-launch sh -c \
    'grep -E "^(Uid|CapInh|CapPrm|CapEff|CapAmb|NoNewPrivs):" /proc/self/status'
../../lab-grade module-03
```

What each line does

- The compile line checks the same source interface used by the grader.
- The continuation backslash keeps the probe readable while remaining one shell command.
- The local probe reads the executed child's kernel-reported state rather than trusting source intent.
- The anchored expression selects identity, four required capability sets, and `NoNewPrivs`.
- The external grader creates fresh evaluation details and tests properties independently.

Plan and verify without a revealed solution

- Identify every state transition that must occur before exec.
- Treat `no_new_privs` and capability removal as distinct properties.
- Preserve the original command and arguments without shell reparsing.
- Handle failure for every security-relevant transition; do not continue in a partially configured state.
- Verify UID fields and every required capability field in the executed process.
- Remember that the module contract does not claim to close descriptors or sanitize the environment; those are separate controls from Module 01.

Integrated understanding

How the first three modules compose

A minimally disciplined local launcher now has several separate responsibilities:

```
parent process
-> select arguments without accidental shell parsing
-> construct an allowlisted environment
-> close or mark unwanted descriptors close-on-exec
-> create and configure namespace views
-> reduce groups and capability channels
-> set no_new_privs
-> exec the workload
-> verify the workload's effective state externally
```

Ordering matters. A descriptor opened before confinement can bypass later path restrictions. A `prctl` after `exec` never runs on success. A PID namespace paired with the old `procfs` gives a contradictory view. A copied environment preserves unknown parent authority.

No single observation proves complete containment. Instead, define properties and collect evidence for each:

- **Execution:** the intended binary and arguments run.
- **Environment:** disallowed keys are absent and required keys remain.
- **Descriptors:** unintended open objects do not survive `exec`.
- **Namespaces:** handles differ where required and resource views agree.
- **Credentials:** IDs, groups, and all capability sets match the declared floor.
- **Privilege gain:** `NoNewPrivs` is 1 in descendants.
- **Host preservation:** parent hostname, mounts, and process view remain unchanged.

A disciplined failure-reading loop

When a grader reports a failed property:

- Restate the property without guessing at a line-level fix.
- Reproduce it with the smallest harmless local probe.
- Observe the executed child, not just the launcher's source.
- Separate setup failure from workload failure using `stderr` and exit status.
- Change one authority channel at a time.
- Rerun the local probe, then the randomized grader.
- Reset and replay to ensure the fix was not tied to one fixture.

Completion, replay, and reference

Replay the completed sequence

```
./lab-status  
./lab-reset --all --dry-run  
./lab-reset --all --yes  
./lab-start 01.01
```

What each line does

- Status records the end state before reset.
- The dry run validates every reset target and resource without mutation.
- The confirmed all-scope reset removes disposable student work and generated fixtures while retaining attempt/pass metadata.
- Starting 01.01 creates a fresh attempt with no retained solution.

If the commands feel familiar but the answer is not sitting in front of you, replayability is working.

Quick command reference

- `ps -o ... -p PID` selects explicit process columns for named PIDs.
- `readlink /proc/PID/ns/TYPE` records namespace membership.
- `grep -E` uses extended regular expressions; `grep -F` matches literal text.
- `strace -f -e trace=LIST` follows descendants and restricts syscall display.
- `unshare` creates specified namespaces for a command.
- `setpriv` changes privilege attributes before executing a command.
- `capsh --decode=MASK` translates a hexadecimal capability mask.
- `getcap -r PATH` finds file capability attributes recursively.
- `fcntl(fd, F_GETFD)` reads descriptor flags.
- `prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0)` sets the one-way no-new-privileges bit.

Glossary

- **Ambient authority:** authority available to a program without an explicit request for each use, such as inherited environment values or descriptors.
- **Argument vector (argv):** the ordered, NUL-terminated array of strings supplied to a new program.
- **Capability:** a named Linux privilege bit evaluated in a namespace context.
- **Close-on-exec:** a descriptor flag instructing the kernel to close that descriptor during successful exec.
- **Environment (envp):** an array of KEY=VALUE strings supplied to an executed program.

- **Exec:** a transition replacing the current process image while preserving defined process state.
- **File descriptor:** a process-local integer reference to an open kernel object.
- **Namespace:** a kernel mechanism giving processes a scoped view of one resource class.
- **Namespace handle:** a procfs link target such as `uts : [ 4026531838 ]` identifying a namespace object.
- **PID 1:** the first process in a PID namespace, with special signal and child-reaping responsibilities.
- **procfs:** the `/proc` virtual filesystem exposing kernel-generated process and system views.
- **Property-based grading:** checking externally observable security outcomes rather than matching one source implementation.
- **Syscall:** a controlled transition from user space into the kernel to request an operation.
- `no_new_privs:` a sticky process attribute preventing exec from granting new privilege.
- **User namespace mapping:** the translation between IDs visible inside a user namespace and IDs in its parent.

What comes next

Modules 04-12 are currently planned scaffolds, not automatically generated lessons. Future releases will build them into the same pattern: guided examples, exact commands, line explanations, deliberate failures, randomized fixtures, independent labs, property-based graders, safe reset, and replay.

The planned path moves from a minimal tool-using agent through filesystem and syscall confinement, resource controls, network mediation, credential brokering, a composed runtime, vulnerable break/fix variants, and an adaptive cold-start capstone.

Module 04 - Build a minimal tool-using agent

Build a small deterministic agent loop with `read_file`, `write_file`, and `argv`-based command execution over synthetic tasks. Begin deliberately over-authorized, inventory inherited authority, then define and verify a narrow tool contract.

No AI service, network call, or API credential is used. A checked-in JSON task acts as the local scripted policy so every decision is reproducible and inspectable.

Play in order: 04.01, 04.02, 04.03, then module-04.

Outcomes:

- execute a deterministic sequence of named tools and emit one complete JSON record per action;
- preserve `argv` boundaries without shell reparsing and record nonzero tool results;
- inventory the launcher's environment, descriptors, and working directory without logging secret values;
- remove synthetic ambient environment authority before launching a child;
- distinguish an action request, an execution result, and evidence that the result actually occurred.

04.01 - Run a deterministic tool loop and record every action

Goal

Turn a local JSON task into explicit tool actions and a complete JSON-lines trace. The JSON task is a deterministic stand-in for a model: it makes the control flow reproducible and requires no network or credential.

Exercise 1 - Inspect the request before execution

```
sed -n '1,200p' task.json  
python3 -m json.tool task.json
```

Line by line

- `sed -n` prints the complete small fixture without modifying it.
- `python3 -m json.tool` parses the file and pretty-prints valid JSON; a nonzero exit means the request is malformed.
- Each action has a stable `id`, a named `tool`, and tool-specific arguments. The action is a request, not proof that the effect occurred.

Expected: two actions appear in order: `read_file` for `input.txt`, then `write_file` for `output.txt`.

Exercise 2 - Run the loop and inspect effects

Complete source:

```
#!/usr/bin/env python3
import json
import sys
from pathlib import Path

def perform(action: dict) -> dict:
    tool = action["tool"]
    if tool == "read_file":
        content = Path(action["path"]).read_text(encoding="utf-8")
        return {"content": content}
    if tool == "write_file":
        Path(action["path"]).write_text(action["content"], encoding="utf-8")
        return {"bytes_written": len(action["content"].encode())}
    raise ValueError(f"unknown tool: {tool}")

def main() -> int:
    if len(sys.argv) != 3:
        print(f"usage: {sys.argv[0]} TASK.json TRACE.jsonl", file=sys.stderr)
        return 2
    task = json.loads(Path(sys.argv[1]).read_text(encoding="utf-8"))
    trace_path = Path(sys.argv[2])
    with trace_path.open("w", encoding="utf-8") as trace:
        for action in task["actions"]:
            record = {"id": action["id"], "tool": action["tool"]}
            try:
                record.update({"ok": True, "result": perform(action)})
            except Exception as error:
                record.update({"ok": False, "error": str(error)})
            trace.write(json.dumps(record, sort_keys=True) + "\n")
            trace.flush()
    return 0

if __name__ == "__main__":
    raise SystemExit(main())
```

Source, line by line

- `json` parses requests and serializes trace records; `sys` supplies `argv` and exit handling; `Path` performs explicit file operations.
- `perform` dispatches on the exact tool name. Unknown names fail closed through `ValueError`.
- `read_file` returns observed content. `write_file` reports the encoded byte count after the write call returns.
- `main` requires exactly a task path and trace path, then parses the task before acting.
- Opening the trace with `"w"` creates one trace for this run. Each action begins with its stable ID and tool name.
- The `try` block converts success or failure into data. It does not silently claim success.
- JSON Lines uses one complete JSON object per line. `sort_keys=True` makes the local fixture deterministic; `flush` makes completed records immediately observable.
- `raise SystemExit(main())` turns the function result into the process exit status.

Run it:

```
rm -f output.txt trace.jsonl
python3 agent_loop.py task.json trace.jsonl
cat output.txt
python3 -c 'import json; [print(json.loads(line)) for line in open("trace.jsonl")]'
```

Line by line

- `rm -f` removes only the two lesson-generated files so an old result cannot masquerade as new evidence.
- The agent receives request and trace paths as separate arguments.
- `cat` verifies the filesystem effect independently of the trace.
- The Python one-liner parses every trace line; merely printing unvalidated text would not prove valid JSON.

Expected: `output.txt` contains `synthetic result`. The trace contains exactly two parseable records in request order, including the read content and write byte count.

Exercise 3 - Create and repair an evidence gap

Temporarily comment out the `trace.write(...)` line, remove old outputs, and rerun:

```
rm -f output.txt trace.jsonl
python3 agent_loop.py task.json trace.jsonl
wc -l trace.jsonl
test -f output.txt
```

Line by line

- The action still changes `output.txt`, but `wc -l` reports zero trace records.
- `test -f` returns success without printing; it proves the effect exists but says nothing about which request caused it.
- Restore `trace.write(...)`, rerun, and require `test "$(wc -l < trace.jsonl)" -eq 2`.

The intentional mistake is not a missing log decoration. It removes the evidence needed to connect requested actions to observed results.

Checkpoint and troubleshooting

```
test "$(wc -l < trace.jsonl)" -eq 2
test "$(cat output.txt)" = "synthetic result"
```

- If JSON parsing fails, inspect the exact line; JSON Lines requires one complete object per line.
- If the trace says success but the file is absent, the record was emitted before the effect or without checking it.
- If an old output survives, repeat the scoped `rm -f` command; never delete outside this lesson workspace.
- Checkpoint: explain why the request, trace record, and filesystem observation are three distinct facts.

04.02 - Preserve argv boundaries and handle tool failure

Goal

Execute a structured argument vector without passing it through a shell, then make nonzero status an explicit result instead of an exception that erases evidence.

Exercise 1 - Observe safe argument preservation

Complete runner source:

```
#!/usr/bin/env python3
import json
import subprocess
import sys
from pathlib import Path

def main() -> int:
    if len(sys.argv) != 2:
        print(f"usage: {sys.argv[0]} TASK.json", file=sys.stderr)
        return 2
    task = json.loads(Path(sys.argv[1]).read_text(encoding="utf-8"))
    child_env = {"PATH": "/usr/bin:/bin", "LANG": "C.UTF-8"}
    run = subprocess.run(
        task["argv"],
        shell=False,
        env=child_env,
        text=True,
        capture_output=True,
        check=False,
    )
    print(json.dumps({"argv": task["argv"], "status": run.returncode, "stdout": run.stdout, "stderr": run.stderr}, sort_keys=True))
    return 0 if run.returncode == 0 else 1

if __name__ == "__main__":
    raise SystemExit(main())
```

Source, line by line

- The imports provide JSON, process launch, argv handling, and path reads.
- The task must contain argv as a JSON array. Each element becomes exactly one child argument.
- `child_env` is constructed from constants; it does not copy unknown parent authority.
- `subprocess.run(task["argv"], shell=False, ...)` launches the vector directly. No shell interprets spaces, dollar signs, parentheses, semicolons, or redirections.
- Text capture keeps stdout and stderr separate. `check=False` preserves a nonzero result for the trace instead of raising before it can be recorded.
- The JSON result records the requested argv, exit status, stdout, and stderr. The runner itself exits nonzero when the tool did.

The observer is deliberately tiny:

```
#!/usr/bin/env python3
import json
import sys

print(json.dumps(sys.argv[1:]))
```

- `sys.argv[1:]` excludes the program name and exposes the exact two arguments received.
- JSON output makes spaces and punctuation unambiguous.

Run and verify:

```
rm -f SHELL_MARKER
python3 argv_runner.py task.json | python3 -m json.tool
test ! -e SHELL_MARKER
```

Expected: stdout contains `hello world` and the literal string `$(touch SHELL_MARKER)` as two array elements. The marker does not exist.

Exercise 2 - Trigger the shell-reparsing failure

Temporarily replace the `subprocess.run` call with this incomplete control:

```
run = subprocess.run(
    " ".join(task["argv"]),
    shell=True,
    env=child_env,
    text=True,
    capture_output=True,
    check=False,
)
```

Line by line

- `" ".join(...)` destroys the original argument boundaries.
- `shell=True` asks `/bin/sh` to interpret the resulting string.
- The task's dollar-sign expression becomes command substitution, so `touch` runs and creates `SHELL_MARKER`.
- Capturing output does not make shell interpretation safe.

Rerun the three verification commands. Expected: the argument output changes and `test ! -e SHELL_MARKER` fails. Restore the `argv-list` call and confirm the marker remains absent.

Exercise 3 - Record a nonzero tool result

Create a failure task and run it:

```
printf '%s\n' '{"argv":["sh","-c","printf failure-message >&2; exit 7"]}' > failure-task.json
python3 argv_runner.py failure-task.json > failure-result.json
STATUS=$?
cat failure-result.json | python3 -m json.tool
printf 'runner status=%s\n' "$STATUS"
```

Line by line

- `printf` creates one synthetic local task; single quotes prevent the outer shell from interpreting its punctuation.
- Redirection writes only inside the lesson workspace.
- The runner captures the child's `stderr` and status before returning failure itself.
- `$?` is saved immediately, before another command can replace it.
- Expected JSON contains status 7 and `failure-message`; the runner status is 1.

Checkpoint and troubleshooting

- If `SHELL_MARKER` exists during the repaired run, remove it first and inspect whether any string command or `shell=True` remains.
- If `python3` is not found, verify the fixed `PATH` for this disposable VM rather than copying the parent environment.
- If failure JSON is empty, the runner probably raised instead of recording `returncode`, `stdout`, and `stderr`.
- Checkpoint: explain why an `argv` array is a security boundary only while every layer preserves the array.

04.03 - Inventory and remove ambient launcher authority

Goal

Inventory the process state inherited by a local agent without printing secret values, reproduce an ambient-environment leak, and repair the child launch with an allowlist.

Exercise 1 - Inventory names and handles, not values

Complete source:

```
#!/usr/bin/env python3
import json
import os
import subprocess
import sys

def inventory() -> dict:
    descriptors = []
    for name in os.listdir("/proc/self/fd"):
        if name.isdigit():
            descriptors.append(int(name))
    return {"cwd": os.getcwd(), "environment_keys": sorted(os.environ), "descriptor_numbers": sorted(descriptors)}

def main() -> int:
    if len(sys.argv) != 2 or sys.argv[1] not in {"unsafe", "safe"}:
        print(f"usage: {sys.argv[0]} unsafe|safe", file=sys.stderr)
        return 2
    print("INVENTORY=" + json.dumps(inventory(), sort_keys=True))
    child_env = os.environ.copy() if sys.argv[1] == "unsafe" else {"PATH": "/usr/bin:/bin", "LANG": "C.UTF-8"}
    run = subprocess.run(["python3", "probe.py"], env=child_env, text=True, capture_output=True, check=False)
    print(run.stdout, end="")
    return run.returncode

if __name__ == "__main__":
    raise SystemExit(main())
```

Source, line by line

- `inventory` reads the current working directory, environment key names, and numeric descriptor entries.
- It deliberately does not read environment values or descriptor contents. An inventory should identify authority channels without copying secrets into logs.
- `/proc/self/fd` is a snapshot; the directory scan itself may temporarily use a descriptor.
- The mode must be the exact word `unsafe` or `safe`.
- Unsafe mode copies the entire parent environment. Safe mode constructs two known values.
- The child is launched as a structured `argv` list, and its output is captured for the observation.

The probe is:

```
#!/usr/bin/env python3
import os

print("NE_AGENT_SECRET=" + os.environ.get("NE_AGENT_SECRET", "<absent>"))
```

- The probe reads one synthetic key and reports absence explicitly.
- Do not adapt this exercise to real credential names or values.

Run the metadata-only inventory:

```
export NE_AGENT_SECRET="synthetic-$(awk -F= '/fixture_id/{print $2}' FIXTURE.txt)"
python3 authority_agent.py safe | sed "s/$NE_AGENT_SECRET/<redacted>/g"
```

Expected: the inventory lists NE_AGENT_SECRET as a key but does not contain its value. The child reports <absent>.

Exercise 2 - Reproduce ambient authority

```
python3 authority_agent.py unsafe | tee unsafe-output.txt
grep -F "$NE_AGENT_SECRET" unsafe-output.txt
```

Line by line

- Unsafe mode uses `os.environ.copy()`, transferring every inherited value.
- `tee` retains only this synthetic lesson output.
- `grep -F` treats the synthetic value literally and succeeds when the child leaked it.

This reproduces the Module 01 environment lesson inside a tool-using agent. Adding tools did not erase the launcher's inherited authority.

Exercise 3 - Repair and verify absence

```
python3 authority_agent.py safe > safe-output.txt
grep -F 'NE_AGENT_SECRET=<absent>' safe-output.txt
if grep -F "$NE_AGENT_SECRET" safe-output.txt; then exit 1; else echo 'AMBIENT ENVIRONMENT: REMOVED'; fi
```

Line by line

- Safe mode constructs a fixed environment at the child-exec boundary.
- The first `grep` requires the explicit absence observation.
- The `if` treats finding the synthetic value as failure; the `else` reports the verified repair.
- This repair does not constrain paths, syscalls, CPU, memory, or network access. Later modules add those independent controls.

Checkpoint and troubleshooting

- If the inventory prints values, reduce it to sorted key names before capturing evidence.

- If safe mode cannot locate Python, inspect the fixed VM path with command `-v python3`; do not copy the whole environment.
- If the unsafe grep fails, confirm the variable was exported in the same shell.
- Remove `unsafe-output.txt` after the observation; it contains only a synthetic value, but it is still disposable fixture data.
- Checkpoint: identify which authority is removed by the repair and name at least three channels it does not address.

Module 04 independent lab - Auditable local tool runner

Implement `agent.py`. The grader invokes:

```
python3 agent.py TASK.json TRACE.jsonl
```

The task is local JSON with an ordered `actions` array. Every action contains a unique string `id`, a `tool`, and the fields required by that tool:

- `read_file`: relative path;
- `write_file`: relative path and string content;
- `run_argv`: nonempty string array `argv`.

Your runner must:

- execute all three tools in request order;
- preserve `argv` boundaries without a shell parser;
- launch commands with a constructed environment that does not inherit `NE_AGENT_SECRET`;
- write exactly one valid JSON object per action to the requested JSONL trace;
- include `id`, `tool`, and Boolean `ok` in every record;
- record read content, write byte count, or command status/stdout/stderr as appropriate;
- record unknown tools and command failures as failures rather than claiming success;
- return nonzero if any action failed, while retaining the completed trace.

The starter is intentionally unsafe: it supports only `run_argv`, joins the vector into shell text, copies the parent environment, and produces incomplete records. The grader changes paths, content, arguments, and a synthetic environment canary on every run. It also checks a shell-metacharacter bypass and a separate failure task.

Use only interfaces practiced in Lessons 04.01-04.03. Do not add network access, an AI API, or a real credential.

```
python3 agent.py sample-task.json trace.jsonl
../../lab-grade module-04
../../lab-grade module-04 --mode exam
```

Test commands, line by line

- The first command exercises the required two-path interface with the included harmless `sample-task.json`.
- The grader creates fresh evaluation files outside the student workspace and supplies its own task and trace paths.
- Practice mode names failed properties and lesson references.
- Exam mode tests the same properties but suppresses repair-oriented references.

The lab intentionally does not provide a complete implementation. Plan the dispatcher, per-tool result fields, trace write point, failure aggregation, `argv` launch, and child environment before coding.

Module 05 - Confine filesystem access

Turn a directory name into an enforced filesystem boundary. First break lexical path checks with traversal and symlinks. Then anchor lookup to a directory descriptor with `openat2(2)` and add Landlock so an entire child process is restricted. Finally confront Landlock's important pre-opened-file-descriptor limit.

Play in order: 05.01, 05.02, 05.03, then `module-05`.

Outcomes:

- distinguish a pathname string from the kernel object reached during lookup;
- reproduce traversal, prefix-collision, and symlink escapes against a naive broker;
- use `openat2` with `RESOLVE_BENEATH`, `RESOLVE_NO_MAGICLINKS`, and `RESOLVE_NO_SYMLINKS` for descriptor-relative lookup;
- query the Landlock ABI, select supported rights, add a path-beneath rule, set `no_new_privs`, and restrict a child before `exec`;
- demonstrate that Landlock does not revoke already-open descriptors and remove that inherited authority before launch.

Prerequisites: Modules 01-04, Linux, a C compiler, Linux UAPI headers containing `openat2.h` and `landlock.h`, and a kernel with `openat2` and Landlock. The lessons require no root privilege, mount, network access, or host policy change.

Cross-layer boundary: `openat2` protects individual brokered lookups. Landlock restricts future filesystem operations by the launched process. Neither one closes an already-open descriptor; descriptor hygiene from Module 01 remains necessary.

05.01 - Break pathname string checks

Goal

Observe that a pathname is a lookup request, not an object identity. Break a plausible string-prefix policy with a similarly named sibling and with a symlink, then repair those demonstrations with resolved-object containment while naming the remaining race.

Exercise 1 - Build only synthetic local paths

```
rm -rf demo
mkdir -p demo/allowed demo/allowed-escape demo/protected
printf 'allowed\n' > demo/allowed/note.txt
printf 'prefix-secret\n' > demo/allowed-escape/secret.txt
printf 'symlink-secret\n' > demo/protected/secret.txt
ln -s ../protected/secret.txt demo/allowed/link.txt
python3 naive_open.py demo/allowed note.txt
```

Line by line

- `rm -rf demo` removes only the lesson-owned directory in the disposable workspace, preventing stale evidence.
- `mkdir -p` creates an allowed tree, a sibling whose name shares the allowed prefix, and a protected sibling.
- Each `printf` creates synthetic text; none is a real credential or host file.
- `ln -s` places a pathname inside the allowed tree whose target object is outside it.
- The final command asks the naive broker for an ordinary allowed file. Expected: allowed.

Exercise 2 - Exploit the lexical policy

Complete source of `naive_open.py`:

```
#!/usr/bin/env python3
import os
import sys
from pathlib import Path

def main() -> int:
    if len(sys.argv) != 3:
        print(f"usage: {sys.argv[0]} ROOT REQUEST", file=sys.stderr)
        return 2
    root = Path(sys.argv[1]).absolute()
    candidate = Path(os.path.abspath(root / sys.argv[2]))
    if not str(candidate).startswith(str(root)):
        print("DENIED: lexical prefix mismatch", file=sys.stderr)
        return 1
    print(candidate.read_text(encoding="utf-8"), end="")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())
```

Source, line by line

- `os.path.abspath` normalizes `..` text but does not resolve symlink targets.
- `Path.absolute` produces a pathname; it does not establish a kernel-enforced root.
- `startswith` compares characters. It cannot distinguish `allowed` from `allowed-escape` and does not identify the object behind `link.txt`.
- `read_text` performs a new lookup after the check, leaving both semantic mismatch and a check/use window.
- The exit status distinguishes a policy denial from normal completion.

Run both bypasses:

```
python3 naive_open.py demo/allowed ../allowed-escape/secret.txt
python3 naive_open.py demo/allowed link.txt
```

Line by line

- `..` reaches the sibling `allowed-escape`; its absolute string still begins with the characters ending in `allowed`.
- `link.txt` has an `allowed` lexical name, but normal lookup follows it to `demo/protected/secret.txt`.
- Expected: both synthetic secret strings print. This proves the check is not a confinement boundary; it does not imply access to any path beyond this lesson.

Exercise 3 - Repair the demonstrations, identify the limit

`resolved_open.py` resolves both root and candidate, then uses `relative_to` as a path-component comparison:

```
python3 resolved_open.py demo/allowed note.txt
python3 resolved_open.py demo/allowed ../allowed-escape/secret.txt || echo 'traversal denied'
python3 resolved_open.py demo/allowed link.txt || echo 'symlink escape denied'
```

Line by line

- The first lookup remains functional and prints `allowed`.
- `resolve(strict=True)` follows existing symlinks and rejects missing components.
- `relative_to(root)` compares path components, avoiding the prefix-collision error.
- `|| echo` runs only after the expected nonzero denial; it makes the result visible without hiding success as failure.
- Expected: the `allowed` read succeeds and both escapes print `DENIED` plus the matching shell message.

Intentional failure: replace `candidate.relative_to(root)` with the original `startswith` check and rerun the traversal. Restore the component-aware version before the checkpoint.

This repair is useful application validation, but it still separates checking from opening. Another process could exchange a checked component before `read_text` opens it. Lesson 05.02 asks the kernel to resolve and open in one operation.

Checkpoint and troubleshooting

```
test "$(python3 resolved_open.py demo/allowed note.txt)" = allowed
! python3 resolved_open.py demo/allowed ../allowed-escape/secret.txt
! python3 resolved_open.py demo/allowed link.txt
```

- ! succeeds only when the guarded command fails, so these are denial assertions.
- If ln says the link exists, rerun the scoped `rm -rf demo` setup.
- If `resolve(strict=True)` reports a missing file, verify the setup paths rather than weakening strict resolution.
- Checkpoint: explain why component-aware resolution repairs these examples but is not atomic authorization.

05.02 - Make lookup descriptor-relative with `openat2`

Goal

Replace check-then-open pathname logic with one kernel operation anchored to an already-open directory. Require resolution to remain beneath that descriptor and reject symlinks.

Exercise 1 - Compile the brokered open

Complete source of `safe_open.c`:

```
#define _GNU_SOURCE
#include <errno.h>
#include <fcntl.h>
#include <linux/openat2.h>
#include <stdio.h>
#include <string.h>
#include <sys/syscall.h>
#include <unistd.h>

int main(int argc, char **argv) {
    if (argc != 3) {
        fprintf(stderr, "usage: %s ROOT RELATIVE_PATH\n", argv[0]);
        return 2;
    }
    int root_fd = open(argv[1], O_PATH | O_DIRECTORY | O_CLOEXEC);
    if (root_fd == -1) {
        perror("open root");
        return 1;
    }
    struct open_how how = {
        .flags = O_RDONLY | O_CLOEXEC,
        .resolve = RESOLVE_BENEATH | RESOLVE_NO_MAGICLINKS | RESOLVE_NO_SYMLINKS,
    };
    int fd = syscall(SYS_openat2, root_fd, argv[2], &how, sizeof(how));
    if (fd == -1) {
        fprintf(stderr, "DENIED: %s\n", strerror(errno));
        close(root_fd);
        return 1;
    }
    char buffer[4096];
    ssize_t count;
    while ((count = read(fd, buffer, sizeof(buffer))) > 0) {
        if (write(STDOUT_FILENO, buffer, (size_t)count) != count) {
            perror("write");
            close(fd);
            close(root_fd);
            return 1;
        }
    }
    if (count == -1)
        perror("read");
    close(fd);
    close(root_fd);
    return count == -1;
}
```

Source, line by line

- `_GNU_SOURCE`, `linux/openat2.h`, and `sys/syscall.h` expose the Linux-specific `syscall` interface and `open_how` structure.
- The program accepts exactly a root and one relative request. A different shape fails with usage status 2.
- `open(..., O_PATH | O_DIRECTORY | O_CLOEXEC)` obtains a reference to the root directory without opening it for file data. `O_CLOEXEC` prevents accidental inheritance.
- `open_how.flags` requests a read-only result descriptor, also close-on-exec.
- `RESOLVE_BENEATH` rejects resolution that escapes above the supplied directory descriptor, including absolute paths.
- `RESOLVE_NO_MAGICLINKS` rejects `procfs`-style magic links; `RESOLVE_NO_SYMLINKS` rejects every symbolic link in the request.
- `syscall(SYS_openat2, root_fd, ...)` makes the resolution policy and open one atomic kernel request. The root descriptor, not the process working directory, is the anchor.
- A failure is printed as `DENIED` with the kernel reason and returns nonzero.
- The read/write loop copies observed bytes to standard output. It checks both input and output errors.
- Both descriptors are closed on every explicit completion path.

Build it:

```
cc -std=c11 -Wall -Wextra -Werror -O2 safe_open.c -o safe-open
```

Line by line

- `-std=c11` selects the language baseline; the Linux interfaces remain explicitly requested by `_GNU_SOURCE`.
- `-Wall -Wextra -Werror` turns common warnings into build failures rather than accepting ambiguous code.
- `-O2` produces a normal optimized binary; it is not a security boundary.
- `-o safe-open` names only the lesson-local output.

Exercise 2 - Verify allowed and denied lookups

```
rm -rf demo
mkdir -p demo/allowed demo/protected
printf 'allowed\n' > demo/allowed/note.txt
printf 'protected\n' > demo/protected/secret.txt
ln -s ../protected/secret.txt demo/allowed/link.txt
./safe-open demo/allowed note.txt
./safe-open demo/allowed ../protected/secret.txt || echo 'traversal denied'
./safe-open demo/allowed link.txt || echo 'symlink denied'
```

Line by line

- The setup uses only synthetic paths in this workspace.
- The plain relative lookup prints `allowed`.
- The `..` request fails because `RESOLVE_BENEATH` will not cross above `root_fd`.
- The link request fails because `RESOLVE_NO_SYMLINKS` rejects it before following the target.

- Expected denial errors commonly report Invalid cross-device link for escape and Too many levels of symbolic links for a forbidden symlink. Match success/failure, not locale-dependent wording.

Exercise 3 - Remove the boundary and repair it

Create a deliberately weakened copy that retains only magic-link denial:

```
sed 's/RESOLVE_BENEATH | RESOLVE_NO_MAGICLINKS | RESOLVE_NO_SYMLINKS/RESOLVE_NO_MAGICLINKS/' safe_open.c > unsafe_open.c
cc -std=c11 -Wall -Wextra -Werror -O2 unsafe_open.c -o unsafe-open
./unsafe-open demo/allowed ../protected/secret.txt
./unsafe-open demo/allowed link.txt
```

Line by line

- sed writes a new lesson-local source; it does not alter the repaired canonical example.
- Without RESOLVE_BENEATH, . . can leave the anchor. Without RESOLVE_NO_SYMLINKS, an ordinary symlink can be followed.
- Expected: both unsafe commands print protected.
- Delete unsafe_open.c and unsafe-open, then rerun the repaired denial commands. The repair is the policy in the openat2 operation itself, not a prior string check.

openat2 constrains only lookups made through this broker. A child can still call ordinary open itself. Lesson 05.03 adds process-wide future-access restrictions.

Checkpoint and troubleshooting

```
test "$(/safe-open demo/allowed note.txt)" = allowed
! ./safe-open demo/allowed ../protected/secret.txt
! ./safe-open demo/allowed link.txt
```

- If linux/openat2.h is absent, install the distribution's normal Linux UAPI development headers; do not copy an untrusted header into the lesson.
- ENOSYS means the running kernel lacks openat2; record the exact kernel and use a supported disposable VM.
- If the safe link succeeds, confirm the repaired binary was rebuilt from safe_open.c.
- Checkpoint: point to the directory descriptor, the kernel resolution flags, and the single operation that joins authorization to use.

05.03 - Restrict a child with Landlock

Goal

Apply an unprivileged Landlock ruleset before exec, then prove both its protection and its pre-opened-descriptor limit. Compose Landlock with descriptor hygiene rather than mistaking either control for the other.

Exercise 1 - Build a static observation probe

`fd_probe.c` opens a named path or reads an existing descriptor. Its complete source is:

```
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main(int argc, char **argv) {
    if (argc != 3) {
        fprintf(stderr, "usage: %s path PATH | fd NUMBER\n", argv[0]);
        return 2;
    }
    int fd = strcmp(argv[1], "path") == 0 ? open(argv[2], O_RDONLY) : atoi(argv[2]);
    char buffer[256];
    ssize_t count = read(fd, buffer, sizeof(buffer));
    if (count < 0) {
        fprintf(stderr, "DENIED: %s\n", strerror(errno));
        return 1;
    }
    return write(STDOUT_FILENO, buffer, (size_t)count) == count ? 0 : 1;
}
```

Source, line by line

- The probe accepts exactly a mode and operand.
- `path` performs a new open; `fd` reuses an integer descriptor inherited across `exec`.
- `read` is the common observation. On failure the probe reports `DENIED` and returns nonzero.
- The final `write` makes successful access independently visible.

Compile it statically inside the future allowed tree so its executable and runtime code need no outside filesystem access:

```
rm -rf demo
mkdir -p demo/allowed demo/protected
printf 'allowed\n' > demo/allowed/note.txt
printf 'synthetic-secret\n' > demo/protected/secret.txt
cc -std=c11 -Wall -Wextra -Werror -O2 -static fd_probe.c -o demo/allowed/fd-probe
```

Line by line

- The files are synthetic and local to the lesson.

- `-static` puts the probe's required runtime code in its binary. This keeps the policy example focused on one allowed tree rather than adding read/execute rules for dynamic-loader paths.
- A static build failure usually means the VM lacks its distribution C development files; install the documented build toolchain rather than broadening the policy.

Exercise 2 - Apply a version-aware Landlock policy

Complete source of `landlock_launch.c`:

```

#define _GNU_SOURCE
#include <errno.h>
#include <fcntl.h>
#include <linux/close_range.h>
#include <linux/landlock.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/prctl.h>
#include <sys/syscall.h>
#include <unistd.h>

static int create_ruleset(const struct landlock_ruleset_attr *attr, size_t size, __u32 flags) {
    return syscall(SYS_landlock_create_ruleset, attr, size, flags);
}

static int add_rule(int ruleset_fd, const struct landlock_path_beneath_attr *attr) {
    return syscall(SYS_landlock_add_rule, ruleset_fd, LANDLOCK_RULE_PATH_BENEATH, attr, 0);
}

static int restrict_self(int ruleset_fd) {
    return syscall(SYS_landlock_restrict_self, ruleset_fd, 0);
}

static __u64 supported_rights(int abi) {
    __u64 rights = LANDLOCK_ACCESS_FS_EXECUTE | LANDLOCK_ACCESS_FS_WRITE_FILE |
        LANDLOCK_ACCESS_FS_READ_FILE | LANDLOCK_ACCESS_FS_READ_DIR |
        LANDLOCK_ACCESS_FS_REMOVE_DIR | LANDLOCK_ACCESS_FS_REMOVE_FILE |
        LANDLOCK_ACCESS_FS_MAKE_CHAR | LANDLOCK_ACCESS_FS_MAKE_DIR |
        LANDLOCK_ACCESS_FS_MAKE_REG | LANDLOCK_ACCESS_FS_MAKE_SOCKET |
        LANDLOCK_ACCESS_FS_MAKE_FIFO | LANDLOCK_ACCESS_FS_MAKE_BLOCK |
        LANDLOCK_ACCESS_FS_MAKE_SYM;

    if (abi >= 2)
        rights |= LANDLOCK_ACCESS_FS_REFER;
    if (abi >= 3)
        rights |= LANDLOCK_ACCESS_FS_TRUNCATE;
    return rights;
}

static int close_inherited(void) {
    if (syscall(SYS_close_range, 3U, ~0U, CLOSE_RANGE_CLOEXEC) == 0)
        return 0;
    if (errno != ENOSYS)
        return -1;
    long maximum = sysconf(_SC_OPEN_MAX);
    for (int fd = 3; fd < maximum; fd++) {
        int flags = fcntl(fd, F_GETFD);
        if (flags != -1 && fcntl(fd, F_SETFD, flags | FD_CLOEXEC) == -1)
            return -1;
    }
    return 0;
}

int main(int argc, char **argv) {
    if (argc < 3) {
        fprintf(stderr, "usage: %s ALLOWED_ROOT COMMAND [ARG...]\n", argv[0]);
        return 2;
    }
    int abi = create_ruleset(NULL, 0, LANDLOCK_CREATE_RULESET_VERSION);
    if (abi < 1) {
        perror("Landlock ABI");
        return 1;
    }
    __u64 rights = supported_rights(abi);
    struct landlock_ruleset_attr ruleset = {.handled_access_fs = rights};
    int ruleset_fd = create_ruleset(&ruleset, sizeof(ruleset), 0);
    int root_fd = open(argv[1], O_PATH | O_CLOEXEC);
    if (ruleset_fd == -1 || root_fd == -1) {
        perror("create policy");
        return 1;
    }
    struct landlock_path_beneath_attr rule = {

```

```

        .allowed_access = rights,
        .parent_fd = root_fd,
    };
    if (add_rule(ruleset_fd, &rule) == -1) {
        perror("landlock_add_rule");
        return 1;
    }
    close(root_fd);
    if (prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0) == -1 || restrict_self(ruleset_fd) == -1) {
        perror("restrict self");
        return 1;
    }
    close(ruleset_fd);
    if (close_inherited() == -1) {
        perror("close inherited descriptors");
        return 1;
    }
    execv(argv[2], &argv[2]);
    perror("execv");
    return 1;
}

```

Source, line by line

- The three small syscall wrappers keep the argument ordering visible and return raw success or failure to the caller.
- `supported_rights` builds the ABI-1 set first. The two version tests prevent a newer right from making ruleset creation fail on an older supported kernel.
- `close_inherited` prefers the kernel's range operation. `CLOSE_RANGE_CLOEXEC` preserves descriptors until `exec` but guarantees the new program cannot inherit them.
- The fallback queries the descriptor limit and adds `FD_CLOEXEC` only to descriptors that are actually open. Any mutation error fails the launch.
- `main` queries the ABI before it creates policy. It never treats an unavailable Landlock interface as permission to continue.
- The ruleset declares which operations Landlock will handle; the path-beneath rule grants that same set only under `root_fd`.
- `PR_SET_NO_NEW_PRIVS` and `restrict_self` are joined by `||`: if either fails, `execv` is unreachable.
- The policy and root descriptors are closed before inherited descriptors are marked. Standard input, output, and error remain available.
- `execv(argv[2], &argv[2])` preserves the caller's structured `argv`. Returning from `execv` is always an error and is reported.

Build it:

```

cc -std=c11 -Wall -Wextra -Werror -O2 landlock_launch.c -o landlock-launch
./landlock-launch "$PWD/demo/allowed" "$PWD/demo/allowed/fd-probe" path "$PWD/demo/allowed/note.txt"
./landlock-launch "$PWD/demo/allowed" "$PWD/demo/allowed/fd-probe" path "$PWD/demo/protected/secret.txt" || echo 'outside

```

Line by line

- The compiler flags reject warnings and create the lesson-local launcher.
- The first launch grants filesystem rights beneath `demo/allowed`; the static probe executes and prints `allowed`.
- The second launches the same child under the same policy but asks it to open a protected sibling. Expected: `DENIED` and the shell message.

The source performs these security-sensitive steps in order:

- `landlock_create_ruleset(..., LANDLOCK_CREATE_RULESET_VERSION)` queries the running kernel ABI. Failure is reported; the launcher never silently runs unconfined.
- `supported_rights` begins with ABI-1 filesystem rights, adds cross-directory `REFER` only for ABI 2+, and adds `TRUNCATE` only for ABI 3+.
- A ruleset handles those rights. A path-beneath rule grants them only under the already-open allowed-root descriptor.
- The root descriptor is closed after the rule is added.
- `PR_SET_NO_NEW_PRIVS` is set before `landlock_restrict_self`; unprivileged callers need this promise that `exec` cannot grant new privilege.
- Restriction occurs before `execv`, so it is inherited by the child. Errors stop the launch.
- `execv` uses the exact argument vector and an explicit executable path; there is no shell parser or `PATH` search.

This proves future path access is constrained. It does not prove existing descriptors were revoked.

Exercise 3 - Reproduce and repair the pre-opened-FD bypass

`pass_fd.py` deliberately opens the protected file before launching:

```
#!/usr/bin/env python3
import os
import subprocess
import sys

with open(sys.argv[1], "rb") as stream:
    run = subprocess.run(
        ["/landlock-launch", sys.argv[2], sys.argv[3], "fd", str(stream.fileno())],
        pass_fds=(stream.fileno(),),
        check=False,
    )
    raise SystemExit(run.returncode)
```

Source, line by line

- `open` creates authority to the protected object before Landlock is installed.
- `pass_fds` deliberately clears `close-on-exec` for that descriptor in the immediate child.
- The launcher receives the allowed root, exact probe path, `fd` mode, and descriptor number as distinct `argv` elements.
- The wrapper returns the observed child status.

The repaired launcher calls `close_inherited` after using its policy descriptors and before `execv`.

`close_range(..., CLOSE_RANGE_CLOEXEC)` atomically marks every descriptor from 3 upward `close-on-exec`; an older-kernel fallback applies `FD_CLOEXEC` one descriptor at a time.

Run the repaired result:

```
python3 pass_fd.py "$PWD/demo/protected/secret.txt" "$PWD/demo/allowed" "$PWD/demo/allowed/fd-probe" || echo 'inherited d
```

Expected: the probe reports a bad descriptor and the shell prints `inherited descriptor denied`.

Intentional failure: make a temporary copy without the `close_inherited()` call and its error block, then compile it while suppressing only the expected unused-helper warning:

```
sed '/if (close_inherited() == -1)/,+3d' landlock_launch.c > unsafe_launch.c
cc -std=c11 -Wall -Wextra -Wno-unused-function -O2 unsafe_launch.c -o unsafe-launch
```

- The addressed sed range removes the four-line call/error block from the temporary source only.
- `-Wno-unused-function` permits the deliberately orphaned helper; it does not suppress other warning classes.
- Substitute `./unsafe-launch` for `./landlock-launch` in the wrapper command. The synthetic secret prints even though the path rule is active.
- Delete the temporary files, return to the shipped launcher, and confirm denial.

Landlock mediates new filesystem operations by path; reading an already-open file description is not a new path lookup.

Checkpoint and troubleshooting

```
test "$(/.landlock-launch "$PWD/demo/allowed" "$PWD/demo/allowed/fd-probe" path "$PWD/demo/allowed/note.txt")" = allowed
! ./landlock-launch "$PWD/demo/allowed" "$PWD/demo/allowed/fd-probe" path "$PWD/demo/protected/secret.txt"
! python3 pass_fd.py "$PWD/demo/protected/secret.txt" "$PWD/demo/allowed" "$PWD/demo/allowed/fd-probe"
```

- Landlock ABI: Function not implemented means the kernel lacks Landlock. Use the supported VM and record the skip; do not run the child unconfined.
- Permission denied on the allowed executable usually means the probe is outside the allowed tree or was not compiled successfully.
- A visible synthetic secret in the final command means inherited descriptors were not marked close-on-exec.
- Checkpoint: identify which assertion tests Landlock and which tests the independent descriptor-hygiene layer.

Module 05 independent lab - Filesystem guard

Implement `fs_guard.c`. The grader builds one executable and invokes these interfaces:

```
fs-guard read ALLOWED_ROOT RELATIVE_PATH
fs-guard write ALLOWED_ROOT RELATIVE_PATH CONTENT
fs-guard run ALLOWED_ROOT COMMAND [ARG...]
```

Required security properties:

- `read` prints an allowed regular file and `write` creates or truncates an allowed regular file with exact content;
- both operations resolve from an opened `ALLOWED_ROOT` descriptor in one kernel operation;
- absolute paths, `..` traversal, magic links, and every symlink are denied rather than normalized and reopened;
- `run` queries the Landlock ABI, handles only rights supported by it, grants the child filesystem access beneath `ALLOWED_ROOT`, sets `no_new_privs`, and restricts before `exec`;
- an executable inside the allowed tree still runs, while that child cannot newly open a protected sibling;
- inherited descriptors numbered 3 and above are closed on `exec`, preventing a pre-opened protected file from bypassing the pathname policy;
- malformed or unknown modes fail closed with a nonzero status.

The grader changes directory names, relative paths, contents, and synthetic canaries on every run. It compiles its observation child statically inside the allowed tree, creates a symlink to a protected sibling, passes a protected descriptor deliberately, and evaluates a read-only copy of your source.

The starter is useful but intentionally unsafe. It joins `root` and `request` as text, follows symlinks, permits traversal, launches a child without Landlock, and preserves inherited descriptors. Replace those behaviors using only the interfaces practiced in Lessons 05.01-05.03.

Build and exercise the harmless sample:

```
cc -std=c11 -Wall -Wextra -O2 fs_guard.c -o fs-guard
rm -rf sample-root
mkdir sample-root
printf 'sample\n' > sample-root/input.txt
./fs-guard read "$PWD/sample-root" input.txt
./fs-guard write "$PWD/sample-root" output.txt 'sample-output'
test "$(cat sample-root/output.txt)" = sample-output
../../lab-grade module-05
../../lab-grade module-05 --mode exam
```

Test commands, line by line

- `cc` builds exactly the submitted source and enables useful warnings.
- The scoped `rm -rf` and `mkdir` create only a lab-local sample root.
- `printf` supplies non-sensitive input.
- `read` and `write` verify the functional interface before confinement is graded.
- `test` independently checks the write effect.

- Practice grading names failed properties and points back to the relevant lesson.
- Exam grading runs the same fresh bypasses but suppresses repair-oriented references.

The lab does not provide a completed implementation. Plan separate read/write and run paths, keep the root descriptor alive only as long as needed, make every setup failure stop the command, and preserve the required order: inspect ABI, create ruleset, add rule, set `no_new_privs`, restrict, close inherited authority, then exec.

This is an unprivileged local exercise. Do not add sudo, mounts, external paths, network access, or real secrets.

Module 06 - Constrain syscalls with seccomp

Measure a real workload before writing policy, observe the operational difference between `errno` and `kill` actions, then launch a static child under a native-architecture default-deny libseccomp filter.

Play in order: 06.01, 06.02, 06.03, then module-06.

Outcomes:

- collect and interpret a syscall profile without treating one trace as universal truth;
- compare `SCMP_ACT_ERRNO` and `SCMP_ACT_KILL_PROCESS` using an observable denied call;
- reproduce an alternate-interface bypass against a narrow blacklist;
- construct a native-architecture default-deny filter and prove it is active before `exec`;
- preserve a stated workload while denying `socket`, `socket-pair`, `ptrace`, and an unlisted syscall;
- explain why syscall filtering is not pathname authorization, resource accounting, or network destination policy.

Prerequisites: Modules 01-05, Linux, `strace`, a C compiler with static `libc` development files, and `libseccomp` headers/library discoverable through `pkg-config`. All probes are local and unprivileged.

06.01 - Measure the workload before filtering

Goal

Collect the syscalls of one explicit file-copy workload, separate startup activity from application intent, and demonstrate why a profile is evidence for policy design rather than a permanent universal allowlist.

Exercise 1 - Build and run without tracing

Complete source of `workload.c`:

```
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>

int main(int argc, char **argv) {
    if (argc != 3) {
        fprintf(stderr, "usage: %s INPUT OUTPUT\n", argv[0]);
        return 2;
    }
    int input = open(argv[1], O_RDONLY);
    int output = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0600);
    if (input == -1 || output == -1) {
        perror("open");
        return 1;
    }
    char buffer[256];
    ssize_t count;
    while ((count = read(input, buffer, sizeof(buffer))) > 0)
        if (write(output, buffer, (size_t)count) != count)
            return 1;
    close(input);
    close(output);
    return count < 0;
}
```

Source, line by line

- The program requires input and output pathnames and returns usage status 2 for any other interface.
- `open` requests read authority for one path and create/truncate/write authority for the other. The kernel may implement libc `open` with `openat`.
- The fixed buffer bounds each transfer. `read` ends at zero and a negative result becomes failure.
- `write` is checked rather than assumed. Both descriptors are closed on normal completion.

```
cc -std=c11 -Wall -Wextra -Werror -O2 workload.c -o workload
printf 'measured-data\n' > input.txt
./workload input.txt output.txt
cmp input.txt output.txt
```

Line by line

- The compiler rejects warnings and writes only the lesson-local executable.

- `printf` creates synthetic input; `cmp` independently verifies exact output.
- Expected: no output and status zero. This establishes the functional baseline before tracing changes observation.

Exercise 2 - Capture counts and exact events

```
strace -f -qq -c ./workload input.txt output.txt
strace -f -qq -o trace.txt ./workload input.txt output.txt
sed -E 's/^[0-9]+ +([^\s]+).*/\1/' trace.txt | sort -u
```

Line by line

- `-f` follows children; this workload creates none, but the policy is explicit.
- `-qq` removes attach/detach chatter. `-c` reports aggregate syscall counts and timing.
- The second run writes complete events to `trace.txt` rather than mixing them with workload output.
- `sed` extracts syscall names from PID-prefixed lines; `sort -u` creates the observed set.
- Expect file operations plus dynamic-loader and process-startup calls such as `mmap`, `mprotect`, `brk`, and architecture-dependent setup. Exact names and counts vary by `libc`, architecture, kernel, and input.

The trace proves calls observed on this run. It does not prove unobserved error paths are unnecessary or that every observed syscall should be broadly allowed.

Exercise 3 - Break the incomplete profile

Trace a zero-length input and compare it with the nonempty run:

```
: > empty.txt
strace -f -qq -o empty-trace.txt ./workload empty.txt empty-output.txt
grep -c 'write(' trace.txt
grep -c 'write(' empty-trace.txt
```

Line by line

- `:` is a shell builtin that succeeds; redirection truncates `empty.txt` to zero bytes.
- The empty workload never enters its data-write body, so its observed set can omit a write that the nonempty input requires.
- `grep -c` prints counts. A zero count returns status 1, so inspect the number instead of chaining it under `set -e`.

Intentional mistake: treat only `empty-trace.txt` as the policy specification. The nonempty workload would then fail. Repair the reasoning by defining representative success and error cases first, combining their measurements, and reviewing each allowed call against the workload contract.

Checkpoint and troubleshooting

```
cmp input.txt output.txt
grep -q 'openat(' trace.txt
grep -q 'read(' trace.txt
```

- If `strace` is blocked, use the documented disposable VM; do not weaken a work host.
- A `libc` may use `openat` even though the source says `open`; policy applies to kernel ABI calls, not C function spelling.

- Checkpoint: name one observed loader syscall and one input-dependent syscall, and explain why measurement alone is not least privilege.

06.02 - Compare errno, kill, and blacklist bypass

Goal

Install one observable deny rule, compare recoverable EPERM with process termination, then bypass a single-syscall blacklist through a related kernel interface.

Exercise 1 - Build the probe and filter launcher

Complete probe source:

```
#include <errno.h>
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

int main(int argc, char **argv) {
    if (argc != 2)
        return 2;
    errno = 0;
    int result;
    if (strcmp(argv[1], "socket") == 0)
        result = socket(AF_UNIX, SOCK_STREAM, 0);
    else {
        int pair[2];
        result = socketpair(AF_UNIX, SOCK_STREAM, 0, pair);
        if (result == 0) {
            close(pair[0]);
            close(pair[1]);
        }
    }
    printf("result=%d errno=%d\n", result, errno);
    return result == -1 ? 1 : 0;
}
```

Source, line by line

- `errno` records why a failed call returned `-1`.
- `socket` requests one local `AF_UNIX` endpoint; `socketpair` requests a connected local pair without any external network.
- Successful pair descriptors are closed. The printed result makes filter behavior visible.
- A denied call returns nonzero; an allowed call returns zero.

Complete launcher source:

```

#include <errno.h>
#include <seccomp.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

int main(int argc, char **argv) {
    if (argc < 3) {
        fprintf(stderr, "usage: %s errno|kill COMMAND [ARG...]\n", argv[0]);
        return 2;
    }
    uint32_t action = strcmp(argv[1], "kill") == 0
        ? SCMP_ACT_KILL_PROCESS : SCMP_ACT_ERRNO(EPERM);
    scmp_filter_ctx context = seccomp_init(SCMP_ACT_ALLOW);
    if (!context || seccomp_rule_add(context, action, SCMP_SYS(socket), 0) < 0 ||
        seccomp_load(context) < 0) {
        fprintf(stderr, "failed to install filter\n");
        seccomp_release(context);
        return 1;
    }
    seccomp_release(context);
    execv(argv[2], &argv[2]);
    perror("execv");
    return 1;
}

```

Source, line by line

- `seccomp_init(SCMP_ACT_ALLOW)` creates a blacklist: every syscall is allowed unless a rule says otherwise.
- The selected rule either returns `EPERM` or kills the whole process when `socket` is attempted.
- Every `libseccomp` operation is checked. Failure cannot fall through to an unfiltered `execv`.
- `seccomp_load` installs the filter on the launcher; `seccomp` then persists across `exec` into the probe.
- `seccomp_release` frees userspace policy memory, not the loaded kernel filter.

```

cc -std=c11 -Wall -Wextra -Werror -O2 -static socket_probe.c -o socket-probe
cc -std=c11 -Wall -Wextra -Werror -O2 deny_socket.c -o deny-socket $(pkg-config --cflags --libs libseccomp)

```

The first binary is static so its runtime does not complicate this one-rule observation. `pkg-config` supplies the distribution's `libseccomp` flags.

Exercise 2 - Observe `errno` and `kill` actions

```

./deny-socket errno ./socket-probe socket || test "$?" -eq 1
set +e
./deny-socket kill ./socket-probe socket
status=$?
set -e
printf 'kill_status=%s\n' "$status"

```

Line by line

- `Errno` mode lets the probe continue: expect `result=-1 errno=1` and `status 1`.
- `set +e` permits the shell to observe an expected fatal signal rather than aborting a scripted session.
- `Kill` mode prevents the probe from printing its post-call result. A shell commonly reports `Bad system call`; `status` is normally 128 plus `SIGSYS`.

- `set -e` restores fail-fast behavior. The status proves termination but does not identify a complete policy.

`Errno` is useful for compatibility and diagnostics; `kill` is useful when continuing would be unsafe. Policy intent decides, not a blanket rule.

Exercise 3 - Bypass the one-name blacklist

```
./deny-socket errno ./socket-probe socketpair
```

Line by line

- The launcher denies only syscall `socket`.
- `socketpair` is a different syscall that creates a related communication primitive.
- Expected: `result=0 errno=0`. This is an intentional bypass of the blacklist, entirely local to the process.

Repair the design in Lesson 06.03 with a default-deny policy whose allowlist is derived from the stated workload. Merely adding names whenever a bypass appears tends toward an incomplete blacklist.

Checkpoint and troubleshooting

```
! ./deny-socket errno ./socket-probe socket
./deny-socket errno ./socket-probe socketpair
```

- If `seccomp.h` or `libseccomp.pc` is absent, install the documented `libseccomp-dev` and `pkg-config` packages in the disposable VM.
- If kill mode emits no probe line, that is expected: the kernel terminates at the denied syscall.
- Checkpoint: explain why both observed actions enforce the same rule but have different failure semantics, and why neither blocks `socketpair`.

06.03 - Launch with a native default-deny filter

Goal

Turn the measured workload into a native-architecture allowlist, prove filter state inside the child, and preserve file access while denying unlisted kernel interfaces.

Exercise 1 - Read the observation probe

Complete `policy_probe.c`:

```
#define _GNU_SOURCE
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <string.h>
#include <sys/ptrace.h>
#include <sys/socket.h>
#include <sys/syscall.h>
#include <unistd.h>

static void report(long result) {
    char line[96];
    int count = snprintf(line, sizeof(line), "result=%ld errno=%d\n", result, errno);
    if (write(STDOUT_FILENO, line, (size_t)count) != count)
        _exit(1);
}

int main(int argc, char **argv) {
    if (argc < 2)
        return 2;
    if (strcmp(argv[1], "status") == 0) {
        char buffer[4096] = {0};
        int fd = open("/proc/self/status", O_RDONLY);
        ssize_t count = read(fd, buffer, sizeof(buffer) - 1);
        close(fd);
        if (count < 0)
            return 1;
        char *nnp = strstr(buffer, "NoNewPrivs:");
        char *seccomp = strstr(buffer, "Seccomp:");
        dprintf(STDOUT_FILENO, "%.13s\n%.10s\n", nnp, seccomp);
        return 0;
    }
    if (strcmp(argv[1], "file") == 0 && argc == 3) {
        int fd = open(argv[2], O_RDONLY);
        char buffer[256];
        ssize_t count = read(fd, buffer, sizeof(buffer));
        return count > 0 && write(STDOUT_FILENO, buffer, (size_t)count) == count ? 0 : 1;
    }
    errno = 0;
    if (strcmp(argv[1], "socket") == 0)
        report(socket(AF_UNIX, SOCK_STREAM, 0));
    else if (strcmp(argv[1], "socketpair") == 0) {
        int pair[2];
        report(socketpair(AF_UNIX, SOCK_STREAM, 0, pair));
    } else if (strcmp(argv[1], "ptrace") == 0)
        report(ptrace(PTRACE_TRACEME, 0, NULL, NULL));
    else
        report(syscall(SYS_getppid));
    return 0;
}
```

Line by line

- `status` reads kernel-reported process state from `procfs` after `exec`; the launcher cannot substitute its own pre-filter claim.
- `file` uses normal `libc` `open`, which becomes the kernel's `openat` on this platform.
- The denial modes invoke distinct syscalls and report raw result plus `errno` through allowed output.
- The fallback uses raw `SYS_getppid`, a harmless syscall intentionally absent from the allowlist.

Read the full small source before compiling:

```
sed -n '1,240p' policy_probe.c  
cc -std=c11 -Wall -Wextra -Werror -O2 -static policy_probe.c -o policy-probe
```

- `sed` displays the complete source; no required behavior is hidden.
- The static binary makes its startup surface stable enough for this VM exercise.

Exercise 2 - Build and inspect the policy

Complete `allowlist.c`:

```

#define _GNU_SOURCE
#include <errno.h>
#include <seccomp.h>
#include <stdio.h>
#include <string.h>
#include <sys/prctl.h>
#include <unistd.h>

static int allow_name(scmp_filter_ctx context, const char *name) {
    int number = seccomp_syscall_resolve_name(name);
    return number == __NR_SCMP_ERROR ? 0 : seccomp_rule_add(context, SCMP_ACT_ALLOW, number, 0);
}

int main(int argc, char **argv) {
    if (argc < 2) {
        fprintf(stderr, "usage: %s COMMAND [ARG...]\n", argv[0]);
        return 2;
    }
    const char *allowed[] = {
        "execve", "read", "write", "close", "openat", "brk", "mmap", "mprotect",
        "munmap", "set_tid_address", "set_robust_list", "prlimit64", "readlink", "readlinkat",
        "getrandom", "rseq", "arch_prctl", "fstat", "newfstatat", "faccessat",
        "exit", "exit_group"
    };
    scmp_filter_ctx context = seccomp_init(SCMP_ACT_ERRNO(EPERM));
    uint32_t native = seccomp_arch_native();
    if (!context || native == 0 || seccomp_arch_exist(context, native) < 0) {
        fprintf(stderr, "native architecture unavailable\n");
        seccomp_release(context);
        return 1;
    }
    for (size_t index = 0; index < sizeof(allowed) / sizeof(allowed[0]); index++) {
        if (allow_name(context, allowed[index]) < 0) {
            fprintf(stderr, "cannot allow %s\n", allowed[index]);
            seccomp_release(context);
            return 1;
        }
    }
    if (prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0) < 0 || seccomp_load(context) < 0) {
        perror("install seccomp");
        seccomp_release(context);
        return 1;
    }
    seccomp_release(context);
    execv(argv[1], &argv[1]);
    perror("execv");
    return 1;
}

```

Line by line

- `SCMP_ACT_ERRNO(EPERM)` is the default, so an omitted syscall is denied rather than silently allowed.
- `seccomp_arch_native` obtains libseccomp's token for the running architecture; `seccomp_arch_exist` verifies the context contains it.
- Names are resolved for the native architecture. A name absent from that architecture is skipped, while a rule failure for a present syscall stops setup. This keeps the measured static-startup set portable between the supported arm64 and x86-64 baselines without hard-coded numbers.
- The list includes static startup, exact argv execution, proc/file reads, output, memory setup, and exit. It deliberately omits socket, socketpair, ptrace, and getppid.
- `no_new_privs` is explicit before loading. Both calls are checked, and `execv` is reachable only after a successful load.
- libseccomp generates the BPF architecture check and syscall-number comparisons; hand-coded numeric syscall tables are avoided.

Display the full source and build it:

```
sed -n '1,260p' allowlist.c
cc -std=c11 -Wall -Wextra -Werror -O2 allowlist.c -o allowlist $(pkg-config --cflags --libs libseccomp)
```

Exercise 3 - Verify effective state and denials

```
./allowlist ./policy-probe status
printf 'allowed-file\n' > note.txt
./allowlist ./policy-probe file note.txt
./allowlist ./policy-probe socket
./allowlist ./policy-probe socketpair
./allowlist ./policy-probe ptrace
./allowlist ./policy-probe unexpected
```

Line by line

- Status must show NoNewPrivs: 1 and Seccomp: 2; mode 2 is filter mode.
- The file workload succeeds because `openat`, `read`, `write`, and `close` are in scope.
- Every denial probe should report `result=-1 errno=1`, including both socket interfaces and the unlisted harmless syscall.
- The probes return normally because the default action is `errno`. This permits evidence collection.

Intentional failure: create a temporary copy that adds `"getppid"`, to the allowed-name array, rebuild it, and run `unexpected`. It returns a real parent PID, proving that default-deny still depends on a justified allowlist. Delete the temporary copy, return to the shipped list, and require `result=-1 errno=1`.

The file test is also a boundary lesson: `seccomp` allowed `openat`, so it cannot distinguish `note.txt` from another pathname. Compose it with Module 05's `openat2/Landlock` layer for pathname policy. It likewise does not meter CPU/memory or authorize network destinations.

Checkpoint and troubleshooting

```
./allowlist ./policy-probe status | grep -q 'Seccomp:[[:space:]]*2'
test "$(./allowlist ./policy-probe unexpected)" = 'result=-1 errno=1'
```

- If the static probe fails before output, compare its `strace` surface with the allowlist on this supported VM and add only justified startup calls.
- `EPERM` from `seccomp_load` usually means `no_new_privs` was not set or policy setup was reordered.
- Checkpoint: identify the functional syscalls, startup syscalls, deliberately denied syscalls, and the separate pathname assumption.

Module 06 independent lab - Default-deny syscall launcher

Implement `seccomp_guard.c`. The grader builds and invokes:

```
seccomp-guard COMMAND [ARG...]
```

The command is an exact executable path to a static local observation probe. Your launcher must:

- validate the interface and preserve the structured argv vector;
- create a libseccomp context whose default action returns EPERM;
- explicitly confirm the context's native architecture;
- allow only the calls needed for static startup, exact `execve`, status/file reads, bounded output, memory setup, and exit as practiced in Lesson 06.03;
- set `PR_SET_NO_NEW_PRIVS` and successfully load the filter before `exec`;
- fail closed if any context, architecture, syscall-resolution, rule, privilege-floor, filter-load, or `exec` step fails.

The fresh external grader compiles a static probe and checks functional execution, `/proc/self/status` evidence, an allowed file read, direct socket denial, socket-pair denial, `ptrace` denial, and denial of a harmless syscall omitted from the workload. It evaluates a read-only copy of your source.

The starter only validates argv and calls `execv`, so the workload runs but every confinement property fails. Use libseccomp names rather than architecture-specific numeric syscall constants. Do not add network access, privilege, or a kill action; `errno` results are required so the grader can observe every denial.

```
cc -std=c11 -Wall -Wextra -Werror -O2 seccomp_guard.c -o seccomp-guard $(pkg-config --cflags --libs libseccomp)
../../lab-grade module-06
../../lab-grade module-06 --mode exam
```

Test commands, line by line

- The compiler and warning flags match the guided launcher; `pkg-config` supplies only the local libseccomp build flags.
- Practice mode creates a fresh probe and names failed properties with lesson references.
- Exam mode repeats fresh behavior checks but suppresses repair-oriented references.

The lab deliberately withholds a complete implementation. Start from the ordering and justified call set you measured and explained in Lesson 06.03. Seccomp is one runtime layer: this lab does not claim pathname policy, resource accounting, network destination authorization, or protection from already-open descriptors.

Module 07 - Bound CPU, memory, and process creation

Use cgroup v2 through the ordinary user's delegated systemd manager. Observe effective controller files from inside workloads, trigger bounded pressure, and prove that transient service collection removes the complete process tree.

Play in order: 07.01, 07.02, 07.03, then module-07.

Outcomes:

- connect a systemd user unit to its effective cgroup v2 path;
- interpret `cpu.max`, `cpu.stat`, `memory.max`, `memory.events`, `pids.max`, and `pids.events`;
- distinguish throttling, allocation failure/OOM, and PID exhaustion;
- apply limits to a process tree rather than a single PID;
- use bounded transient units whose cleanup is synchronous and ownership-verifiable.

Prerequisites: Modules 01-06, unified cgroup v2, a running delegated systemd user manager, and the `cpu`, `memory`, and `pids` controllers. No root access is used by the exercises.

07.01 - Observe an effective CPU quota

Goal

Launch a bounded transient user service, find its actual cgroup from inside the workload, and distinguish a configured percentage from the kernel's quota/period representation.

Complete `inspect_cpu.py` reads `0: :PATH` from `/proc/self/cgroup`, anchors that path below `/sys/fs/cgroup`, samples `cpu.stat`, burns CPU for one second, and emits JSON. `time.monotonic()` makes elapsed time independent of wall-clock changes; the loop counter is comparative evidence, not a portable benchmark.

```
python3 inspect_cpu.py
systemd-run --user --wait --pipe --collect --quiet \
  --property=CPUQuota=50% -- python3 "$PWD/inspect_cpu.py"
```

Line by line

- The first run shows the shell's inherited cgroup and commonly reports `cpu.max` as `max 100000`.
- `--user` confines management to the ordinary user's delegated manager.
- `--wait` waits for workload status, `--pipe` preserves output, and `--collect` removes the transient unit and cgroup.
- `CPUQuota=50%` becomes `50000 100000`: 50,000 microseconds of CPU time per 100,000-microsecond period.
- `--` ends `systemd-run` options; Python and its script remain separate `argv` elements.

Intentional failure: omit `CPUQuota`. The workload still succeeds, but `cpu.max` reports `max 100000`, proving no CPU ceiling was installed. Restore the property and require `50000 100000`.

Checkpoint and troubleshooting

```
systemd-run --user --wait --pipe --collect --quiet --property=CPUQuota=50% -- python3 "$PWD/inspect_cpu.py" | grep -q '50000 100000'
```

- If the user manager is unavailable, use the documented VM login session; do not switch to system units or `sudo`.
- Quota throttles aggregate CPU time for the cgroup tree; it does not guarantee wall-clock latency or fair scheduling.

07.02 - Bound memory and observe OOM

Goal

Apply a memory ceiling to an entire transient service and observe a synthetic allocator crossing it without risking the VM.

`memory_hog.py` converts one argument to a count, retains one-megabyte bytearray blocks so pages stay charged, prints bounded progress every eight blocks, and sleeps only after allocation completes.

```
python3 memory_hog.py 16
set +e
systemd-run --user --wait --pipe --collect --quiet \
  --property=MemoryMax=33554432 --property=MemorySwapMax=0 \
  -- python3 "$PWD/memory_hog.py" 96
status=$?
set -e
printf 'bounded_status=%s\n' "$status"
```

Line by line

- The 16 MiB baseline should complete normally.
- `MemoryMax=33554432` is an exact 32 MiB cgroup ceiling; `MemorySwapMax=0` prevents the exercise from shifting pressure into swap.
- The 96 MiB request is synthetic and bounded. The kernel kills the charged process tree before it reaches that size.
- `set +e` permits observation of the expected failure; the saved nonzero status is restored to visible evidence before fail-fast mode returns.
- `--collect` removes the failed unit and its cgroup after systemd reports memory peak and service result.

Intentional failure: run the 96 MiB request without `MemoryMax`. It completes and proves observation alone is not enforcement. Restore both memory properties.

Checkpoint and troubleshooting

```
! systemd-run --user --wait --pipe --collect --quiet --property=MemoryMax=33554432 --property=MemorySwapMax=0 -- python3 'memory_hog.py 96'
```

- A nonzero result is expected; do not increase the allocation or change host overcommit settings.
- `memory.max` bounds charged memory, while OOM outcome can vary with interpreter startup and kernel accounting. The stable property is containment below the configured ceiling.

07.03 - Bound process-tree growth and collect it

Goal

Apply a PID ceiling to a process tree, observe a rejected fork, and prove collection removes the transient cgroup.

`fork_pressure.py` resolves its own cgroup, attempts a bounded number of forks, makes each child sleep briefly, reaps every created child, and prints `pids.max` plus `pids.events`. Children use `_exit` so they do not rerun parent cleanup.

```
systemd-run --user --wait --pipe --collect --quiet \
  --property=TasksMax=12 -- python3 "$PWD/fork_pressure.py" 64
systemctl --user list-units --all 'run-*.service'
```

Line by line

- `TasksMax=12` writes 12 to `pids.max` for the whole service tree, including the parent.
- The loop asks for at most 64 children; fork fails before that when the cgroup reaches its ceiling.
- Reaping prevents zombies from obscuring the count.
- `pids.events` contains a positive max counter after a rejected creation.
- The listing should not contain the collected transient unit. Collection, not killing only one PID, is the cleanup property.

Intentional failure: omit `TasksMax`; all 64 bounded children can be created and `pids.max` is max. Restore the ceiling and require fewer than 64 children plus a positive max event.

Checkpoint and troubleshooting

```
systemd-run --user --wait --pipe --collect --quiet --property=TasksMax=12 -- python3 "$PWD/fork_pressure.py" 64 | grep -q
```

- Never substitute an unbounded fork loop. The fixed upper bound and short sleep make failure recoverable.
- A PID controller bounds task creation; it does not limit CPU or memory, so compose all required controllers.

Module 07 independent lab - Bounded transient runner

Implement `resource_runner.py`. The grader invokes:

```
python3 resource_runner.py SPEC.json RESULT.json
```

The specification contains an absolute-path command string array plus integer `memory_max`, `tasks_max`, and `cpu_percent` fields. Your runner must validate conservative bounds, launch the exact argv in a uniquely named transient systemd user service, and set `MemoryMax`, `MemorySwapMax=0`, `TasksMax`, and `CPUQuota`. Use `--wait`, `--pipe`, and `--collect` so completion is synchronous and the cgroup is removed.

The result must be one JSON object containing integer `status` and string `stdout/stderr`. Preserve nonzero workload status. If your own timeout or setup fails, stop the exact owned unit before returning. Never use a shell, `sudo`, a system unit, or an unbounded resource value.

The starter runs argv directly, so function succeeds but every effective-limit property fails. The fresh external grader checks the child's real `cpu.max`, `memory.max`, `memory.swap.max`, and `pids.max`, an argv literal, bounded process creation, invalid-spec denial, failure propagation, and post-run unit collection.

```
python3 resource_runner.py sample-spec.json result.json
python3 -m json.tool result.json
../../lab-grade module-07
../../lab-grade module-07 --mode exam
```

Test commands, line by line

- The first command exercises the two-file interface; the checked-in sample uses only `/bin/echo`.
- `json.tool` proves the result is valid JSON rather than trusting display text.
- Practice and exam modes run the same kernel properties with different hint detail.

The lab withholds a complete implementation. Reuse the structured subprocess, unit naming, property ordering, timeout cleanup, and effective-state observations practiced in the guided lessons.

North Echo field manual - Module 08

This chapter is self-contained for the guided networking exercises and independent lab. Every service is synthetic and loopback-only; no public or LAN endpoint is used.

Module overview

Module 08 - Isolate the network and mediate egress

Remove the workload's inherited IP network, then expose only a narrow HTTP capability through a filesystem Unix socket. The broker uses an explicit synthetic name-to-loopback map, binds requests to a run identity, and authorizes every redirect again.

Play in order: 08.01, 08.02, 08.03, then module-08.

Outcomes:

- prove a new network namespace starts with loopback down and cannot reach a service on host loopback;
- distinguish removing direct IP connectivity from authorizing one mediated operation;
- carry a structured request over AF_UNIX without restoring an IP route;
- authorize a synthetic hostname, resolved address, port, path shape, and run identity;
- reauthorize redirects before contacting their destinations;
- bound request and response sizes and cleanly remove the broker socket.

All services are synthetic and bind only to 127.0.0.1. The exercises do not create veth devices, routes, firewall rules, DNS traffic, or public requests. Prerequisites are Modules 01-07 and Linux support for unprivileged user plus network namespaces.

Lesson 08.01

08.01 - Remove the inherited IP network

Goal

Prove that a new network namespace has different kernel network state and cannot reach a synthetic service bound to the parent namespace's loopback address.

Exercise 1 - Inspect the boundary

Read both complete programs before running them:

```
sed -n '1,240p' local_http.py
sed -n '1,240p' connect_probe.py
```

Source, line by line

- `local_http.py` refuses a pre-existing readiness path, creates one `HTTPServer` on `127.0.0.1`, then writes `ready`. It exits after one request and removes the readiness file in `finally`. Its fixed response is synthetic and its disabled request log avoids host-identifying noise.
- `connect_probe.py` calls `readlink` on `/proc/self/ns/net`, so the namespace identity comes from the process being tested.
- `socket.create_connection` makes exactly one bounded attempt to `127.0.0.1:PORT`; its one-second timeout prevents a failed path from hanging.
- The probe records either `connected: true` or the exception type and returns nonzero on denial. It does not treat an error message alone as proof.

Choose a lesson-local high port, start the one-request service, and connect from the current namespace:

```
PORT=$((42000 + $$ % 10000))
READY="$PWD/server.ready"
python3 local_http.py "$PORT" "$READY" &
server_pid=$!
trap 'kill "$server_pid" 2>/dev/null || true; wait "$server_pid" 2>/dev/null || true; rm -f "$READY"' EXIT
for attempt in 1 2 3 4 5; do test -f "$READY" && break; sleep 0.1; done
test -f "$READY"
python3 connect_probe.py "$PORT"
wait "$server_pid"
trap - EXIT
```

Line by line

- The arithmetic selects a high synthetic port without privileged binding. A collision is harmless and visible; choose another value if needed.
- `&` launches only the local service; `$!` captures its exact PID for bounded cleanup.
- `trap` names that exact PID and lesson-local readiness path. It does not use a process-name pattern.
- The bounded loop waits for the server-created readiness evidence before connecting.

- The probe should print its network-namespace link, connected: true, and exit zero. wait proves the one-request server exited.

Exercise 2 - Enter an empty network namespace

Start another one-request service, then run the probe after unshare creates user and network namespaces:

```
python3 local_http.py "$PORT" "$READY" &
server_pid=$!
trap 'kill "$server_pid" 2>/dev/null || true; wait "$server_pid" 2>/dev/null || true; rm -f "$READY"' EXIT
for attempt in 1 2 3 4 5; do test -f "$READY" && break; sleep 0.1; done
test -f "$READY"
set +e
unshare --user --map-root-user --net python3 connect_probe.py "$PORT"
isolated_status=$?
set -e
printf 'isolated_status=%s\n' "$isolated_status"
kill "$server_pid"
wait "$server_pid" 2>/dev/null || true
trap - EXIT
```

Line by line

- --user --map-root-user gives the calling user namespace-local capability to create and configure its new network namespace; it does not grant host root.
- --net creates a new network namespace. Its loopback device starts down and it has no inherited host interfaces or routes.
- set +e permits the expected failed connection to be observed. The saved status must be nonzero.
- Killing the exact still-waiting server proves the isolated request never reached it.

Expected output includes a different net : [NUMBER], connected: false, and usually Network is unreachable. Error text varies; the stable facts are the distinct namespace and failed connection.

Exercise 3 - Repair loopback, not host reachability

Intentional mistake: assume that enabling loopback reconnects the namespace to the parent's 127.0.0.1.

```
python3 local_http.py "$PORT" "$READY" &
server_pid=$!
trap 'kill "$server_pid" 2>/dev/null || true; wait "$server_pid" 2>/dev/null || true; rm -f "$READY"' EXIT
for attempt in 1 2 3 4 5; do test -f "$READY" && break; sleep 0.1; done
test -f "$READY"
set +e
unshare --user --map-root-user --net sh -c 'ip link set lo up; ip -brief address show lo; python3 connect_probe.py "$1" &
loopback_status=$?
set -e
printf 'private_loopback_status=%s\n' "$loopback_status"
kill "$server_pid"
wait "$server_pid" 2>/dev/null || true
trap - EXIT
```

Line by line

- ip link set lo up changes only the new namespace's loopback device.
- The quoted script receives the port as \$1; no lesson value is interpolated as shell code.

- 127.0.0.1 now refers to the new namespace itself, not the parent's loopback service, so the connection remains denied (commonly Connection refused).
- The repair is conceptual: use a non-IP mediation channel in Lesson 08.02 instead of adding a veth path that broadens connectivity.

Checkpoint and troubleshooting

```
test "$isolated_status" -ne 0
test "$loopback_status" -ne 0
```

- If unshare reports Operation not permitted, use the documented disposable VM and its narrow user-namespaces setup; do not weaken a work host.
- If the first connection fails, select another high port and ensure no stale lesson process remains.
- Checkpoint: explain why both namespaces can have an interface named lo while their 127.0.0.1 services remain disjoint.

Complete guided source listings

``local_http.py``

```
#!/usr/bin/env python3
"""One bounded synthetic HTTP response on loopback."""

from http.server import BaseHTTPRequestHandler, HTTPServer
from pathlib import Path
import sys

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        body = b"synthetic-loopback-service\n"
        self.send_response(200)
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)

    def log_message(self, _format, *_arguments):
        pass

if len(sys.argv) != 3:
    raise SystemExit(f"usage: {sys.argv[0]} PORT READY_FILE")
ready = Path(sys.argv[2])
if ready.exists() or ready.is_symlink():
    raise SystemExit("ready path already exists")
server = HTTPServer(("127.0.0.1", int(sys.argv[1])), Handler)
ready.write_text("ready\n", encoding="utf-8")
try:
    server.handle_request()
finally:
    server.server_close()
    ready.unlink(missing_ok=True)
```

``connect_probe.py``

```
#!/usr/bin/env python3
"""Attempt one TCP connection and report the network namespace identity."""

import json
import os
import socket
import sys

if len(sys.argv) != 2:
    raise SystemExit(f"usage: {sys.argv[0]} PORT")

result = {
    "network_namespace": os.readlink("/proc/self/ns/net"),
    "connected": False,
}
try:
    with socket.create_connection(("127.0.0.1", int(sys.argv[1])), timeout=1) as stream:
        stream.sendall(b"GET / HTTP/1.0\r\nHost: synthetic.test\r\n\r\n")
        result["connected"] = b"synthetic-loopback-service" in stream.recv(4096)
except OSError as error:
    result["error"] = f"{type(error).__name__}: {error}"
print(json.dumps(result, sort_keys=True))
raise SystemExit(0 if result["connected"] else 1)
```

Lesson 08.02

08.02 - Reach one service through a Unix-socket broker

Goal

Keep the workload's IP network empty while giving it one structured operation through a filesystem AF_UNIX socket.

Exercise 1 - Read the three complete components

```
sed -n '1,240p' synthetic_http.py
sed -n '1,280p' one_host_broker.py
sed -n '1,240p' broker_client.py
```

Source, line by line

- `synthetic_http.py` serves a fixed response on host loopback and writes a lesson-local readiness file after binding. It is the only IP service.
- `one_host_broker.py` refuses an existing socket path, binds AF_UNIX, sets mode 0600, and accepts one client.
- The broker requires an exact JSON object containing the allowed run, host, port, and /ok path. Extra fields fail because dictionary equality is exact.
- Only after authorization does the broker connect to literal 127.0.0.1. The requested synthetic name becomes the HTTP Host header, not a DNS lookup.
- The broker bounds its input line and upstream body, returns one JSON line, closes descriptors, and unlinks its socket in finally.
- `broker_client.py` preserves typed fields in JSON, uses AF_UNIX, half-closes its write side, and reads one response line.

Exercise 2 - Compare direct and mediated paths

Launch exact owned processes with a cleanup trap:

```
PORT=$((43000 + $$ % 9000))
RUN_ID="lesson-$$"
SOCKET="$PWD/broker.sock"
READY="$PWD/service.ready"
python3 synthetic_http.py "$PORT" "$READY" & service_pid=$!
python3 one_host_broker.py "$SOCKET" "$RUN_ID" allowed.test "$PORT" & broker_pid=$!
trap 'kill "$broker_pid" "$service_pid" 2>/dev/null || true; wait "$broker_pid" "$service_pid" 2>/dev/null || true; rm -f
for attempt in 1 2 3 4 5; do test -S "$SOCKET" && test -f "$READY" && break; sleep 0.1; done
test -S "$SOCKET"
test -f "$READY"
```

Line by line

- Both background PIDs come from processes launched in this shell and are saved immediately.
- The socket is placed in the disposable lesson workspace, not a shared system directory.

- The bounded readiness loop checks the actual socket type; it does not sleep indefinitely.
- The trap removes only exact PIDs and the exact lesson path.

First prove direct IP still fails, then use the Unix socket:

```
set +e
unshare --user --map-root-user --net python3 -c 'import socket,sys; socket.create_connection(("127.0.0.1",int(sys.argv[1])))'
direct_status=$?
set -e
unshare --user --map-root-user --net python3 broker_client.py "$SOCKET" "$RUN_ID" allowed.test "$PORT" /ok
wait "$broker_pid"
kill "$service_pid"
wait "$service_pid" 2>/dev/null || true
trap - EXIT
```

Line by line

- The direct probe runs inside a new empty network namespace and must return nonzero.
- The second unshare creates another empty network namespace, but Unix sockets are filesystem objects and remain reachable through the shared workspace mount.
- The broker, outside the network namespace, performs the one authorized loopback request and returns approved-through-broker.
- The one-request broker exits and removes its socket. The exact service PID is then stopped and reaped.

The workload did not gain a route, interface, DNS resolver, or general TCP proxy. It gained one operation-shaped capability.

Exercise 3 - Break and repair request binding

Repeat the launch block, then intentionally send the wrong run ID:

```
unshare --user --map-root-user --net python3 broker_client.py "$SOCKET" wrong-run allowed.test "$PORT" /ok
```

Expected JSON has "ok": false and request is not authorized; the upstream service receives nothing. The mistake is treating access to the socket pathname as sufficient authority. Repair it by sending the exact \$RUN_ID and retain both checks in later policy.

Checkpoint and troubleshooting

```
test "$direct_status" -ne 0
test ! -e "$SOCKET"
```

- If the socket path already exists, remove it only after proving it is the exact lesson path and no owned broker is running.
- If the client gets Connection refused, inspect the bounded readiness loop and exact saved PID.
- Checkpoint: explain why an AF_UNIX connection crosses this network-namespaces boundary without restoring any IP destination authority.

Complete guided source listings

`synthetic\_http.py`

```
#!/usr/bin/env python3
"""Serve synthetic content on host loopback until interrupted."""

from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from pathlib import Path
import sys

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        body = b"approved-through-broker\n"
        self.send_response(200)
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)

    def log_message(self, _format, *_arguments):
        pass

if len(sys.argv) != 3:
    raise SystemExit(f"usage: {sys.argv[0]} PORT READY_FILE")
ready = Path(sys.argv[2])
if ready.exists() or ready.is_symlink():
    raise SystemExit("ready path already exists")
server = ThreadingHTTPServer(("127.0.0.1", int(sys.argv[1])), Handler)
ready.write_text("ready\n", encoding="utf-8")
try:
    server.serve_forever()
finally:
    server.server_close()
    ready.unlink(missing_ok=True)
```

`one_host_broker.py`

```

#!/usr/bin/env python3
"""A one-request Unix-socket broker for one exact synthetic destination."""

import http.client
import json
import os
from pathlib import Path
import socket
import sys

if len(sys.argv) != 5:
    raise SystemExit(f"usage: {sys.argv[0]} SOCKET RUN_ID HOST PORT")
socket_path, allowed_run, allowed_host, port_text = sys.argv[1:]
allowed_port = int(port_text)
path = Path(socket_path)
if path.exists():
    raise SystemExit("socket path already exists")

listener = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
listener.bind(socket_path)
os.chmod(socket_path, 0o600)
listener.listen(1)
try:
    peer, _ = listener.accept()
    with peer:
        request = json.loads(peer.makefile("rb").readline(8192))
        if request != {"run_id": allowed_run, "host": allowed_host, "port": allowed_port, "path": "/ok"}:
            response = {"ok": False, "error": "request is not authorized"}
        else:
            connection = http.client.HTTPConnection("127.0.0.1", allowed_port, timeout=2)
            connection.request("GET", "/ok", headers={"Host": allowed_host})
            upstream = connection.getresponse()
            body = upstream.read(65536).decode("utf-8", "replace")
            connection.close()
            response = {"ok": True, "status": upstream.status, "body": body}
        peer.sendall(json.dumps(response, sort_keys=True).encode() + b"\n")
finally:
    listener.close()
    path.unlink(missing_ok=True)

```

`broker_client.py`

```

#!/usr/bin/env python3
"""Send one structured request to a filesystem Unix socket."""

import json
import socket
import sys

if len(sys.argv) != 6:
    raise SystemExit(f"usage: {sys.argv[0]} SOCKET RUN_ID HOST PORT PATH")
request = {
    "run_id": sys.argv[2],
    "host": sys.argv[3],
    "port": int(sys.argv[4]),
    "path": sys.argv[5],
}
with socket.socket(socket.AF_UNIX, socket.SOCK_STREAM) as stream:
    stream.connect(sys.argv[1])
    stream.sendall(json.dumps(request, sort_keys=True).encode() + b"\n")
    stream.shutdown(socket.SHUT_WR)
    print(stream.makefile("r", encoding="utf-8").readline(), end="")

```

Lesson 08.03

08.03 - Reauthorize names, ports, redirects, and runs

Goal

Replace one exact request with a reusable fail-closed broker that validates policy once, authorizes every hop, and never performs live DNS.

Exercise 1 - Read the complete policy engine

```
sed -n '1,260p' egress_broker.py
sed -n '1,240p' redirect_services.py
sed -n '1,200p' broker_client.py
```

Broker, block by block

- `load_policy` accepts exactly `run_id` and `destinations`. Every synthetic hostname maps to one literal IPv4 loopback address and a nonempty set of ports. Non-loopback policy is rejected before listening.
- `authorize` compares the run ID, parses only credential-free `http` URLs, rejects fragments and malformed authorities, then checks the hostname and port against the policy map. It builds an origin-form path and a controlled `Host` header.
- `fetch` accepts exactly four request fields. It authorizes the initial URL and repeats `authorize` after every redirect. It limits redirects, connection time, and response bytes.
- `receive_line` permits one bounded newline-terminated JSON document. This prevents an unbounded stream from becoming broker memory authority.
- `serve` refuses a pre-existing path, creates a mode-0600 Unix socket, handles each request independently, converts expected failures into `ok: false`, and unlinks its socket in `finally`.
- Signal handlers request a clean loop exit. Setup or policy failure returns nonzero and never falls through to a permissive mode.

`redirect_services.py` creates an allowed service and a protected service, both on loopback, then writes a readiness file. `/same` redirects within the allowed origin; `/escape` redirects to the protected synthetic hostname. The services contain no public endpoint or real secret.

Exercise 2 - Build a synthetic resolution policy

Choose two ports and write the only allowed name-to-address decision:

```

ALLOWED_PORT=$((44000 + $$ % 4000))
PROTECTED_PORT=$((48000 + $$ % 4000))
RUN_ID="redirect-$$"
SOCKET="$PWD/policy-broker.sock"
READY="$PWD/services.ready"
cat > policy.json <<EOF
{
  "run_id": "$RUN_ID",
  "destinations": {
    "allowed.test": {"address": "127.0.0.1", "ports": [$ALLOWED_PORT]}
  }
}
EOF
python3 -m json.tool policy.json

```

Line by line

- The two ports are distinct high, synthetic endpoints.
- The here-document expands only lesson-controlled shell variables into local JSON.
- `allowed.test` is not sent to DNS. The policy's literal loopback address is the resolver decision the broker will authorize.
- `json.tool` checks syntax before the broker consumes the file.

Launch the services and broker with exact cleanup ownership:

```

python3 redirect_services.py "$ALLOWED_PORT" "$PROTECTED_PORT" "$READY" & service_pid=$!
python3 egress_broker.py policy.json "$SOCKET" & broker_pid=$!
trap 'kill "$broker_pid" "$service_pid" 2>/dev/null || true; wait "$broker_pid" "$service_pid" 2>/dev/null || true; rm -f
for attempt in 1 2 3 4 5; do test -S "$SOCKET" && test -f "$READY" && break; sleep 0.1; done
test "$(stat -c '%a' "$SOCKET")" = 600
test -f "$READY"

```

The mode check verifies an effective filesystem property rather than trusting the `chmod` call.

Exercise 3 - Follow one redirect and deny an escape

```

unshare --user --map-root-user --net python3 broker_client.py "$SOCKET" "$RUN_ID" allowed.test "$ALLOWED_PORT" /same
unshare --user --map-root-user --net python3 broker_client.py "$SOCKET" "$RUN_ID" allowed.test "$ALLOWED_PORT" /escape
unshare --user --map-root-user --net python3 broker_client.py "$SOCKET" wrong-run allowed.test "$ALLOWED_PORT" /ok

```

Line by line

- `/same` produces one relative redirect. `urljoin` keeps the allowed origin, the next loop reauthorizes it, and the result contains `allowed-final` plus redirects: `1`.
- `/escape` is the intentional attack. The first origin is allowed, but the absolute redirect names `blocked.test` on the protected port. Reauthorization denies it before connection; the protected body is absent.
- The `wrong-run` request uses an otherwise valid destination but fails before network access.

Intentional mistake: temporarily add `blocked.test` and `$PROTECTED_PORT` to policy. The escape succeeds, proving that a redirect-safe algorithm cannot repair an overbroad policy. Restore the one-destination policy and require `ok : false`.

Stop and verify cleanup:

```
kill "$broker_pid" "$service_pid"
wait "$broker_pid" "$service_pid" 2>/dev/null || true
trap - EXIT
test ! -e "$SOCKET"
```

Checkpoint and troubleshooting

```
python3 egress_broker.py policy.json "$PWD/check.sock" & check_pid=$!
for attempt in 1 2 3 4 5; do test -S "$PWD/check.sock" && break; sleep 0.1; done
kill "$check_pid"
wait "$check_pid"
test ! -e "$PWD/check.sock"
```

- If a policy hostname contains uppercase characters, normalize the policy authoring process; the broker intentionally rejects ambiguous casing.
- If a redirect reports destination is not authorized, inspect the new hostname and effective port. Do not allow it merely to silence the failure.
- If termination leaves a path, confirm you signaled the exact broker PID and that the process exited normally.
- Checkpoint: identify the authorization decision made before the first connection and the same decision made again before a redirect connection.

Complete guided source listings

`egress\_broker.py`

```

#!/usr/bin/env python3
"""Policy-bound loopback HTTP broker over a Unix-domain socket."""

from __future__ import annotations

import http.client
import ipaddress
import json
import os
from pathlib import Path
import signal
import socket
import sys
from urllib.parse import urljoin, urlsplit

MAX_LINE = 8192
MAX_BODY = 65536
MAX_REDIRECTS = 4
stopping = False

class Denied(Exception):
    pass

def load_policy(path: Path) -> dict:
    raw = json.loads(path.read_text(encoding="utf-8"))
    if set(raw) != {"run_id", "destinations"} or not isinstance(raw["run_id"], str):
        raise Denied("invalid policy shape")
    if not raw["run_id"] or len(raw["run_id"]) > 128 or not isinstance(raw["destinations"], dict):
        raise Denied("invalid policy values")
    destinations = {}
    for host, rule in raw["destinations"].items():
        if not isinstance(host, str) or host != host.lower() or not host or len(host) > 253:
            raise Denied("invalid policy hostname")
        if not isinstance(rule, dict) or set(rule) != {"address", "ports"}:
            raise Denied("invalid destination rule")
        address = ipaddress.ip_address(rule["address"])
        if address.version != 4 or not address.is_loopback:
            raise Denied("only IPv4 loopback destinations are permitted")
        ports = rule["ports"]
        if not isinstance(ports, list) or not ports or any(type(p) is not int or not 1 <= p <= 65535 for p in ports):
            raise Denied("invalid destination ports")
        destinations[host] = {"address": str(address), "ports": frozenset(ports)}
    if not destinations:
        raise Denied("policy has no destinations")
    return {"run_id": raw["run_id"], "destinations": destinations}

def authorize(policy: dict, run_id: object, url: str) -> tuple[str, str, int, str]:
    if run_id != policy["run_id"]:
        raise Denied("run identity is not authorized")
    parsed = urlsplit(url)
    if parsed.scheme != "http" or parsed.username is not None or parsed.password is not None:
        raise Denied("only credential-free HTTP URLs are permitted")
    if parsed.fragment or not parsed.hostname or parsed.hostname != parsed.hostname.lower():
        raise Denied("invalid destination URL")
    try:
        port = parsed.port or 80
    except ValueError as error:
        raise Denied("invalid destination port") from error
    rule = policy["destinations"].get(parsed.hostname)
    if rule is None or port not in rule["ports"]:
        raise Denied("destination is not authorized")
    request_path = parsed.path or "/"
    if not request_path.startswith("/") or request_path.startswith("//"):
        raise Denied("invalid request path")
    if parsed.query:
        request_path += "?" + parsed.query
    authority = parsed.hostname if port == 80 else f"{parsed.hostname}:{port}"
    return rule["address"], authority, port, request_path

```

```

def fetch(policy: dict, request: dict) -> dict:
    if not isinstance(request, dict) or set(request) != {"run_id", "host", "port", "path"}:
        raise Denied("invalid request shape")
    host, port, path = request["host"], request["port"], request["path"]
    if not isinstance(host, str) or type(port) is not int or not isinstance(path, str):
        raise Denied("invalid request values")
    url = f"http://{host}:{port}{path}"
    for redirects in range(MAX_REDIRECTS + 1):
        address, authority, port, request_path = authorize(policy, request["run_id"], url)
        connection = http.client.HTTPConnection(address, port, timeout=2)
        try:
            connection.request("GET", request_path, headers={"Host": authority, "Connection": "close"})
            response = connection.getresponse()
            body = response.read(MAX_BODY + 1)
            if len(body) > MAX_BODY:
                raise Denied("response body exceeds limit")
            location = response.getheader("Location")
            status = response.status
        finally:
            connection.close()
        if status not in {301, 302, 303, 307, 308}:
            return {"ok": True, "status": status, "body": body.decode("utf-8", "replace"), "redirects": redirects}
        if not location:
            raise Denied("redirect has no location")
        url = urljoin(url, location)
    raise Denied("redirect limit exceeded")

def receive_line(peer: socket.socket) -> bytes:
    data = bytearray()
    while b"\n" not in data and len(data) <= MAX_LINE:
        chunk = peer.recv(min(1024, MAX_LINE + 1 - len(data)))
        if not chunk:
            break
        data.extend(chunk)
    if len(data) > MAX_LINE or not data.endswith(b"\n"):
        raise Denied("request must be one bounded line")
    return bytes(data)

def serve(policy: dict, socket_path: Path) -> None:
    if socket_path.exists() or socket_path.is_symlink():
        raise Denied("socket path already exists")
    listener = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
    listener.bind(str(socket_path))
    os.chmod(socket_path, 0o600)
    listener.listen(8)
    listener.settimeout(0.2)
    try:
        while not stopping:
            try:
                peer, _ = listener.accept()
            except TimeoutError:
                continue
            with peer:
                try:
                    request = json.loads(receive_line(peer))
                    response = fetch(policy, request)
                except (Denied, json.JSONDecodeError, UnicodeError, OSError, http.client.HTTPException) as error:
                    response = {"ok": False, "error": str(error)}
                peer.sendall(json.dumps(response, sort_keys=True).encode() + b"\n")
    finally:
        listener.close()
        socket_path.unlink(missing_ok=True)

def stop(_signum, _frame):
    global stopping
    stopping = True

def main() -> int:
    if len(sys.argv) != 3:

```

```

        print(f"usage: {sys.argv[0]} POLICY.json SOCKET", file=sys.stderr)
        return 2
    signal.signal(signal.SIGTERM, stop)
    signal.signal(signal.SIGINT, stop)
    try:
        serve(load_policy(Path(sys.argv[1])), Path(sys.argv[2]))
    except (OSError, ValueError, json.JSONDecodeError, Denied) as error:
        print(f"broker setup denied: {error}", file=sys.stderr)
        return 1
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

``redirect_services.py``

```

#!/usr/bin/env python3
"""Synthetic allowed and protected HTTP services for redirect exercises."""

from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from pathlib import Path
import sys
import threading

if len(sys.argv) != 4:
    raise SystemExit(f"usage: {sys.argv[0]} ALLOWED_PORT PROTECTED_PORT READY_FILE")
allowed_port, protected_port = map(int, sys.argv[1:3])
ready = Path(sys.argv[3])
if ready.exists() or ready.is_symlink():
    raise SystemExit("ready path already exists")

class Allowed(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == "/same":
            self.send_response(302)
            self.send_header("Location", "/ok")
            self.end_headers()
        elif self.path == "/escape":
            self.send_response(302)
            self.send_header("Location", f"http://blocked.test:{protected_port}/secret")
            self.end_headers()
        else:
            body = b"allowed-final\n"
            self.send_response(200)
            self.send_header("Content-Length", str(len(body)))
            self.end_headers()
            self.wfile.write(body)

    def log_message(self, _format, *_arguments):
        pass

class Protected(BaseHTTPRequestHandler):
    def do_GET(self):
        body = b"protected-service-must-not-be-read\n"
        self.send_response(200)
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)

    def log_message(self, _format, *_arguments):
        pass

protected = ThreadingHTTPServer(("127.0.0.1", protected_port), Protected)
threading.Thread(target=protected.serve_forever, daemon=True).start()
allowed = ThreadingHTTPServer(("127.0.0.1", allowed_port), Allowed)
ready.write_text("ready\n", encoding="utf-8")
try:
    allowed.serve_forever()
finally:
    allowed.server_close()
    protected.shutdown()
    protected.server_close()
    ready.unlink(missing_ok=True)

```

`broker\_client.py`

```
#!/usr/bin/env python3
"""Send one JSON request to the lesson broker."""

import json
import socket
import sys

if len(sys.argv) != 6:
    raise SystemExit(f"usage: {sys.argv[0]} SOCKET RUN_ID HOST PORT PATH")
request = {"run_id": sys.argv[2], "host": sys.argv[3], "port": int(sys.argv[4]), "path": sys.argv[5]}
with socket.socket(socket.AF_UNIX, socket.SOCK_STREAM) as stream:
    stream.connect(sys.argv[1])
    stream.sendall(json.dumps(request).encode() + b"\n")
    stream.shutdown(socket.SHUT_WR)
    print(stream.makefile("r", encoding="utf-8").readline(), end="")
```

Independent lab

Module 08 independent lab - Policy-bound egress broker

Implement `egress_broker.py`. The grader invokes:

```
python3 egress_broker.py POLICY.json SOCKET
```

The broker must validate the complete policy before creating a mode-0600 filesystem Unix socket. Policy contains an exact `run_id` and a map from synthetic lowercase hostnames to a literal address plus allowed ports. Permit only IPv4 loopback addresses; never perform live DNS or make a non-loopback request.

Each client connection carries one bounded newline-terminated JSON object:

```
{"run_id": "...", "host": "allowed.test", "port": 45123, "path": "/ok"}
```

Require exactly those fields. Authorize the run, hostname, mapped address, and port before each connection. Support bounded HTTP redirects, but parse and reauthorize every new URL before contacting it. Reject credentials, non-HTTP schemes, malformed ports, ambiguous paths, oversized input or output, excessive redirects, extra authority-bearing fields, and upstream failures. Responses are one JSON line: success includes `ok`, `status`, `body`, and `redirects`; denial includes `ok: false` and an error. Clean termination must remove the exact socket.

The fresh external grader starts randomized allowed and protected loopback services. It checks approved access, run binding, hostname and port denial, an allowed relative redirect, denied name and literal-address redirect escapes before protected contact, strict request shape, non-loopback policy rejection, absence of a direct IP path from a new network namespace, socket mode, and teardown.

The starter fails closed without creating a socket. Build from the interfaces and ordering practiced in the three lessons; the lab deliberately does not include a complete implementation.

```
python3 -m py_compile egress_broker.py
../../lab-grade module-08
../../lab-grade module-08 --mode exam
```

Test commands, line by line

- `py_compile` catches syntax errors without starting a listener or contacting a service.
- Practice mode creates fresh names, ports, run identity, and bodies, then reports failed properties with lesson references.
- Exam mode runs the same behavior checks with reduced repair guidance.

Use only the disposable VM and grader-created synthetic loopback services. Do not add veth devices, routes, firewall rules, DNS requests, public endpoints, or a permissive fallback.

Security model summary

The network namespace removes the workload's inherited IP interfaces and routes. The shared filesystem exposes only a mode-0600 Unix socket. The broker is a separate authority boundary: it accepts a fixed request schema, binds it to a run ID, maps only synthetic names to IPv4 loopback, checks the port, bounds transport data, and repeats

authorization after redirects. Network isolation does not make broker policy correct, and broker policy does not replace the filesystem, syscall, privilege, or resource controls from earlier modules.

Glossary

- **Network namespace:** per-namespace interfaces, routes, and related network state.
- **Loopback:** an address that refers only to the network namespace containing it.
- **Unix-domain socket:** a local IPC endpoint addressed through the filesystem rather than IP routing.
- **Mediated egress:** a narrower process performs an approved network operation for an isolated caller.
- **Reauthorization:** repeating policy checks after a request changes, including at every redirect.
- **Run binding:** requiring a request to name the exact synthetic execution identity authorized by policy.

North Echo field manual - Module 09

This chapter is self-contained for the guided credential-brokering exercises and independent lab. Every credential, key, token, service, resource, and value is synthetic and local.

Module overview

Module 09 - Broker credentials with operation capabilities

Remove fake credentials from workload environments, then place them behind a local operation broker. Signed, short-lived capabilities bind the exact operation, resource, arguments, audience, run, and one-use nonce without containing the underlying credential.

Play in order: 09.01, 09.02, 09.03, then module-09.

Outcomes:

- observe that an environment credential is ambient authority inherited across exec;
- remove that authority with an explicit minimal environment;
- distinguish a signed capability from an encrypted secret;
- bind operation, resource, canonical input, audience, run identity, issue time, expiry, and nonce;
- deny tampering, expiry, excessive lifetime, replay, and confused-deputy substitutions before upstream contact;
- prove a broker used a fake credential upstream without returning it to the client.

Every credential, signing key, capability, resource, and upstream service is synthetic and lesson-local. Communication uses filesystem Unix sockets only; no DNS, LAN, public, cloud, employer, or production service is involved.

Prerequisites are Modules 01, 04, and 08.

Lesson 09.01

09.01 - Remove ambient credential authority

Goal

Observe a fake credential crossing `exec` through the environment, trigger a deliberate disclosure, then prove an `exact-argv` launcher removes it.

Exercise 1 - Read the complete probes

```
sed -n '1,200p' credential_probe.py
sed -n '1,200p' clean_launcher.py
```

Source, line by line

- `credential_probe.py` asks `os.environ` for one explicitly fake variable. It reports presence and length without revealing the value by default.
- Its `leak` mode represents an untrusted tool that can read every inherited variable; it adds the value to JSON only to make the synthetic failure observable.
- `clean_launcher.py` requires structured `argv`, constructs a new environment containing only fixed `PATH` and `LANG`, and calls `subprocess.run` with `shell=False`.
- The child status becomes the launcher status. Failure does not trigger a retry with the inherited environment.

Exercise 2 - Trigger ambient disclosure

Set a visibly fake lesson value and invoke the probe through a new Python process:

```
export NORTH_ECHO_FAKE_CREDENTIAL="FAKE-LESSON-$$-DO-NOT-USE"
python3 credential_probe.py
python3 credential_probe.py leak
```

Line by line

- `export` places the name in the environment inherited by later `exec` calls. The value is synthetic and must never be replaced with a real credential.
- The first run should report `"present": true` and a nonzero length. This proves inheritance without disclosure.
- The intentional `leak` run prints `leaked_value`. The child needed no separate credential API or user confirmation; possession of the environment was authority.

This does not mean environment variables are always logged. It proves every launched child can read the value and may copy it elsewhere.

Exercise 3 - Repair inheritance

Run the same exact probe through the clean launcher:

```
python3 clean_launch.py /usr/bin/python3 "$PWD/credential_probe.py"
python3 clean_launch.py /usr/bin/python3 "$PWD/credential_probe.py" leak
test -n "$NORTH_ECHO_FAKE_CREDENTIAL"
unset NORTH_ECHO_FAKE_CREDENTIAL
```

Line by line

- `/usr/bin/python3` is an exact executable path and the script is a separate `argv` element.
- Both children should report `"present": false`; Leak cannot print a value it did not inherit.
- The parent-side test proves the fake credential still existed outside the child. The observation is about the launch boundary, not accidental deletion.
- `unset` removes the synthetic value from the lesson shell when the observation is complete.

Intentional mistake: change `env=clean` to `env=os.environ.copy()`. The probe sees the value again. Restore the fixed dictionary and require both modes to report absence.

The repair also removes functionality for a tool that genuinely needs the credential. Lessons 09.02-09.03 restore only an approved operation through a broker; they do not put the credential back into the workload.

Checkpoint and troubleshooting

```
export NORTH_ECHO_FAKE_CREDENTIAL=FAKE-CHECKPOINT
test "$(python3 clean_launch.py /usr/bin/python3 "$PWD/credential_probe.py")" = '{"length": 0, "present": false}'
unset NORTH_ECHO_FAKE_CREDENTIAL
```

- If the child still sees the value, confirm `env=clean` is passed to the exact `subprocess.run` call.
- Do not inspect `/proc` entries belonging to unrelated processes; this exercise needs only its own child environment.
- Checkpoint: explain why a shorter-lived environment credential remains ambient authority during its lifetime.

Complete guided source listings

`credential_probe.py`

```
#!/usr/bin/env python3
"""Observe whether a synthetic credential crossed exec."""

import json
import os
import sys

name = "NORTH_ECHO_FAKE_CREDENTIAL"
value = os.environ.get(name)
result = {"present": value is not None, "length": len(value) if value else 0}
if len(sys.argv) == 2 and sys.argv[1] == "leak" and value is not None:
    result["leaked_value"] = value
print(json.dumps(result, sort_keys=True))
```

`clean_launch.py`

```
#!/usr/bin/env python3
"""Launch exact argv with a small, credential-free environment."""

import os
import subprocess
import sys

if len(sys.argv) < 2:
    raise SystemExit(f"usage: {sys.argv[0]} COMMAND [ARG...]")
clean = {"PATH": "/usr/bin:/bin", "LANG": "C.UTF-8"}
run = subprocess.run(sys.argv[1:], shell=False, env=clean, check=False)
raise SystemExit(run.returncode)
```

Lesson 09.02

09.02 - Bind a signed operation capability

Goal

Mint a short-lived token whose signature covers the exact operation context, then deny tampering, substitution, and expiry.

Exercise 1 - Read the complete capability tool

```
sed -n '1,300p' capability.py
```

Source, block by block

- encode produces unpadded URL-safe base64. It is an encoding, not encryption.
- canonical gives JSON one deterministic byte representation; input_digest hashes parsed input rather than caller whitespace.
- mint bounds lifetime to 1-300 seconds and creates claims for version, operation, resource, audience, run, issue/expiry time, random nonce, and input digest.
- HMAC-SHA256 signs the encoded payload with a lesson-local fake key. The token contains payload plus signature, never a service credential.
- verify recomputes the signature with compare_digest, compares every request binding, then checks current time inside the validity interval.
- The CLI writes a token file rather than passing a bearer token through process arguments. Every parse or verification failure returns nonzero.

Exercise 2 - Mint, inspect, and verify

Create a fake signing key with restrictive permissions and mint one read capability:

```
umask 077
printf '%s' 'FAKE-SIGNING-KEY-09.02-ONLY-000000000000' > signing.key
RUN_ID="run-$$"
python3 capability.py mint signing.key token.txt read record:alpha broker.lesson "$RUN_ID" 60 '{}'
python3 capability.py verify signing.key token.txt read record:alpha broker.lesson "$RUN_ID" '{}'
```

Line by line

- umask 077 makes newly created key and token files private to the current user.
- The signing material is synthetic. It authenticates lesson tokens but grants access to no real system.
- RUN_ID prevents a capability from another run being accepted merely because its other fields match.
- Mint binds empty canonical input {}. Verification must provide the same operation, resource, audience, run, and input.
- Expected verification output is JSON containing those claims, a 32-hex-character nonce, and nearby integer timestamps.

Inspect the readable payload without the key:

```
python3 -c 'import base64,json,pathlib; p=pathlib.Path("token.txt").read_text().split(".")[0]; print(json.dumps(json.loads(base64.b64decode(p))))'
! grep -q 'FAKE-SIGNING' token.txt
```

- Anyone holding the capability can decode its claims; signing provides integrity and issuer authenticity, not confidentiality.
- The grep checkpoint proves the signing key itself is not embedded. A service credential must likewise never be placed in claims.

Exercise 3 - Break bindings, then expire the token

```
set +e
python3 capability.py verify signing.key token.txt read record:beta broker.lesson "$RUN_ID" '{}'
resource_status=$?
python3 capability.py verify signing.key token.txt append record:alpha broker.lesson "$RUN_ID" '{}'
operation_status=$?
set -e
test "$resource_status" -ne 0
test "$operation_status" -ne 0
python3 capability.py verify signing.key token.txt read record:alpha broker.lesson "$RUN_ID" '{}'
```

Line by line

- The first two calls are intentional confused-deputy attempts: reuse a valid token while asking for a different resource or operation.
- Both signatures remain valid, but request-to-claim comparison fails. A valid signature alone is not authorization.
- The final exact request repairs the mismatch and succeeds.

Now observe expiry with a separate one-second capability:

```
python3 capability.py mint signing.key short.txt read record:alpha broker.lesson "$RUN_ID" 1 '{}'
sleep 2
! python3 capability.py verify signing.key short.txt read record:alpha broker.lesson "$RUN_ID" '{}'
rm -f signing.key token.txt short.txt
```

- `sleep 2` crosses the one-second validity window; verification must return nonzero.
- Removing the lesson-local fake artifacts limits accidental reuse. Expiry does not erase bearer tokens by itself.

Checkpoint and troubleshooting

```
test "$resource_status" -ne 0
test "$operation_status" -ne 0
```

- If every verification reports signature mismatch, use the same unmodified key file for mint and verify.
- If JSON input fails, quote it as one argv element; the tool hashes parsed canonical JSON.
- Checkpoint: identify which properties are visible in the payload and which property depends on possession of the signing key.

Complete guided source listings

`capability.py`

```

#!/usr/bin/env python3
"""Mint and verify one short-lived, operation-bound synthetic capability."""

from __future__ import annotations

import base64
import hashlib
import hmac
import json
from pathlib import Path
import secrets
import sys
import time

def encode(raw: bytes) -> str:
    return base64.urlsafe_b64encode(raw).rstrip(b"=").decode("ascii")

def canonical(value: object) -> bytes:
    return json.dumps(value, sort_keys=True, separators=(",", ":"), allow_nan=False).encode()

def input_digest(raw: str) -> str:
    return hashlib.sha256(canonical(json.loads(raw))).hexdigest()

def mint(key: bytes, operation: str, resource: str, audience: str, run_id: str, ttl: int, input_json: str) -> str:
    if not 1 <= ttl <= 300:
        raise ValueError("TTL must be 1..300 seconds")
    issued = int(time.time())
    claims = {
        "version": 1,
        "operation": operation,
        "resource": resource,
        "audience": audience,
        "run_id": run_id,
        "issued_at": issued,
        "expires_at": issued + ttl,
        "nonce": secrets.token_hex(16),
        "input_sha256": input_digest(input_json),
    }
    payload = encode(canonical(claims))
    signature = encode(hmac.new(key, payload.encode(), hashlib.sha256).digest())
    return payload + "." + signature

def verify(key: bytes, token: str, operation: str, resource: str, audience: str, run_id: str, input_json: str) -> dict:
    payload, supplied = token.strip().split(".")
    expected = encode(hmac.new(key, payload.encode(), hashlib.sha256).digest())
    if not hmac.compare_digest(supplied, expected):
        raise ValueError("signature mismatch")
    padded = payload + "=" * (-len(payload) % 4)
    claims = json.loads(base64.urlsafe_b64decode(padded))
    now = int(time.time())
    expected_binding = (operation, resource, audience, run_id, input_digest(input_json))
    actual_binding = tuple(claims[name] for name in ("operation", "resource", "audience", "run_id", "input_sha256"))
    if actual_binding != expected_binding:
        raise ValueError("capability binding mismatch")
    if not claims["issued_at"] <= now < claims["expires_at"]:
        raise ValueError("capability is not currently valid")
    return claims

def main() -> int:
    if len(sys.argv) < 2:
        return 2
    if sys.argv[1] == "mint" and len(sys.argv) == 10:
        key = Path(sys.argv[2]).read_bytes()
        token = mint(key, sys.argv[4], sys.argv[5], sys.argv[6], sys.argv[7], int(sys.argv[8]), sys.argv[9])
        Path(sys.argv[3]).write_text(token + "\n", encoding="utf-8")

```

```

        print("capability minted")
        return 0
    if sys.argv[1] == "verify" and len(sys.argv) == 9:
        key = Path(sys.argv[2]).read_bytes()
        token = Path(sys.argv[3]).read_text(encoding="utf-8")
        claims = verify(key, token, sys.argv[4], sys.argv[5], sys.argv[6], sys.argv[7], sys.argv[8])
        print(json.dumps(claims, sort_keys=True))
        return 0
    print("usage: capability.py mint KEY TOKEN OP RESOURCE AUDIENCE RUN TTL INPUT_JSON", file=sys.stderr)
    print("      or: capability.py verify KEY TOKEN OP RESOURCE AUDIENCE RUN INPUT_JSON", file=sys.stderr)
    return 2

if __name__ == "__main__":
    try:
        raise SystemExit(main())
    except (OSError, ValueError, KeyError, json.JSONDecodeError) as error:
        print(f"capability denied: {error}", file=sys.stderr)
        raise SystemExit(1)

```

Lesson 09.03

09.03 - Deny replay and confused-deputy substitution

Goal

Put a fake credential behind a Unix-socket operation broker, consume each capability once, and recheck every binding before contacting a separate synthetic upstream.

Exercise 1 - Read the complete local system

```
sed -n '1,360p' capability_broker.py
sed -n '1,220p' synthetic_upstream.py
sed -n '1,180p' mint_token.py
sed -n '1,160p' broker_client.py
```

Components, block by block

- `mint_token.py` creates the same canonical signed claims as Lesson 09.02. The client receives the token file, not the signing key or service credential.
- `broker_client.py` sends exactly token, operation, resource, audience, run, and parsed input over a filesystem Unix socket.
- `capability_broker.py` validates policy and mode-0600 secret files before listening. It strictly decodes base64, verifies HMAC, validates the complete claim set and lifetime, and compares request fields to claims and policy.
- The broker hashes canonical request input, checks its digest, and rejects a previously consumed nonce. It consumes the nonce before calling upstream, caps the in-memory replay set, and fails closed at capacity, so a failed operation cannot make the same bearer capability reusable.
- Only after those checks does `call_upstream` read the broker-held fake credential into a request for a separate Unix socket. An upstream response that reflects the credential is rejected.
- `synthetic_upstream.py` accepts one exact fake credential, read, resource, and empty input. It never listens on IP and returns only a synthetic value.
- Both services use bounded lines, timeouts, mode-0600 sockets, signal-driven exit, and exact-path unlinking.

Exercise 2 - Launch the brokered operation

Create only fake secret material and a local policy:

```

umask 077
printf '%s' 'FAKE-SIGNING-KEY-09.03-ONLY-000000000000' > signing.key
printf '%s' 'FAKE-UPSTREAM-CREDENTIAL-09.03' > credential.txt
RUN_ID="run-$$"
AUDIENCE="records.lesson"
RESOURCE="record:alpha"
UPSTREAM="$PWD/upstream.sock"
BROKER="$PWD/broker.sock"
cat > policy.json <<EOF
{"version":1,"audience":"$AUDIENCE","run_id":"$RUN_ID","max_ttl":120,"upstream_socket":"$UPSTREAM","permissions":{"record:
EOF
python3 -m json.tool policy.json

```

Line by line

- `umask 077` is verified by the broker; group- or world-accessible secret files fail setup.
- The signing key and credential are different synthetic authorities. A token signature never contains the credential.
- Policy names one audience and run, a maximum lifetime, one exact upstream socket, and an operation allowlist.
- `record:beta` exists in policy specifically so the confused-deputy test isolates token binding from broad policy denial.

Start both exact owned services and wait for their socket types:

```

python3 synthetic_upstream.py credential.txt "$RESOURCE" synthetic-record-value "$UPSTREAM" & upstream_pid=$!
python3 capability_broker.py policy.json signing.key credential.txt "$BROKER" & broker_pid=$!
trap 'kill "$broker_pid" "$upstream_pid" 2>/dev/null || true; wait "$broker_pid" "$upstream_pid" 2>/dev/null || true; rm -f
for attempt in 1 2 3 4 5; do test -S "$BROKER" && test -S "$UPSTREAM" && break; sleep 0.1; done
test "$(stat -c '%a' "$BROKER")" = 600
test "$(stat -c '%a' "$UPSTREAM")" = 600

```

The readiness and mode checks observe effective socket state. No name-prefix cleanup or unrelated process signaling is used.

Mint and spend one capability:

```

python3 mint_token.py signing.key token.txt read "$RESOURCE" "$AUDIENCE" "$RUN_ID" 60 '{}'
python3 broker_client.py "$BROKER" token.txt read "$RESOURCE" "$AUDIENCE" "$RUN_ID" '{}'

```

Expected JSON is ok: `true` with `synthetic-record-value`. Neither response nor client environment contains `FAKE-UPSTREAM-CREDENTIAL-09.03`.

Exercise 3 - Replay and substitute

Replay the same token, then attempt to make the deputy use it for another policy-allowed resource:

```

python3 broker_client.py "$BROKER" token.txt read "$RESOURCE" "$AUDIENCE" "$RUN_ID" '{}'
python3 mint_token.py signing.key deputy.txt read "$RESOURCE" "$AUDIENCE" "$RUN_ID" 60 '{}'
python3 broker_client.py "$BROKER" deputy.txt read record:beta "$AUDIENCE" "$RUN_ID" '{}'

```

Line by line

- The replay returns ok: `false` because the first successful request consumed the nonce.
- `deputy.txt` is valid for `record:alpha`. The request names `record:beta`, which policy generally allows, but the token-to-request comparison denies the substitution before upstream contact.

- This is the confused-deputy repair: possession of broad broker authority does not let the caller reinterpret a narrower capability.

Intentional mistake: temporarily remove the operation/resource/request comparisons in `verify_token`. The second request advances to upstream instead of failing at the broker. Restore all comparisons and require `request` does not match `capability`.

Stop exact processes and verify socket cleanup:

```
kill "$broker_pid" "$upstream_pid"
wait "$broker_pid" "$upstream_pid"
trap - EXIT
test ! -e "$BROKER"
test ! -e "$UPSTREAM"
rm -f signing.key credential.txt policy.json token.txt deputy.txt
```

Checkpoint and troubleshooting

```
test ! -e "$BROKER"
test ! -e "$UPSTREAM"
```

- If setup says secret files must be mode 0600, inspect with `stat -c '%a'` and recreate synthetic files under `umask 077`; never broaden permissions.
- If a fresh token reports invalid time, confirm the VM clock is sane and the requested TTL does not exceed policy.
- If upstream denies an exact request, compare its resource and empty input with the policy and token; do not expose the credential for debugging.
- Checkpoint: explain why the replay set must be broker state and why signature verification alone cannot prevent replay.

Complete guided source listings

``capability_broker.py``

```

#!/usr/bin/env python3
"""Use fake credentials only after a capability passes every binding check."""

from __future__ import annotations

import base64
import binascii
import hashlib
import hmac
import json
import os
from pathlib import Path
import re
import signal
import socket
import stat
import sys
import time

MAX_LINE = 16384
MAX_UPSTREAM = 65536
MAX_USED_NONCES = 4096
stopping = False

class Denied(Exception):
    pass

def canonical(value: object) -> bytes:
    try:
        return json.dumps(value, sort_keys=True, separators=(",", ":"), allow_nan=False).encode()
    except (TypeError, ValueError) as error:
        raise Denied("input is not canonical JSON") from error

def encode(raw: bytes) -> str:
    return base64.urlsafe_b64encode(raw).rstrip(b"=").decode("ascii")

def decode(segment: str) -> bytes:
    if not isinstance(segment, str) or not segment or "=" in segment or len(segment) > 8192:
        raise Denied("invalid token encoding")
    try:
        raw = base64.b64decode(segment + "=" * (-len(segment) % 4), altchars=b"._", validate=True)
    except (ValueError, binascii.Error) as error:
        raise Denied("invalid token encoding") from error
    if encode(raw) != segment:
        raise Denied("noncanonical token encoding")
    return raw

def load_policy(path: Path) -> dict:
    raw = json.loads(path.read_text(encoding="utf-8"))
    expected = {"version", "audience", "run_id", "max_ttl", "upstream_socket", "permissions"}
    if not isinstance(raw, dict) or set(raw) != expected or raw["version"] != 1:
        raise Denied("invalid policy shape")
    if not all(isinstance(raw[name], str) and 1 <= len(raw[name]) <= 128 for name in ("audience", "run_id")):
        raise Denied("invalid policy identity")
    if type(raw["max_ttl"]) is not int or not 1 <= raw["max_ttl"] <= 300:
        raise Denied("invalid maximum lifetime")
    upstream = Path(raw["upstream_socket"])
    if not upstream.is_absolute():
        raise Denied("upstream socket must be absolute")
    permissions = raw["permissions"]
    if not isinstance(permissions, dict) or not permissions:
        raise Denied("policy has no permissions")
    for resource, operations in permissions.items():
        if not isinstance(resource, str) or not re.fullmatch(r"[a-z0-9][a-z0-9._-]{0,127}", resource):
            raise Denied("invalid policy resource")
        if not isinstance(operations, list) or not operations or any(not isinstance(op, str) or not re.fullmatch(r"[a-z][a-z0-9._-]{0,127}", op) for op in operations):
            raise Denied("invalid policy operations")

```

```

        raise Denied("invalid policy operation")
    return raw

def receive_line(peer: socket.socket, limit: int) -> bytes:
    data = bytearray()
    while b"\n" not in data and len(data) <= limit:
        chunk = peer.recv(min(1024, limit + 1 - len(data)))
        if not chunk:
            break
        data.extend(chunk)
    if len(data) > limit or not data.endswith(b"\n") or data.count(b"\n") != 1:
        raise Denied("request must be one bounded line")
    return bytes(data)

def verify_token(policy: dict, key: bytes, request: dict) -> str:
    required = {"token", "operation", "resource", "audience", "run_id", "input"}
    if not isinstance(request, dict) or set(request) != required:
        raise Denied("invalid request shape")
    if not all(isinstance(request[name], str) for name in ("token", "operation", "resource", "audience", "run_id")):
        raise Denied("invalid request values")
    try:
        payload_text, signature_text = request["token"].split(".")
    except ValueError as error:
        raise Denied("invalid token shape") from error
    supplied = decode(signature_text)
    expected = hmac.new(key, payload_text.encode("ascii"), hashlib.sha256).digest()
    if not hmac.compare_digest(supplied, expected):
        raise Denied("invalid token signature")
    try:
        claims = json.loads(decode(payload_text))
    except (json.JSONDecodeError, UnicodeError) as error:
        raise Denied("invalid token payload") from error
    claim_names = {"version", "operation", "resource", "audience", "run_id", "issued_at", "expires_at", "nonce", "input_sha256"}
    if not isinstance(claims, dict) or set(claims) != claim_names or claims["version"] != 1:
        raise Denied("invalid token claims")
    for name in ("operation", "resource", "audience", "run_id", "nonce", "input_sha256"):
        if not isinstance(claims[name], str):
            raise Denied("invalid token claim type")
    if type(claims["issued_at"]) is not int or type(claims["expires_at"]) is not int:
        raise Denied("invalid token time type")
    now = int(time.time())
    if claims["issued_at"] > now or claims["expires_at"] <= now:
        raise Denied("capability is not currently valid")
    if claims["expires_at"] <= claims["issued_at"] or claims["expires_at"] - claims["issued_at"] > policy["max_ttl"]:
        raise Denied("capability lifetime exceeds policy")
    if not re.fullmatch(r"[0-9a-f]{32}", claims["nonce"]):
        raise Denied("invalid capability nonce")
    if not re.fullmatch(r"[0-9a-f]{64}", claims["input_sha256"]):
        raise Denied("invalid input digest")
    bindings = ("operation", "resource", "audience", "run_id")
    if any(claims[name] != request[name] for name in bindings):
        raise Denied("request does not match capability")
    if request["audience"] != policy["audience"] or request["run_id"] != policy["run_id"]:
        raise Denied("request identity is not authorized")
    if request["operation"] not in policy["permissions"].get(request["resource"], []):
        raise Denied("operation is not permitted")
    digest = hashlib.sha256(canonical(request["input"])).hexdigest()
    if not hmac.compare_digest(digest, claims["input_sha256"]):
        raise Denied("request input does not match capability")
    return claims["nonce"]

def call_upstream(policy: dict, credential: str, request: dict) -> object:
    message = {"credential": credential, "operation": request["operation"], "resource": request["resource"], "input": request["input"]}
    with socket.socket(socket.AF_UNIX, socket.SOCK_STREAM) as stream:
        stream.settimeout(2)
        stream.connect(policy["upstream_socket"])
        stream.sendall(canonical(message) + b"\n")
        stream.shutdown(socket.SHUT_WR)

```

```

        raw = receive_line(stream, MAX_UPSTREAM)
    if credential.encode() in raw:
        raise Denied("upstream attempted credential disclosure")
    response = json.loads(raw)
    if not isinstance(response, dict) or set(response) != {"ok", "result"} or response["ok"] is not True:
        raise Denied("upstream denied operation")
    return response["result"]

def serve(policy: dict, key: bytes, credential: str, socket_path: Path) -> None:
    if socket_path.exists() or socket_path.is_symlink():
        raise Denied("broker socket path already exists")
    listener = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
    listener.bind(str(socket_path))
    os.chmod(socket_path, 0o600)
    listener.listen(8)
    listener.settimeout(0.2)
    used_nonces: set[str] = set()
    try:
        while not stopping:
            try:
                peer, _ = listener.accept()
            except TimeoutError:
                continue
            with peer:
                try:
                    request = json.loads(receive_line(peer, MAX_LINE))
                    nonce = verify_token(policy, key, request)
                    if nonce in used_nonces:
                        raise Denied("capability replay denied")
                    if len(used_nonces) >= MAX_USED_NONCES:
                        raise Denied("replay cache capacity reached")
                    used_nonces.add(nonce)
                    result = call_upstream(policy, credential, request)
                    response = {"ok": True, "result": result}
                except (Denied, json.JSONDecodeError, UnicodeError, OSError) as error:
                    response = {"ok": False, "error": str(error)}
                peer.sendall(canonical(response) + b"\n")
    finally:
        listener.close()
        socket_path.unlink(missing_ok=True)

def stop(_signal, _frame):
    global stopping
    stopping = True

def main() -> int:
    if len(sys.argv) != 5:
        print(f"usage: {sys.argv[0]} POLICY SIGNING_KEY CREDENTIAL SOCKET", file=sys.stderr)
        return 2
    signal.signal(signal.SIGTERM, stop)
    signal.signal(signal.SIGINT, stop)
    try:
        policy = load_policy(Path(sys.argv[1]))
        key_path, credential_path = Path(sys.argv[2]), Path(sys.argv[3])
        if key_path.is_symlink() or credential_path.is_symlink():
            raise Denied("broker secret files cannot be symlinks")
        if not stat.S_ISREG(key_path.stat().st_mode) or not stat.S_ISREG(credential_path.stat().st_mode):
            raise Denied("broker secret paths must be regular files")
        key = key_path.read_bytes()
        credential = credential_path.read_text(encoding="utf-8").strip()
        if not 32 <= len(key) <= 128 or not 8 <= len(credential) <= 256:
            raise Denied("invalid broker secret material")
        if stat.S_IMODE(key_path.stat().st_mode) & 0o077 or stat.S_IMODE(credential_path.stat().st_mode) & 0o077:
            raise Denied("broker secret files must be mode 0600")
        serve(policy, key, credential, Path(sys.argv[4]))
    except (Denied, OSError, ValueError, json.JSONDecodeError) as error:
        print(f"broker setup denied: {error}", file=sys.stderr)
        return 1

```

```
return 0
```

```
if __name__ == "__main__":  
    raise SystemExit(main())
```

``synthetic_upstream.py``

```
#!/usr/bin/env python3  
"""Synthetic credential-protected operation service over a Unix socket."""  
  
import json  
import os  
from pathlib import Path  
import signal  
import socket  
import sys  
  
stopping = False  
  
def stop(_sigum, _frame):  
    global stopping  
    stopping = True  
  
if len(sys.argv) != 5:  
    raise SystemExit(f"usage: {sys.argv[0]} CREDENTIAL RESOURCE VALUE SOCKET")  
credential = Path(sys.argv[1]).read_text(encoding="utf-8").strip()  
resource, value, socket_text = sys.argv[2:]  
socket_path = Path(socket_text)  
if socket_path.exists() or socket_path.is_symlink():  
    raise SystemExit("upstream socket path already exists")  
signal.signal(signal.SIGTERM, stop)  
listener = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)  
listener.bind(str(socket_path))  
os.chmod(socket_path, 0o600)  
listener.listen(8)  
listener.settimeout(0.2)  
try:  
    while not stopping:  
        try:  
            peer, _ = listener.accept()  
        except TimeoutError:  
            continue  
        with peer:  
            try:  
                request = json.loads(peer.makefile("rb").readline(16384))  
                allowed = request == {"credential": credential, "operation": "read", "resource": resource, "input": {}}  
                response = {"ok": True, "result": {"value": value}} if allowed else {"ok": False, "error": "upstream denied"}  
            except (json.JSONDecodeError, OSError):  
                response = {"ok": False, "error": "invalid upstream request"}  
            peer.sendall(json.dumps(response, sort_keys=True).encode() + b"\n")  
finally:  
    listener.close()  
    socket_path.unlink(missing_ok=True)
```

``mint_token.py``

```
#!/usr/bin/env python3
"""Mint a lesson capability; signing material stays outside the client."""

import base64
import hashlib
import hmac
import json
from pathlib import Path
import secrets
import sys
import time

def encode(raw):
    return base64.urlsafe_b64encode(raw).rstrip(b"=").decode("ascii")

def canonical(value):
    return json.dumps(value, sort_keys=True, separators=(",", ":"), allow_nan=False).encode()

if len(sys.argv) != 9:
    raise SystemExit(f"usage: {sys.argv[0]} KEY TOKEN OP RESOURCE AUDIENCE RUN TTL INPUT_JSON")
key = Path(sys.argv[1]).read_bytes()
issued = int(time.time())
input_value = json.loads(sys.argv[8])
claims = {
    "version": 1,
    "operation": sys.argv[3],
    "resource": sys.argv[4],
    "audience": sys.argv[5],
    "run_id": sys.argv[6],
    "issued_at": issued,
    "expires_at": issued + int(sys.argv[7]),
    "nonce": secrets.token_hex(16),
    "input_sha256": hashlib.sha256(canonical(input_value)).hexdigest(),
}
payload = encode(canonical(claims))
signature = encode(hmac.new(key, payload.encode(), hashlib.sha256).digest())
Path(sys.argv[2]).write_text(payload + "." + signature + "\n", encoding="utf-8")
print("capability minted")
```

broker_client.py

```
#!/usr/bin/env python3
"""Send one capability-bound operation request."""

import json
from pathlib import Path
import socket
import sys

if len(sys.argv) != 8:
    raise SystemExit(f"usage: {sys.argv[0]} SOCKET TOKEN OP RESOURCE AUDIENCE RUN INPUT_JSON")
request = {
    "token": Path(sys.argv[2]).read_text(encoding="utf-8").strip(),
    "operation": sys.argv[3],
    "resource": sys.argv[4],
    "audience": sys.argv[5],
    "run_id": sys.argv[6],
    "input": json.loads(sys.argv[7]),
}
with socket.socket(socket.AF_UNIX, socket.SOCK_STREAM) as stream:
    stream.connect(sys.argv[1])
    stream.sendall(json.dumps(request, sort_keys=True).encode() + b"\n")
    stream.shutdown(socket.SHUT_WR)
    print(stream.makefile("r", encoding="utf-8").readline(), end="")
```

Independent lab

Module 09 independent lab - Capability-bound credential broker

Implement `capability_broker.py`. The grader invokes:

```
python3 capability_broker.py POLICY.json SIGNING_KEY CREDENTIAL SOCKET
```

The policy contains version 1, exact audience and run identity, a maximum lifetime, an absolute synthetic-upstream Unix socket, and a resource-to-operation allowlist. Signing-key and fake-credential files are mode 0600. The broker must validate all setup before creating its own mode-0600 Unix socket.

Clients send one bounded JSON line containing exactly:

```
{"token":"PAYLOAD.SIGNATURE","operation":"read","resource":"record:example","audience":"records.example","run_id":"run-exa"}
```

Verify canonical URL-safe base64 and HMAC-SHA256 with constant-time comparison. Require the exact version and claim set: operation, resource, audience, run, issue time, expiry, nonce, and canonical-input SHA-256. Deny future, expired, nonpositive, or policy-overlong lifetimes. Match request fields and input to claims, then match audience/run and permission to policy. Consume a valid 32-hex nonce before upstream contact and deny reuse.

Only after every check may the broker read the fake credential and send it with the authorized operation to the exact upstream Unix socket. Bound the in-memory replay set and fail closed at capacity. Never place the credential or signing key in tokens, client responses, errors, logs, argv values, or environment variables. Reject an upstream response that reflects the credential. Clean termination must remove the exact broker socket.

The external grader uses fresh fake secrets, identities, resources, values, times, nonces, and a separate synthetic upstream. It checks success and credential use, non-disclosure, replay, signature tampering, audience/run binding, time bounds, operation/resource/input substitution, policy denial, strict request shape, private socket mode, insecure secret-file rejection before listen, and cleanup. Protected attempts must be denied before upstream contact.

The starter is a runnable, private, fail-closed Unix server that denies every operation. It intentionally does not implement capabilities or contact upstream. The lab withholds a complete solution; use only interfaces practiced in Lessons 09.01-09.03.

```
python3 -m py_compile capability_broker.py
../../lab-grade module-09
../../lab-grade module-09 --mode exam
```

Test commands, line by line

- `py_compile` checks syntax without opening a socket or reading secret material.
- Practice mode runs fresh behavior checks and names relevant lessons for failed properties.
- Exam mode repeats the same properties while withholding lesson references.

Use only the generated fake credential and synthetic Unix-socket upstream. Never substitute a real key, token, credential, network service, or production resource.

Security model summary

The workload receives neither the fake service credential nor the signing key. It receives a bearer capability whose readable claims are integrity-protected and narrowly bind an operation. The broker independently rechecks signature, claim shape, time, audience, run, policy, operation, resource, canonical input, and nonce before using its broader credential. The upstream proves credential use, while response filtering and external grading prove non-disclosure. A capability remains authority until expiry or consumption, so storage and replay state still matter.

Glossary

- **Ambient authority:** authority available to code without an explicit operation-specific grant.
- **Capability:** an unforgeable or integrity-protected value that conveys specific authority to its holder.
- **Audience:** the broker or service for which a capability is intended.
- **Nonce:** a unique identifier consumed to make a bearer capability one-use.
- **Replay:** reuse of a previously accepted authorization value.
- **Confused deputy:** a more-privileged component induced to use its authority for a caller-selected target outside the caller's grant.
- **HMAC:** a keyed message-authentication code that provides integrity and issuer authentication, not confidentiality.

North Echo field manual - Module 10

This chapter is self-contained for the complete-runtime composition lessons and independent lab. It repeats every guided command and complete source listing used by the module. All services, capabilities, credentials, paths, and data are synthetic and local to the disposable Linux VM.

The central security rule is conjunctive: the runtime is acceptable only when every layer is effective at the same time and the owned process tree is collected afterward. Configuration intent is not evidence; the lessons inspect state from inside the final workload and separately inspect teardown from outside it.

Module overview

Module 10 - Compose a complete agent runtime

Assemble the controls from Modules 01-09 in dependency order, then ask the running workload and kernel for evidence that the composition is effective. The final runtime has bounded resources, fresh user and network namespaces, empty capability sets, `no_new_privs`, Landlock filesystem policy, a default-deny seccomp filter, capability-mediated Unix-socket access, a clean credential environment, structured telemetry, and synchronous process-tree collection.

Play in order: 10.01, 10.02, 10.03, then `module-10`.

Outcomes:

- distinguish launch dependencies from an arbitrary checklist order;
- apply pathname policy before a syscall filter can remove the setup syscalls;
- keep the broker outside the restricted network namespace while exposing only its Unix socket;
- place the launcher and every descendant under cgroup limits before workload creation;
- attest effective namespaces, capabilities, Landlock effects, seccomp mode, cgroup values, credential absence, and broker success;
- propagate workload failure and synchronously collect the exact transient unit.

All data, credentials, capabilities, services, paths, and requests are synthetic and local. The module creates no veth pair, route, firewall rule, public request, real credential, or production target. Prerequisites are Modules 01-09 and the Linux features reported by `./scripts/linux-preflight`.

Lesson 10.01

10.01 - Order the complete runtime by dependency

Goal

Turn the earlier controls into a launch dependency graph, trigger an invalid order, and prove the repaired plan satisfies every dependency.

Exercise 1 - Read the plan validator

```
sed -n '1,240p' check_order.py
python3 -m json.tool broken-plan.json
python3 -m json.tool repaired-plan.json
```

Source, block by block

- REQUIRED names every phase that changes the security result. A plan cannot silently omit teardown or broker readiness.
- BEFORE contains dependency edges, not a preferred cosmetic order. The cgroup precedes descendants; namespace creation precedes capability removal; Landlock setup precedes a filter that does not allow Landlock syscalls; every restriction precedes workload exec; collection follows workload exit.
- main accepts only a JSON string array, rejects duplicate, missing, and unknown steps, maps each step to its position, and reports every reversed edge.
- The exit status is nonzero whenever an effective dependency is violated, so the check can gate a launcher build.

Exercise 2 - Trigger the ordering failure

```
python3 check_order.py broken-plan.json; test "$?" -eq 1
```

Expected output includes `apply_landlock must precede apply_seccomp`. The deliberate mistake installs seccomp first. A useful workload policy may deny the Landlock setup syscalls; applying that filter first can make the filesystem control impossible to install. A list containing both controls is not proof of composition.

Exercise 3 - Repair and challenge the graph

```
python3 check_order.py repaired-plan.json
cp repaired-plan.json challenge-plan.json
python3 - <<'PY'
import json
from pathlib import Path
p = Path("challenge-plan.json")
steps = json.loads(p.read_text())
steps.remove("enter_cgroup")
steps.insert(3, "enter_cgroup")
p.write_text(json.dumps(steps) + "\n")
PY
python3 check_order.py challenge-plan.json; test "$?" -eq 1
rm challenge-plan.json
```

Line by line

- The repaired plan moves Landlock before seccomp and returns `ok: true`.
- The small Python edit deliberately moves cgroup entry after namespace creation. The validator rejects it because processes created during namespace setup would otherwise exist before accounting is attached.
- Removing the temporary challenge leaves only shipped lesson files.

The broker starts before the isolated child because the child has no inherited IP network and must find a ready filesystem socket. The user namespace is entered before dropping capabilities because mapping root in a new user namespace creates namespace-scoped capabilities that must then be removed. Collection is last because `--collect` is a lifecycle guarantee, not a workload restriction.

Checkpoint and troubleshooting

```
test "$(python3 check_order.py repaired-plan.json)" = '{"ok": true, "violations": []}'
```

- If JSON parsing fails, restore an array of quoted step names; ordering is evaluated only after shape validation.
- If a step appears harmless to move, identify what it creates, what syscalls it needs, and which later phase removes that authority.
- Checkpoint: explain why `apply_landlock` before `apply_seccomp` and `enter_namespaces` before `drop_privilege` are dependencies rather than style choices.

Complete source: `check\_order.py`

Canonical path: course/module-10-complete-runtime/lesson-01/check_order.py

```
#!/usr/bin/env python3
"""Validate a complete-runtime launch plan against security dependencies."""

import json
import sys
from pathlib import Path

REQUIRED = {
    "start_broker",
    "enter_cgroup",
    "enter_namespaces",
    "drop_privilege",
    "apply_landlock",
    "apply_seccomp",
    "exec_workload",
    "collect_unit",
}

BEFORE = {
    ("start_broker", "enter_namespaces"),
    ("enter_cgroup", "enter_namespaces"),
    ("enter_namespaces", "drop_privilege"),
    ("drop_privilege", "apply_landlock"),
    ("apply_landlock", "apply_seccomp"),
    ("apply_seccomp", "exec_workload"),
    ("exec_workload", "collect_unit"),
}

def main() -> int:
    if len(sys.argv) != 2:
        print(f"usage: {sys.argv[0]} PLAN.json", file=sys.stderr)
        return 2
    try:
        plan = json.loads(Path(sys.argv[1]).read_text(encoding="utf-8"))
    except (OSError, json.JSONDecodeError) as error:
        print(f"invalid plan: {error}", file=sys.stderr)
        return 1
    if not isinstance(plan, list) or any(not isinstance(item, str) for item in plan):
        print("invalid plan: expected a JSON array of step names", file=sys.stderr)
        return 1
    if len(plan) != len(set(plan)):
        print("invalid plan: duplicate step", file=sys.stderr)
        return 1
    missing = sorted(REQUIRED - set(plan))
    extra = sorted(set(plan) - REQUIRED)
    if missing or extra:
        print(json.dumps({"ok": False, "missing": missing, "extra": extra}, sort_keys=True))
        return 1
    position = {name: index for index, name in enumerate(plan)}
    violations = sorted(f"{left} must precede {right}" for left, right in BEFORE if position[left] > position[right])
    print(json.dumps({"ok": not violations, "violations": violations}, sort_keys=True))
    return bool(violations)

if __name__ == "__main__":
    raise SystemExit(main())
```

Complete source: `broken-plan.json`

Canonical path: `course/module-10-complete-runtime/lesson-01/broken-plan.json`

```
[  
  "start_broker",  
  "enter_cgroup",  
  "enter_namespaces",  
  "drop_privilege",  
  "apply_seccomp",  
  "apply_landlock",  
  "exec_workload",  
  "collect_unit"  
]
```

Complete source: `repaired-plan.json`

Canonical path: `course/module-10-complete-runtime/lesson-01/repaired-plan.json`

```
[  
  "start_broker",  
  "enter_cgroup",  
  "enter_namespaces",  
  "drop_privilege",  
  "apply_landlock",  
  "apply_seccomp",  
  "exec_workload",  
  "collect_unit"  
]
```

Lesson 10.02

10.02 - Seal filesystem and syscall policy before exec

Goal

Observe an unrestricted native probe, then place the same static executable behind one guard that applies Landlock before a native-architecture, default-deny seccomp filter.

Exercise 1 - Read and compile the complete programs

```
sed -n '1,360p' runtime_guard.c
sed -n '1,240p' guard_probe.c
gcc -O2 -Wall -Wextra -static runtime_guard.c -o runtime_guard -lseccomp
gcc -O2 -Wall -Wextra -static guard_probe.c -o guard_probe
```

Line by line

- Both sed commands display the complete local sources before execution.
- -O2 makes an ordinary optimized binary; -Wall -Wextra keep diagnostics visible; -static removes post-policy dynamic-loader dependencies.
- Only the guard links libseccomp because the probe merely observes the installed filter.

`runtime\_guard.c`, block by block

- The syscall wrappers call the Landlock ABI directly. `supported_rights` handles only rights known to the running kernel, including `REFER` and `TRUNCATE` when their ABI versions exist.
- `install_landlock` handles the full filesystem rights set but grants the workload root only execute/read and grants `/proc` plus `/sys/fs/cgroup` only read. Unmentioned paths receive no handled access.
- `PR_SET_NO_NEW_PRIVS` precedes `landlock_restrict_self`; a regular user cannot otherwise enforce the ruleset on itself.
- `install_seccomp` starts from `EPERM`, adds a small static-program syscall surface, and allows socket only when argument zero is `AF_UNIX`. `AF_INET` and alternate socket domains therefore remain denied by the default.
- The guard loads the filter only after Landlock setup is complete and calls `execv` with the original `argv` boundaries.

`guard\_probe.c`, block by block

- `readable` performs a real open and read; it does not infer access from path text or mode bits.
- `status_number` reads the probe's own `NoNewPrivs` and `Seccomp` fields from `/proc/self/status`.
- `main` reads one allowed and one protected file, attempts an IPv4 socket, and prints observed return values and kernel state.

Static linking keeps the execution policy small: the post-Landlock exec does not need dynamic-loader reads from host library directories.

Exercise 2 - Observe the incomplete launch

```
DEMO=$(mktemp -d /tmp/north-echo-10.02.XXXXXX)
mkdir "$DEMO/allowed"
cp guard_probe "$DEMO/allowed/guard_probe"
printf 'allowed\n' > "$DEMO/allowed/input.txt"
printf 'synthetic-protected\n' > "$DEMO/protected.txt"
"$DEMO/allowed/guard_probe" "$DEMO/allowed/input.txt" "$DEMO/protected.txt"
```

Expected patterns are `allowed=1 protected=1`, a nonnegative `inet_fd`, and the host process's `seccomp/no-new-privileges` values. The intentional mistake is launching the probe merely because its binary is trusted. Nothing prevents it from opening the protected file or creating an IP socket.

Exercise 3 - Repair with ordered kernel policy

```
./runtime_guard "$DEMO/allowed" "$DEMO/allowed/guard_probe" \
"$DEMO/allowed/input.txt" "$DEMO/protected.txt"
```

Expected patterns are:

```
allowed=1 protected=0 protected_errno=Permission denied inet_fd=-1 inet_errno=Operation not permitted nnp=1 seccomp=2
```

- The allowed read proves the policy did not simply break all file access.
- `Permission denied` is an effective Landlock observation against a file whose Unix mode still permits the user.
- `AF_INET` returns `EPERM`, while Lesson 10.03 will prove the explicitly allowed `AF_UNIX` broker path still works.
- `NoNewPrivs: 1` and `Seccomp: 2` are kernel-reported state after `exec`, not claims made by the launcher.

Clean only the exact lesson objects:

```
rm "$DEMO/allowed/guard_probe" "$DEMO/allowed/input.txt" "$DEMO/protected.txt"
rmdir "$DEMO/allowed" "$DEMO"
rm runtime_guard guard_probe
```

Checkpoint and troubleshooting

```
test ! -e "$DEMO" || { echo "lesson temporary directory remains" >&2; false; }
```

- If static linking fails, install the packages named by `./scripts/linux-preflight`; do not weaken the Landlock policy to expose host libraries.
- If Landlock reports unsupported, use the disposable VM kernel required by Module 05.
- If `execv` returns `EPERM`, confirm the static workload executable is beneath the allowed root and Landlock was installed before `seccomp`.
- Checkpoint: explain why `seccomp` mode 2 does not prove pathname confinement, and why a protected-file denial does not prove IP denial.

Complete source: ``runtime_guard.c``

Canonical path: `course/module-10-complete-runtime/lesson-02/runtime_guard.c`

```

#define _GNU_SOURCE
#include <errno.h>
#include <fcntl.h>
#include <linux/landlock.h>
#include <seccomp.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/prctl.h>
#include <sys/socket.h>
#include <sys/syscall.h>
#include <unistd.h>

static int create_ruleset(const struct landlock_ruleset_attr *attr, size_t size, __u32 flags) {
    return syscall(SYS_landlock_create_ruleset, attr, size, flags);
}

static int add_path_rule(int ruleset_fd, const char *path, __u64 access) {
    int path_fd = open(path, O_PATH | O_CLOEXEC);
    if (path_fd == -1)
        return -1;
    struct landlock_path_beneath_attr rule = {.allowed_access = access, .parent_fd = path_fd};
    int result = syscall(SYS_landlock_add_rule, ruleset_fd, LANDLOCK_RULE_PATH_BENEATH, &rule, 0);
    close(path_fd);
    return result;
}

static __u64 supported_rights(int abi) {
    __u64 rights = LANDLOCK_ACCESS_FS_EXECUTE | LANDLOCK_ACCESS_FS_WRITE_FILE |
        LANDLOCK_ACCESS_FS_READ_FILE | LANDLOCK_ACCESS_FS_READ_DIR |
        LANDLOCK_ACCESS_FS_REMOVE_DIR | LANDLOCK_ACCESS_FS_REMOVE_FILE |
        LANDLOCK_ACCESS_FS_MAKE_CHAR | LANDLOCK_ACCESS_FS_MAKE_DIR |
        LANDLOCK_ACCESS_FS_MAKE_REG | LANDLOCK_ACCESS_FS_MAKE SOCK |
        LANDLOCK_ACCESS_FS_MAKE_FIFO | LANDLOCK_ACCESS_FS_MAKE_BLOCK |
        LANDLOCK_ACCESS_FS_MAKE_SYM;

    if (abi >= 2)
        rights |= LANDLOCK_ACCESS_FS_REFER;
    if (abi >= 3)
        rights |= LANDLOCK_ACCESS_FS_TRUNCATE;
    return rights;
}

static int install_landlock(const char *allowed_root) {
    int abi = create_ruleset(NULL, 0, LANDLOCK_CREATE_RULESET_VERSION);
    if (abi < 1)
        return -1;
    __u64 handled = supported_rights(abi);
    __u64 read_execute = LANDLOCK_ACCESS_FS_EXECUTE | LANDLOCK_ACCESS_FS_READ_FILE |
        LANDLOCK_ACCESS_FS_READ_DIR;
    __u64 read_only = LANDLOCK_ACCESS_FS_READ_FILE | LANDLOCK_ACCESS_FS_READ_DIR;
    struct landlock_ruleset_attr ruleset = {.handled_access_fs = handled};
    int ruleset_fd = create_ruleset(&ruleset, sizeof(ruleset), 0);
    if (ruleset_fd == -1)
        return -1;
    if (add_path_rule(ruleset_fd, allowed_root, read_execute) == -1 ||
        add_path_rule(ruleset_fd, "/proc", read_only) == -1 ||
        add_path_rule(ruleset_fd, "/sys/fs/cgroup", read_only) == -1) {
        close(ruleset_fd);
        return -1;
    }
    if (prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0) == -1 ||
        syscall(SYS_landlock_restrict_self, ruleset_fd, 0) == -1) {
        close(ruleset_fd);
        return -1;
    }
    close(ruleset_fd);
    return 0;
}

static int allow_name(scmp_filter_ctx context, const char *name) {
    int number = seccomp_syscall_resolve_name(name);
    return number == __NR_SCMP_ERROR ? 0 : seccomp_rule_add(context, SCMP_ACT_ALLOW, number, 0);
}

```

```

}

static int install_seccomp(void) {
    const char *allowed[] = {
        "execve", "read", "write", "close", "openat", "brk", "mmap", "mprotect",
        "munmap", "set_tid_address", "set_robust_list", "prlimit64", "readlink",
        "readlinkat", "getrandom", "rseq", "arch_prctl", "fstat", "newfstatat",
        "faccessat", "lseek", "rt_sigaction", "rt_sigprocmask", "statx", "connect",
        "shutdown", "exit", "exit_group"
    };
    scmp_filter_ctx context = seccomp_init(SCMP_ACT_ERRNO(EPERM));
    if (!context)
        return -1;
    for (size_t index = 0; index < sizeof(allowed) / sizeof(allowed[0]); index++) {
        if (allow_name(context, allowed[index]) < 0) {
            seccomp_release(context);
            return -1;
        }
    }
    int socket_number = seccomp_syscall_resolve_name("socket");
    if (socket_number == __NR_SCMP_ERROR ||
        seccomp_rule_add(context, SCMP_ACT_ALLOW, socket_number, 1,
            SCMP_A0(SCMP_CMP_EQ, AF_UNIX)) < 0 ||
        seccomp_load(context) < 0) {
        seccomp_release(context);
        return -1;
    }
    seccomp_release(context);
    return 0;
}

int main(int argc, char **argv) {
    if (argc < 3) {
        fprintf(stderr, "usage: %s ALLOWED_ROOT COMMAND [ARG...]\n", argv[0]);
        return 2;
    }
    if (install_landlock(argv[1]) == -1) {
        perror("install Landlock");
        return 1;
    }
    if (install_seccomp() == -1) {
        perror("install seccomp");
        return 1;
    }
    execv(argv[2], &argv[2]);
    perror("execv");
    return 1;
}

```

Complete source: `guard\_probe.c`

Canonical path: course/module-10-complete-runtime/lesson-02/guard_probe.c

```
#define _GNU_SOURCE
#include <errno.h>
#include <fcntl.h>
#include <netinet/in.h>
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

static int readable(const char *path) {
    int fd = open(path, O_RDONLY | O_CLOEXEC);
    if (fd == -1)
        return 0;
    char byte;
    int ok = read(fd, &byte, 1) == 1;
    close(fd);
    return ok;
}

static int status_number(const char *name) {
    FILE *stream = fopen("/proc/self/status", "r");
    char line[256];
    int value = -1;
    while (stream && fgets(line, sizeof(line), stream))
        if (sscanf(line, name, &value) == 1)
            break;
    if (stream)
        fclose(stream);
    return value;
}

int main(int argc, char **argv) {
    if (argc != 3) {
        fprintf(stderr, "usage: %s ALLOWED_FILE PROTECTED_FILE\n", argv[0]);
        return 2;
    }
    int allowed = readable(argv[1]);
    errno = 0;
    int protected = readable(argv[2]);
    int protected_errno = errno;
    errno = 0;
    int inet = socket(AF_INET, SOCK_STREAM, 0);
    int inet_errno = errno;
    if (inet != -1)
        close(inet);
    printf("allowed=%d protected=%d protected_errno=%s inet_fd=%d inet_errno=%s nnp=%d seccomp=%d\n",
        allowed, protected, strerror(protected_errno), inet, strerror(inet_errno),
        status_number("NoNewPrivs:\t%d"), status_number("Seccomp:\t%d"));
    return 0;
}
```

Lesson 10.03

10.03 - Launch, attest, and collect the complete runtime

Goal

Compose the outer cgroup/namespace/privilege launcher with the native guard, make one capability-mediated synthetic operation, emit effective-state telemetry, and prove exact teardown.

Exercise 1 - Read the complete composition

```
sed -n '1,340p' complete_runtime.py
sed -n '1,360p' runtime_probe.c
sed -n '1,220p' demo_broker.py
```

Line by line

- The first file is the outer orchestrator and result writer.
- The second is the confined workload that obtains kernel-visible evidence from inside every layer.
- The third is a one-shot synthetic host-side broker; it owns the fake credential and exact socket lifecycle.

`complete\_runtime.py`, block by block

- `validated` accepts only a regular trusted guard, an absolute directory, an `argv` array whose executable resolves beneath that directory, and bounded CPU, memory, and task values. It rejects before creating a result or unit.
- The exact transient unit name contains the ordinary UID and random material. `systemd-run --wait --collect` creates the cgroup before descendants, applies CPU/memory/swap/PID limits, uses control-group kill semantics, and synchronously collects the unit.
- `unshare` creates user and network namespaces. `setpriv` then removes bounding, inheritable, and ambient capabilities and sets `no_new_privs` after user mapping has created namespace-scoped capabilities.
- The native guard receives the allowed root, installs Landlock and seccomp, and executes the structured workload `argv`. No shell interprets paths or arguments.
- The `systemd` client receives only its IPC variables plus a fixed locale/path; the transient service explicitly unsets the fake credential, and `/usr/bin/env -i` gives the guard only fixed locale/path values. Timeout handling stops only the exact generated unit.
- The result preserves status, stdout, stderr, unit identity, and the parsed final JSON attestation.

Probe and broker, block by block

- `runtime_probe.c` performs real allowed/protected reads; compares its network-namespace inode with the host value; reads its own capability, `NoNewPrivs`, and `Seccomp` fields; and reads effective cgroup controller files.
- Its `AF_INET` call must fail, while its `AF_UNIX` call sends one prebuilt capability request to the broker. The response must be successful and must not contain the public fake-credential prefix marker; the full credential is never in workload `argv`.

- `demo_broker.py` holds the expected synthetic capability and fake credential outside the workload. On one exact request it uses the credential to derive an event proof but returns only a synthetic value. Its finally block unlinks the exact socket.

Exercise 2 - Build a randomized local system

```
DEMO=$(mktemp -d /tmp/north-echo-10.03.XXXXXX)
mkdir "$DEMO/allowed"
gcc -O2 -Wall -Wextra -static \
  ../../course/module-10-complete-runtime/lesson-02/runtime_guard.c \
  -o "$DEMO/runtime_guard" -lseccomp
gcc -O2 -Wall -Wextra -static runtime_probe.c -o "$DEMO/allowed/runtime_probe"
python3 - "$DEMO" <<'PY'
import json, secrets, sys
from pathlib import Path
root = Path(sys.argv[1])
token = "CAP-" + secrets.token_hex(16)
credential = "FAKE-CREDENTIAL-" + secrets.token_hex(16)
(root / "allowed" / "allowed.txt").write_text("allowed\n")
(root / "protected.txt").write_text("synthetic-protected\n")
(root / "token.txt").write_text(token + "\n")
(root / "credential.txt").write_text(credential + "\n")
(root / "allowed" / "request.json").write_text(json.dumps({
    "operation": "read", "resource": "synthetic:record", "token": token,
}, sort_keys=True, separators=(",", ":")) + "\n")
PY
chmod 600 "$DEMO/token.txt" "$DEMO/credential.txt"
HOST_NET=$(stat -Lc '%i' /proc/self/ns/net)
HOST_USER=$(stat -Lc '%i' /proc/self/ns/user)
```

Every identifier is synthetic and fresh. The broker's secret files are outside the allowed root; the request file contains only a narrow bearer capability.

Exercise 3 - Trigger the incomplete composition

Start the one-shot broker and invoke only the inner guard:

```
python3 demo_broker.py "$DEMO/allowed/broker.sock" "$DEMO/token.txt" \
  "$DEMO/credential.txt" "$DEMO/direct-event.json" & direct_broker=$!
for attempt in 1 2 3 4 5; do test -S "$DEMO/allowed/broker.sock" && break; sleep 0.1; done
NORTH_ECHO_FAKE_CREDENTIAL=$(sed -n '1p' "$DEMO/credential.txt") \
  "$DEMO/runtime_guard" "$DEMO/allowed" "$DEMO/allowed/runtime_probe" \
  "$DEMO/allowed/allowed.txt" "$DEMO/protected.txt" "$DEMO/allowed/broker.sock" \
  "$DEMO/allowed/request.json" "$HOST_NET" "$HOST_USER" 'FAKE-CREDENTIAL-' || true
wait "$direct_broker"
```

The deliberate incomplete launch already has Landlock, seccomp, and broker access, but its attestation reports `private_net:false`, host cgroup values, and `credential_absent:false`. Composition is conjunctive: several successful layers do not compensate for missing outer controls.

Exercise 4 - Repair, attest, and collect

Start a fresh one-shot broker, generate the structured spec without shell-built JSON, and run the complete launcher:

```

rm "$DEMO/direct-event.json"
python3 demo_broker.py "$DEMO/allowed/broker.sock" "$DEMO/token.txt" \
"$DEMO/credential.txt" "$DEMO/event.json" & broker_pid=$!
trap 'kill "$broker_pid" 2>/dev/null || true; wait "$broker_pid" 2>/dev/null || true' EXIT
for attempt in 1 2 3 4 5; do test -S "$DEMO/allowed/broker.sock" && break; sleep 0.1; done
python3 - "$DEMO" "$HOST_NET" "$HOST_USER" <<'PY'
import json, sys
from pathlib import Path
root = Path(sys.argv[1])
spec = {
    "guard": str(root / "runtime_guard"),
    "allowed_root": str(root / "allowed"),
    "command": [str(root / "allowed" / "runtime_probe"),
                str(root / "allowed" / "allowed.txt"), str(root / "protected.txt"),
                str(root / "allowed" / "broker.sock"), str(root / "allowed" / "request.json"),
                sys.argv[2], sys.argv[3], "FAKE-CREDENTIAL-"],
    "memory_max": 50331648, "tasks_max": 16, "cpu_percent": 50,
}
(root / "spec.json").write_text(json.dumps(spec) + "\n")
PY
NORTH_ECHO_FAKE_CREDENTIAL="$(sed -n '1p' "$DEMO/credential.txt")" \
python3 complete_runtime.py "$DEMO/spec.json" "$DEMO/result.json"
wait "$broker_pid"
trap - EXIT
python3 -m json.tool "$DEMO/result.json"
python3 -m json.tool "$DEMO/event.json"

```

The attestation must report allowed-read and protected-denial true, direct-IP denial and private user/network namespaces true, zero capability sets, NoNewPrivs 1, Seccomp 2, CPU 50000 100000, memory 50331648, swap 0, PIDs 16, broker success, and credential absence. The separate event proves the outside broker used its credential; neither result contains it.

Prove non-disclosure and teardown before removing exact paths:

```

! grep -F "$(sed -n '1p' "$DEMO/credential.txt")" "$DEMO/result.json"
test ! -e "$DEMO/allowed/broker.sock"
! systemctl --user list-units --all --plain --no-legend 'north-echo-*-10-lab-*.service' | grep .
rm "$DEMO/allowed/runtime_probe" "$DEMO/allowed/allowed.txt" "$DEMO/allowed/request.json"
rm "$DEMO/runtime_guard" "$DEMO/protected.txt" "$DEMO/token.txt" "$DEMO/credential.txt"
rm "$DEMO/spec.json" "$DEMO/result.json" "$DEMO/event.json"
rmdir "$DEMO/allowed" "$DEMO"

```

Checkpoint and troubleshooting

- If the broker socket is not ready, inspect only the owned broker PID and its exact synthetic paths; do not search for or kill name-matched host processes.
- If cgroup values differ, verify the delegated user manager and controllers with `./scripts/linux-preflight`.
- If the probe exits at `execv`, confirm the static workload executable resolves beneath `allowed_root`.
- If broker access fails while `AF_INET` is denied, confirm the socket path is below the allowed root and shorter than the Unix-socket path limit.
- Checkpoint: point to one attested field for each of privilege, filesystem, syscalls, resources, network, and credential mediation, then explain which separate evidence proves teardown.

Complete source: `complete_runtime.py`

Canonical path: `course/module-10-complete-runtime/lesson-03/complete_runtime.py`

```

#!/usr/bin/env python3
"""Launch one workload through the complete North Echo containment stack."""

import json
import os
from pathlib import Path
import secrets
import subprocess
import sys

def integer(spec: dict, name: str, minimum: int, maximum: int) -> int:
    value = spec.get(name)
    if type(value) is not int or not minimum <= value <= maximum:
        raise ValueError(f"{name} must be an integer from {minimum} through {maximum}")
    return value

def validated(spec: dict) -> tuple[Path, Path, list[str], int, int, int]:
    if not isinstance(spec, dict):
        raise ValueError("spec must be an object")
    guard = Path(spec.get("guard", ""))
    root = Path(spec.get("allowed_root", ""))
    command = spec.get("command")
    if not guard.is_absolute() or not guard.is_file() or guard.is_symlink():
        raise ValueError("guard must be an absolute regular non-symlink path")
    if not root.is_absolute() or not root.is_dir() or root.is_symlink():
        raise ValueError("allowed_root must be an absolute directory without a symlink leaf")
    if not isinstance(command, list) or not command or any(not isinstance(item, str) or "\x00" in item for item in command):
        raise ValueError("command must be a nonempty argv string array")
    root = root.resolve(strict=True)
    executable = Path(command[0]).resolve(strict=True)
    if os.path.commonpath((root, executable)) != str(root) or not executable.is_file():
        raise ValueError("command executable must resolve beneath allowed_root")
    command = [str(executable), *command[1:]]
    return (
        guard.resolve(strict=True), root, command,
        integer(spec, "memory_max", 16 * 1024 * 1024, 1024 * 1024 * 1024),
        integer(spec, "tasks_max", 4, 256),
        integer(spec, "cpu_percent", 10, 100),
    )

def main() -> int:
    if len(sys.argv) != 3:
        return 2
    result_path = Path(sys.argv[2])
    try:
        spec = json.loads(Path(sys.argv[1]).read_text(encoding="utf-8"))
        guard, root, command, memory, tasks, cpu = validated(spec)
    except (OSError, ValueError, json.JSONDecodeError) as error:
        print(f"runtime specification denied: {error}", file=sys.stderr)
        return 1

    unit = f"north-echo-{os.getuid()}-10-lab-{secrets.token_hex(6)}.service"
    argv = [
        "systemd-run", "--user", "--wait", "--pipe", "--collect", "--quiet",
        f"--unit={unit}", "--property=KillMode=control-group", "--property=TimeoutStopSec=5s",
        f"--property=CPUQuota={cpu}%", f"--property=MemoryMax={memory}",
        f"--property=MemorySwapMax=0", f"--property=TasksMax={tasks}",
        "--property=UnsetEnvironment=NORTH_ECHO_FAKE_CREDENTIAL",
        "--setenv=PATH=/usr/bin:/bin", "--setenv=LANG=C.UTF-8", "--setenv=LC_ALL=C.UTF-8",
        "--", "unshare", "--user", "--map-root-user", "--net", "--",
        "setpriv", "--bounding-set=-all", "--inh-caps=-all", "--ambient-caps=-all",
        "--no-new-privs", "--", "/usr/bin/env", "-i", "PATH=/usr/bin:/bin",
        "LANG=C.UTF-8", "LC_ALL=C.UTF-8", str(guard), str(root), *command,
    ]
    client_env = {"PATH": "/usr/bin:/bin", "LANG": "C.UTF-8", "LC_ALL": "C.UTF-8"}
    for name in ("XDG_RUNTIME_DIR", "DBUS_SESSION_BUS_ADDRESS"):
        if name in os.environ:
            client_env[name] = os.environ[name]

```

```

try:
    run = subprocess.run(argv, text=True, capture_output=True, check=False,
                          timeout=20, env=client_env)
except subprocess.TimeoutExpired:
    subprocess.run(["systemctl", "--user", "stop", unit], check=False,
                  stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL, env=client_env)
    print("runtime timed out and its exact unit was stopped", file=sys.stderr)
    return 1
attestation = None
try:
    attestation = json.loads(run.stdout.strip().splitlines()[-1])
except (IndexError, json.JSONDecodeError):
    pass
result_path.write_text(json.dumps({
    "schema": 1, "status": run.returncode, "unit": unit,
    "stdout": run.stdout, "stderr": run.stderr, "attestation": attestation,
}, sort_keys=True) + "\n", encoding="utf-8")
return run.returncode != 0

if __name__ == "__main__":
    raise SystemExit(main())

```

Complete source: ``runtime_probe.c``

Canonical path: `course/module-10-complete-runtime/lesson-03/runtime_probe.c`

```

#define _GNU_SOURCE
#include <arpa/inet.h>
#include <errno.h>
#include <fcntl.h>
#include <limits.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/stat.h>
#include <sys/un.h>
#include <unistd.h>

static int read_text(const char *path, char *buffer, size_t size) {
    int fd = open(path, O_RDONLY | O_CLOEXEC);
    if (fd == -1)
        return -1;
    ssize_t count = read(fd, buffer, size - 1);
    close(fd);
    if (count < 0)
        return -1;
    buffer[count] = '\0';
    return 0;
}

static int status_value(const char *status, const char *name, unsigned long long *value) {
    const char *line = strstr(status, name);
    if (!line)
        return -1;
    return sscanf(line + strlen(name), "%llx", value) == 1 ? 0 : -1;
}

static int status_decimal(const char *status, const char *name, int *value) {
    const char *line = strstr(status, name);
    if (!line)
        return -1;
    return sscanf(line + strlen(name), "%d", value) == 1 ? 0 : -1;
}

static int read_cgroup_value(const char *name, char *value, size_t size) {
    char membership[4096], relative[PATH_MAX], path[PATH_MAX];
    if (read_text("/proc/self/cgroup", membership, sizeof(membership)) == -1)
        return -1;
    char *line = strstr(membership, "0::");
    if (!line || sscanf(line, "0::%4095[^\n]", relative) != 1)
        return -1;
    if (snprintf(path, sizeof(path), "/sys/fs/cgroup%s/%s", relative, name) >= (int)sizeof(path))
        return -1;
    if (read_text(path, value, size) == -1)
        return -1;
    value[strcspn(value, "\n")] = '\0';
    return 0;
}

static int broker_request(const char *socket_path, const char *request_path,
                        const char *forbidden_marker, char *response, size_t size) {
    char request[16384];
    if (read_text(request_path, request, sizeof(request)) == -1)
        return 0;
    size_t length = strlen(request);
    if (length == 0 || request[length - 1] != '\n')
        return 0;
    int fd = socket(AF_UNIX, SOCK_STREAM, 0);
    if (fd == -1)
        return 0;
    struct sockaddr_un address = {.sun_family = AF_UNIX};
    if (strlen(socket_path) >= sizeof(address.sun_path)) {
        close(fd);
        return 0;
    }
}

```

```

strcpy(address.sun_path, socket_path);
if (connect(fd, (struct sockaddr *)&address, sizeof(address)) == -1 ||
    write(fd, request, length) != (ssize_t)length) {
    close(fd);
    return 0;
}
shutdown(fd, SHUT_WR);
ssize_t count = read(fd, response, size - 1);
close(fd);
if (count <= 0)
    return 0;
response[count] = '\0';
return strstr(response, "\"ok\":true") != NULL && strstr(response, forbidden_marker) == NULL;
}

int main(int argc, char **argv) {
    if (argc != 8) {
        fprintf(stderr, "usage: %s ALLOWED PROTECTED BROKER REQUEST HOST_NET_INODE HOST_USER_INODE FORBIDDEN_MARKER\n", argv[0]);
        return 2;
    }
    char byte[8], status[16384], cpu[128] = "?", memory[128] = "?";
    char swap[128] = "?", pids[128] = "?", broker_response[65536];
    int allowed_read = read_text(argv[1], byte, sizeof(byte)) == 0;
    errno = 0;
    int protected_fd = open(argv[2], O_RDONLY | O_CLOEXEC);
    int protected_denied = protected_fd == -1 && (errno == EACCES || errno == EPERM);
    if (protected_fd != -1)
        close(protected_fd);

    int inet_fd = socket(AF_INET, SOCK_STREAM, 0);
    int inet_denied = inet_fd == -1 && errno == EPERM;
    if (inet_fd != -1)
        close(inet_fd);

    struct stat net_stat, user_stat;
    unsigned long long host_net = strtoull(argv[5], NULL, 10);
    int private_net = stat("/proc/self/ns/net", &net_stat) == 0 &&
        (unsigned long long)net_stat.st_ino != host_net;
    unsigned long long host_user = strtoull(argv[6], NULL, 10);
    int private_user = stat("/proc/self/ns/user", &user_stat) == 0 &&
        (unsigned long long)user_stat.st_ino != host_user;

    unsigned long long cap_eff = 1, cap_bnd = 1, cap_amb = 1;
    int no_new_privs = 0, seccomp = 0;
    int status_ok = read_text("/proc/self/status", status, sizeof(status)) == 0 &&
        status_value(status, "CapEff:\t", &cap_eff) == 0 &&
        status_value(status, "CapBnd:\t", &cap_bnd) == 0 &&
        status_value(status, "CapAmb:\t", &cap_amb) == 0 &&
        status_decimal(status, "NoNewPrivs:\t", &no_new_privs) == 0 &&
        status_decimal(status, "Seccomp:\t", &seccomp) == 0;
    read_cgroup_value("cpu.max", cpu, sizeof(cpu));
    read_cgroup_value("memory.max", memory, sizeof(memory));
    read_cgroup_value("memory.swap.max", swap, sizeof(swap));
    read_cgroup_value("pids.max", pids, sizeof(pids));
    int broker_ok = broker_request(argv[3], argv[4], argv[7], broker_response, sizeof(broker_response));
    int credential_absent = getenv("NORTH_ECHO_FAKE_CREDENTIAL") == NULL;

    printf("{\n  \"allowed_read\":%s,\n  \"protected_denied\":%s,\n  \"inet_denied\":%s,\n  \"private_net\":%s,\n  \"private_user\":%s,\n  \"caps_zero\":%s,\n  \"no_new_privs\":%d,\n  \"seccomp\":%d,\n  \"cpu\": \"%s\",\n  \"memory\": \"%s\",\n  \"swap\": \"%s\",\n  \"pids\": \"%s\",\n  \"broker_ok\":%s,\n  \"credential_absent\":%s\n}\n",
        allowed_read ? "true" : "false", protected_denied ? "true" : "false",
        inet_denied ? "true" : "false", private_net ? "true" : "false",
        private_user ? "true" : "false",
        status_ok && cap_eff == 0 && cap_bnd == 0 && cap_amb == 0 ? "true" : "false",
        no_new_privs, seccomp, cpu, memory, swap, pids,
        broker_ok ? "true" : "false", credential_absent ? "true" : "false");
    return !(allowed_read && protected_denied && inet_denied && private_net && private_user &&
        status_ok && cap_eff == 0 && cap_bnd == 0 && cap_amb == 0 &&
        no_new_privs == 1 && seccomp == 2 && broker_ok && credential_absent);
}

```

}

Complete source: `demo\_broker.py`

Canonical path: course/module-10-complete-runtime/lesson-03/demo_broker.py

```
#!/usr/bin/env python3
"""One-shot synthetic operation broker for the composition lesson."""

import hashlib
import json
import os
from pathlib import Path
import socket
import sys

def main() -> int:
    if len(sys.argv) != 5:
        return 2
    socket_path, token_path, credential_path, event_path = map(Path, sys.argv[1:])
    token = token_path.read_text(encoding="utf-8").strip()
    credential = credential_path.read_text(encoding="utf-8").strip()
    if socket_path.exists() or socket_path.is_symlink():
        raise SystemExit("broker socket path already exists")
    listener = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
    try:
        listener.bind(str(socket_path))
        os.chmod(socket_path, 0o600)
        listener.listen(1)
        peer, _ = listener.accept()
        with peer:
            request = json.loads(peer.makefile("rb").readline(16384))
            expected = {"operation": "read", "resource": "synthetic:record", "token": token}
            if request != expected:
                response = {"ok": False, "error": "capability denied"}
            else:
                proof = hashlib.sha256((credential + ":read:synthetic:record").encode()).hexdigest()
                event_path.write_text(json.dumps({"credential_used": True, "proof": proof}) + "\n",
                                       encoding="utf-8")
                response = {"ok": True, "result": {"value": "synthetic-result"}}
            peer.sendall(json.dumps(response, sort_keys=True, separators=(",", ":")).encode() + b"\n")
    finally:
        listener.close()
        socket_path.unlink(missing_ok=True)
    return 0

if __name__ == "__main__":
    raise SystemExit(main())
```

Module 10 independent lab

Module 10 independent lab - Composed contained runtime

Implement `complete_runtime.py`. The grader invokes:

```
python3 complete_runtime.py SPEC.json RESULT.json
```

The JSON spec contains an absolute trusted guard executable, an absolute `allowed_root`, structured workload command `argv`, and integer `memory_max`, `tasks_max`, and `cpu_percent` limits. The workload executable must resolve beneath the allowed root. Treat the guard as trusted runtime TCB and every command argument as literal data.

Create a uniquely named transient systemd user service with the validated CPU quota, memory limit, zero swap, PID limit, control-group kill behavior, bounded stop time, synchronous waiting, and collection. Inside that cgroup, create fresh user and network namespaces, then empty bounding/inheritable/ambient capability sets and set `no_new_privs`. Remove `NORTH_ECHO_FAKE_CREDENTIAL` from the workload environment. Invoke the guard with the allowed root and exact workload `argv`; do not use a shell.

Write one result object containing schema 1, workload status, exact unit name, stdout, stderr, and the parsed final-line attestation. Propagate nonzero workload status through a nonzero launcher status. Invalid roots, commands, limits, or shapes must fail before a result or unit exists. On timeout, stop only the exact unit created by this invocation. Successful and failed runs must leave no transient unit.

The external grader compiles its own static guard and workload probe, rotates synthetic paths, capabilities, fake credentials, canaries, and unit names, and runs a separate host-side Unix-socket broker. It verifies:

- exact CPU, memory, swap, and PID controller values;
- different user and network namespaces plus `AF_INET` denial;
- empty effective/bounding/ambient capability sets and `no_new_privs`;
- effective Landlock allowed/protected reads and seccomp filter mode;
- successful `AF_UNIX` capability mediation and broker-side fake-credential use;
- credential absence from the workload and all returned output;
- literal path handling without shell evaluation;
- failure status/stderr propagation, invalid-spec rejection, and exact cleanup.

The starter runs the guard and workload directly, so useful inner controls execute but outer composition properties fail. The lab withholds a complete implementation; use the launch dependency and interface practiced in Lessons 10.01-10.03.

```
python3 -m py_compile complete_runtime.py
../../lab-grade module-10
../../lab-grade module-10 --mode exam
```

- `py_compile` checks syntax without creating a unit.
- Practice mode names the lesson associated with each failed property.
- Exam mode repeats fresh external checks but withholds lesson references.

Use only the grader's generated local objects. Never substitute a real credential, external service, host firewall change, privileged cgroup, or production path.

Integrated security model

The outer-to-inner execution chain is:

```
host-side synthetic broker
-> transient delegated cgroup
  -> user and network namespaces
    -> empty capability sets plus no_new_privs
      -> empty fixed workload environment
        -> Landlock filesystem rules
          -> native default-deny seccomp filter
            -> static workload and effective-state attestation
  -> synchronous unit collection
```

The ordering is policy. Entering the cgroup first accounts for every descendant. Entering the user namespace before capability removal ensures UID-mapping capabilities are removed. Applying Landlock before seccomp preserves setup syscalls while keeping them unavailable to the workload. Starting the broker before removing IP connectivity makes the Unix-socket capability usable without giving the workload a route. Collection after exit covers success and failure.

No field subsumes another. Seccomp: 2 says a filter exists, not that it is useful. A protected-path denial says nothing about inherited descriptors or sockets. Fresh namespaces say nothing about Unix-socket authorization. Empty capabilities do not bound memory. Broker success does not prove credential absence. The grader checks each property independently and requires all of them.

Failure interpretation

- A missing result means validation, setup, or timeout handling failed before a trustworthy record existed.
- A nonzero recorded status with stderr is a workload failure the launcher must preserve.
- Correct cgroup values with `private_net:false` identify a namespace-layer failure.
- Seccomp: 2 with a successful IPv4 socket identifies a policy-content failure.
- Broker success with `credential_absent:false` identifies ambient authority.
- A passing attestation followed by a remaining unit identifies a lifecycle failure.

Cleanup and replay

The guided broker is a foreground, one-shot process retained by exact PID and unlinks its exact socket in `finally`. The launcher uses a random exact unit name, waits synchronously, and requests collection. Timeout stops only that unit. No veth pair, firewall object, host cgroup, background daemon, or public service is created, so the runtime registry needs no new persistent resource type.

Lesson, module, and all-scope reset remove student work and randomized fixtures while preserving only progress metadata. A later start rotates fixture details; every grade creates a separate fresh fixture.

Optional primary references

Required teaching is above. Optional depth: `landlock(7)`, `seccomp(2)`, `proc_pid_status(5)`, `user_namespaces(7)`, `network_namespaces(7)`, `unshare(1)`, `setpriv(1)`, `systemd-run(1)`, `systemd.resource-control(5)`, the cgroup v2 kernel documentation, and `unix(7)`.

Completion checkpoint

A complete run has three evidence classes: workload attestation for effective controls, a broker event proving fake-credential use outside the workload, and post-run unit/socket checks proving teardown. Passing only one or two is not a passing composition.

North Echo field manual - Module 11

This chapter is self-contained for the randomized break/fix research lessons and independent lab. It repeats every guided command, fixture, and complete source listing. All effects are synthetic and local; the harness does not connect to a target or delete cleanup candidates.

The method is evidence first: reproduce a bounded effect, state the invariant it violates, repair every applicable layer, preserve useful behavior, and rerun the same probe against the hardened counterpart.

Module overview

Module 11 - Vulnerable variants and break/fix research

Analyze seeded runtime weaknesses without being told which control changed. Use a deterministic non-agent harness to reproduce effects, map observations to failed invariants, repair the plan, and compare it with a hardened counterpart.

Play in order: 11.01, 11.02, 11.03, then module-11.

Outcomes:

- distinguish a configuration difference from evidence of security impact;
- reproduce ambient credentials, shell interpretation, symlink escape, direct IP authority, and unsafe cleanup selection using synthetic local objects;
- map observed effects to stable cross-layer invariants rather than variant labels;
- repair multiple simultaneous weaknesses while preserving the allowed operation and opaque run identity;
- prove a repair is idempotent and survives fresh randomized variants.

The harness never exploits an external target and never deletes its cleanup candidates. Every credential, path, marker, resource, and socket is synthetic and confined to the disposable workspace. Prerequisites are Modules 01, 04, 05, 08, 09, and 10.

Lesson 11.01

11.01 - Reproduce a seeded weakness without an agent

Goal

Generate a repeatable unknown variant, use a deterministic harness to reproduce its effects, and separate configuration clues from behavioral evidence.

Exercise 1 - Read the complete generator and harness

```
sed -n '1,200p' make_variant.py
sed -n '1,300p' variant_harness.py
```

Line by line

- `make_variant.py` starts from five hardened controls, uses only an explicit integer seed, and weakens two through four randomly selected controls. The seed makes research replayable; it is not security randomness.
- The workload contains one allowed synthetic read and a literal shell-looking argument. Variant identity is opaque evidence metadata.
- `variant_harness.py` creates a new private directory, allowed file, protected sibling, symlink, and marker.
- Execution mode passes the same literal through either `shell=True` or an `argv` array. Environment mode launches `/usr/bin/env` with inherited or minimal variables.
- Filesystem mode compares lexical spelling or the resolved target. Network mode creates, but never connects, an IPv4 socket. Cleanup mode selects candidate names but never deletes them.
- The evidence contains observed effects, not the control labels, and preserves allowed-operation success.

Exercise 2 - Trigger and replay the variant

```
python3 make_variant.py 1101 variant.json
NORTH_ECHO_FAKE_CREDENTIAL=FAKE-LESSON-11.01 \
python3 variant_harness.py variant.json evidence.json run-a
python3 -m json.tool variant.json
python3 -m json.tool evidence.json
```

At least two adverse evidence fields are true. `allowed_operation` remains true. The intentional mistake is to call the changed configuration itself proof: a `direct` field is suspicious, but `inet_created:true` is the reproduced effect.

Replay the exact seed and compare:

```
python3 make_variant.py 1101 replay.json
cmp variant.json replay.json
```

The byte-identical plan makes the observation reproducible. A different seed may select a different weakness set without changing the research method.

Exercise 3 - Prove the harness is bounded

```
test ! -e run-a/not-a-workspace
test -e run-a/protected.txt
find run-a -maxdepth 2 -type f -o -type l | sort
rm run-a/allowed/data.txt run-a/allowed/link.txt run-a/protected.txt
test ! -e run-a/shell-marker || rm run-a/shell-marker
rmdir run-a/allowed run-a
rm variant.json replay.json evidence.json
```

The protected file still exists because the reproducer reads but never modifies it. Cleanup evidence is only a selected-name list inside the harness; no service or host object is removed.

Checkpoint and troubleshooting

- If run-a exists, remove only the exact files listed above before replaying; the harness refuses to reuse a directory.
- If no credential is visible in an inherited variant, confirm the explicitly fake variable prefixes the harness command.
- Do not replace the synthetic literal with a command affecting anything outside the generated directory.
- Checkpoint: identify one field that is configuration and one field that is reproduced evidence.

Complete source: `make_variant.py`

Canonical path: `course/module-11-break-fix-research/lesson-01/make_variant.py`

```
#!/usr/bin/env python3
"""Create a reproducible vulnerable plan from an explicit integer seed."""

import json
import random
import sys
from pathlib import Path

SECURE = {"execution": "argv", "environment": "minimal", "filesystem": "resolved",
          "network": "broker_only", "cleanup": "owned"}
WEAK = {"execution": "shell", "environment": "inherit", "filesystem": "lexical",
        "network": "direct", "cleanup": "prefix"}

if len(sys.argv) != 3:
    raise SystemExit(f"usage: {sys.argv[0]} SEED OUTPUT.json")
rng = random.Random(int(sys.argv[1]))
controls = dict(SECURE)
for name in rng.sample(sorted(WEAK), rng.randint(2, 4)):
    controls[name] = WEAK[name]
plan = {
    "schema": 1,
    "variant_id": f"seed-{sys.argv[1]}",
    "workload": {"operation": "read", "resource": "record:alpha",
                 "literal": "literal ${touch shell-marker}"},
    "controls": controls,
}
Path(sys.argv[2]).write_text(json.dumps(plan, indent=2, sort_keys=True) + "\n", encoding="utf-8")
```

Complete source: ``variant_harness.py``

Canonical path: `course/module-11-break-fix-research/lesson-01/variant_harness.py`

```

#!/usr/bin/env python3
"""Reproduce runtime-plan effects without an autonomous agent or external target."""

import json
import os
from pathlib import Path
import socket
import subprocess
import sys

def main() -> int:
    if len(sys.argv) != 4:
        return 2
    plan = json.loads(Path(sys.argv[1]).read_text(encoding="utf-8"))
    evidence_path, work = Path(sys.argv[2]), Path(sys.argv[3])
    work.mkdir(mode=0o700, parents=True, exist_ok=False)
    allowed = work / "allowed"
    allowed.mkdir()
    (allowed / "data.txt").write_text("allowed\n", encoding="utf-8")
    protected = work / "protected.txt"
    protected.write_text("SYNTHETIC-PROTECTED\n", encoding="utf-8")
    (allowed / "link.txt").symlink_to(protected.resolve())
    marker = work / "shell-marker"
    literal = plan["workload"]["literal"].replace("shell-marker", str(marker))
    controls = plan["controls"]

    if controls["execution"] == "shell":
        subprocess.run("/usr/bin/printf '%s\\n' " + literal, shell=True, cwd=work,
                       text=True, capture_output=True, check=False)
    else:
        subprocess.run(["/usr/bin/printf", "%s\\n", literal], cwd=work,
                       text=True, capture_output=True, check=False)

    child_env = dict(os.environ) if controls["environment"] == "inherit" else {
        "PATH": "/usr/bin:/bin", "LANG": "C.UTF-8"
    }
    environment = subprocess.run(["/usr/bin/env"], text=True, capture_output=True,
                                  check=False, env=child_env).stdout
    credential_visible = "NORTH_ECHO_FAKE_CREDENTIAL=" in environment

    candidate = allowed / "link.txt"
    if controls["filesystem"] == "lexical":
        authorized = str(candidate).startswith(str(allowed) + os.sep)
    else:
        authorized = allowed.resolve() in candidate.resolve().parents
    protected_read = authorized and candidate.read_text(encoding="utf-8").startswith("SYNTHETIC-")

    inet_created = False
    if controls["network"] == "direct":
        stream = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        inet_created = stream.fileno() >= 0
        stream.close()

    owned = f"north-echo-{plan['variant_id']}-owned"
    decoy = f"north-echo-{plan['variant_id']}-decoy"
    candidates = [owned, decoy]
    selected = candidates if controls["cleanup"] == "prefix" else [owned]
    evidence = {
        "schema": 1,
        "variant_id": plan["variant_id"],
        "allowed_operation": plan["workload"]["operation"] == "read",
        "shell_marker_created": marker.exists(),
        "credential_visible": credential_visible,
        "protected_read": protected_read,
        "inet_created": inet_created,
        "cleanup_decoy_selected": decoy in selected,
    }
    evidence_path.write_text(json.dumps(evidence, indent=2, sort_keys=True) + "\n", encoding="utf-8")
    return 0
if __name__ == "__main__":

```

```
raise SystemExit(main())
```

Lesson 11.02

11.02 - Identify invariants from an evidence matrix

Goal

Map observable failure signals to stable invariants, then show why repairing only the most visible symptom does not harden a mixed variant.

Exercise 1 - Read the classifier and evidence

```
sed -n '1,220p' classify_evidence.py
python3 -m json.tool overfit-evidence.json
python3 -m json.tool mixed-evidence.json
```

Line by line

- INVARIANTS maps each bad effect to a positive statement that must remain true across implementations.
- The classifier selects only signals explicitly observed as true. It does not guess at missing fields or inspect a variant's control labels.
- `overfit-evidence.json` has one dramatic shell marker. `mixed-evidence.json` has no marker but shows ambient credential, protected read, and cleanup over-selection.

Exercise 2 - Make and expose an overfit diagnosis

```
python3 classify_evidence.py overfit-evidence.json
python3 classify_evidence.py mixed-evidence.json
```

The first output names the argv invariant. The intentional mistake is to generalize that single diagnosis to all variants. The second output names three different invariants and contains no shell failure. A repair chosen from exploit theatrics rather than a property matrix would miss them.

Exercise 3 - Build the research matrix

```
python3 - <<'PY'
import json
for name in ("overfit-evidence.json", "mixed-evidence.json"):
    value = json.load(open(name))
    bad = sorted(key for key, state in value.items() if key not in {"schema", "variant_id", "allowed_operation"} and state)
    print(name, "allowed=", value["allowed_operation"], "bad=", ", ".join(bad) or "none")
PY
```

Line by line

- The loop reads both fixed evidence rows rather than relying on memory of one run.
- Metadata and the positive functional signal are separated from adverse booleans.

- Sorting makes comparison stable. The matrix preserves the fact that useful work succeeds even when containment fails.

Checkpoint and troubleshooting

```
test "$(python3 classify_evidence.py mixed-evidence.json | grep -c 'invariant')" -ge 3
```

- If the count differs, inspect the complete JSON rather than changing the classifier to match a desired answer.
- An absence of one signal proves only that probe did not observe that effect; it is not universal proof of safety.
- Checkpoint: state the filesystem invariant without naming `lexical`, `resolved`, or a particular exploit path.

Complete source: `classify\_evidence.py`

Canonical path: course/module-11-break-fix-research/lesson-02/classify_evidence.py

```
#!/usr/bin/env python3
"""Map reproduced effects to invariant statements."""

import json
import sys
from pathlib import Path

INVARIANTS = {
    "shell_marker_created": "argv is data and is never shell syntax",
    "credential_visible": "the workload receives no ambient credential",
    "protected_read": "authorization follows resolved objects, not lexical paths",
    "inet_created": "the workload has no direct IP socket authority",
    "cleanup_decoy_selected": "cleanup selects only exact owned objects",
}

if len(sys.argv) != 2:
    raise SystemExit(f"usage: {sys.argv[0]} EVIDENCE.json")
evidence = json.loads(Path(sys.argv[1]).read_text(encoding="utf-8"))
failed = [{"signal": name, "invariant": statement}
          for name, statement in INVARIANTS.items() if evidence.get(name) is True]
print(json.dumps({"variant_id": evidence.get("variant_id"), "failed": failed}, indent=2, sort_keys=True))
raise SystemExit(not failed)
```

Complete fixture: `overfit-evidence.json`

Canonical path: `course/module-11-break-fix-research/lesson-02/overfit-evidence.json`

```
{"schema":1,"variant_id":"lesson-overfit","allowed_operation":true,"shell_marker_created":true,"credential_visible":false,
```

Complete fixture: `mixed-evidence.json`

Canonical path: `course/module-11-break-fix-research/lesson-02/mixed-evidence.json`

```
{"schema":1,"variant_id":"lesson-mixed","allowed_operation":true,"shell_marker_created":false,"credential_visible":true,"
```

Lesson 11.03

11.03 - Repair and prove the hardened counterpart

Goal

Repair all known weak controls in one pass, preserve the workload and variant identity, prove behavior with the same harness, and require idempotence.

Exercise 1 - Read the complete repair tool

```
sed -n '1,300p' repair_variant.py
python3 -m json.tool vulnerable-plan.json
```

Line by line

- SECURE expresses the five positive control choices; ALLOWED bounds both secure and deliberately weak input values.
- repair requires the exact top-level, workload, and control shapes, validates types and known values, copies opaque workload/identity fields, and replaces only the controls.
- main parses completely before output, writes a sibling temporary file, and atomically replaces the result. Invalid input leaves no claimed repair.
- Applying the function to an already hardened plan produces the same JSON object: repair is idempotent.

Exercise 2 - Observe the vulnerable counterpart

```
cp vulnerable-plan.json before.json
NORTH_ECHO_FAKE_CREDENTIAL=FAKE-LESSON-11.03 \
python3 ../../course/module-11-break-fix-research/lesson-01/variant_harness.py \
before.json before-evidence.json before-run
python3 -m json.tool before-evidence.json
```

All five adverse signals are true while allowed_operation is true. This is the intentional vulnerable counterpart, not an instruction to target another system.

Exercise 3 - Repair and compare

```
python3 repair_variant.py before.json after.json
NORTH_ECHO_FAKE_CREDENTIAL=FAKE-LESSON-11.03 \
python3 ../../course/module-11-break-fix-research/lesson-01/variant_harness.py \
after.json after-evidence.json after-run
python3 -m json.tool after-evidence.json
python3 repair_variant.py after.json second.json
cmp after.json second.json
```

Expected: the allowed operation stays true and every adverse signal is false. cmp proves a second repair does not mutate an already hardened plan.

Clean the exact generated objects:

```
rm before-run/allowed/data.txt before-run/allowed/link.txt before-run/protected.txt before-run/shell-marker
rmdir before-run/allowed before-run
rm after-run/allowed/data.txt after-run/allowed/link.txt after-run/protected.txt
rmdir after-run/allowed after-run
rm before.json after.json second.json before-evidence.json after-evidence.json
```

Checkpoint and troubleshooting

- If the hardened evidence still contains a true adverse field, compare that field with the corresponding invariant before editing another layer.
- If cleanup says a marker is absent, remove it conditionally; only the vulnerable execution mode creates it.
- If relative canonical paths fail, confirm the command is run from `.student/11.03`.
- Checkpoint: explain why preserving workload bytes and allowed behavior matters as much as making adverse signals false.

Complete source: `repair\_variant.py`

Canonical path: course/module-11-break-fix-research/lesson-03/repair_variant.py

```
#!/usr/bin/env python3
"""Repair every known weak control while preserving the workload contract."""

import json
from pathlib import Path
import sys

SECURE = {"execution": "argv", "environment": "minimal", "filesystem": "resolved",
          "network": "broker_only", "cleanup": "owned"}
ALLOWED = {
    "execution": {"argv", "shell"}, "environment": {"minimal", "inherit"},
    "filesystem": {"resolved", "lexical"}, "network": {"broker_only", "direct"},
    "cleanup": {"owned", "prefix"},
}

def repair(value: object) -> dict:
    if not isinstance(value, dict) or set(value) != {"schema", "variant_id", "workload", "controls"}:
        raise ValueError("invalid plan shape")
    if value["schema"] != 1 or not isinstance(value["variant_id"], str) or not value["variant_id"]:
        raise ValueError("invalid plan identity")
    workload, controls = value["workload"], value["controls"]
    if not isinstance(workload, dict) or set(workload) != {"operation", "resource", "literal"}:
        raise ValueError("invalid workload")
    if not all(isinstance(item, str) for item in workload.values()):
        raise ValueError("invalid workload values")
    if not isinstance(controls, dict) or set(controls) != set(SECURE):
        raise ValueError("invalid controls")
    if any(controls[name] not in ALLOWED[name] for name in SECURE):
        raise ValueError("unknown control value")
    return {"schema": 1, "variant_id": value["variant_id"],
            "workload": dict(workload), "controls": dict(SECURE)}

def main() -> int:
    if len(sys.argv) != 3:
        return 2
    output = Path(sys.argv[2])
    try:
        source = json.loads(Path(sys.argv[1]).read_text(encoding="utf-8"))
        repaired = repair(source)
    except (OSError, ValueError, json.JSONDecodeError) as error:
        print(f"repair denied: {error}", file=sys.stderr)
        return 1
    temporary = output.with_name(output.name + ".tmp")
    temporary.write_text(json.dumps(repaired, indent=2, sort_keys=True) + "\n", encoding="utf-8")
    temporary.replace(output)
    return 0

if __name__ == "__main__":
    raise SystemExit(main())
```

Complete fixture: `vulnerable-plan.json`

Canonical path: course/module-11-break-fix-research/lesson-03/vulnerable-plan.json

```
{"schema":1,"variant_id":"lesson-repair","workload":{"operation":"read","resource":"record:alpha","literal":"literal $(to
```

Module 11 independent lab

Module 11 independent lab - Repair randomized runtime variants

Implement `repair_variant.py`. The grader invokes:

```
python3 repair_variant.py INPUT.json OUTPUT.json
```

Input is a schema-1 object with opaque `variant_id`, an exact workload object (`operation`, `resource`, `literal`), and five controls: `execution`, `environment`, `filesystem`, `network`, and `cleanup`. Each control contains either the hardened value practiced in Lesson 11.03 or its deliberately weak counterpart. One through five weaknesses may coexist.

Validate the complete shape and known values before writing output. Preserve the input file, variant identity, and workload exactly. Repair every control, not only the first observed weakness. Write a valid plan atomically. A malformed or unknown plan must return nonzero without output. Applying the repair to its own output must be idempotent.

The external grader uses fresh identities, resources, literals, canaries, and six weakness combinations. It evaluates each output with the canonical non-agent harness. The allowed operation must still work; literal argv must not create a marker; the fake credential must be absent; the protected symlink target must not be read; direct IPv4 socket authority must be absent; and cleanup must exclude an unowned prefix-matching decoy. The harness never connects, deletes a candidate, or touches an external target.

The starter repairs only execution mode, so it is functional but fails mixed variants. The lab withholds the full repair implementation.

```
python3 -m py_compile repair_variant.py
../../lab-grade module-11
../../lab-grade module-11 --mode exam
```

- Syntax checking creates no research object.
- Practice mode reports failed properties with lesson references.
- Exam mode repeats fresh variants while withholding repair hints.

Use only generated synthetic plans. Do not turn the evidence harness into an external scanner or destructive cleanup tool.

Integrated research method

A variant label is not a finding. The generator supplies a replayable hypothesis; the non-agent harness produces effects; the evidence matrix maps effects to invariants; and the repair is accepted only after the same harness shows useful behavior remains while every adverse signal becomes false.

The five invariants deliberately span earlier modules:

- `argv` is data, never shell syntax;
- the workload receives no ambient credential;
- filesystem authorization follows resolved objects;
- the workload has no direct IP socket authority;
- cleanup selects only exact owned objects.

Multiple failures can coexist. Repairing the most dramatic signal is overfitting. The lab varies combination size and order, checks preservation, then applies the repair again to prove convergence rather than repeated mutation.

Interpretation limits

A false signal means this bounded probe did not reproduce that effect for this plan. It is not universal assurance, exploit absence, or a claim about unrelated implementations. A true signal is sufficient evidence that the corresponding invariant failed in the synthetic harness. The module favors small reproducible observations over novelty or exploit spectacle.

Safety and cleanup

The shell marker is constrained to a fresh lesson directory. IPv4 testing creates an unconnected socket only. The fake credential has no external authority. The protected file contains a literal synthetic string. Cleanup candidates are compared as strings and never removed by the harness. Lesson cleanup names exact generated paths; platform reset handles student work and fixtures.

Optional primary references

Required teaching is above. Optional depth: `subprocess(3)` Python documentation, `openat2(2)`, `unix(7)`, `network_namespaces(7)`, and the earlier North Echo module chapters governing each invariant.

Completion checkpoint

A credible repair package contains the input plan, bounded reproducer, before evidence, invariant statement, repaired plan, after evidence, functional preservation, idempotence result, and exact cleanup record. Removing any one of these weakens the conclusion.

North Echo field manual - Module 12

This self-contained chapter preserves every guided command and complete source for the bounded adaptive-adversary module. All observations come from a synthetic local oracle.

[README.md](#)

Module 12 - Adaptive adversary and Boundary Atlas graduation

Compare a fixed scripted baseline with a bounded adaptive policy against the same synthetic local oracle. Preserve every action and observation, distinguish non-discovery from evidence of safety, and package a reproducible candidate experiment.

Play in order: 12.01, 12.02, 12.03, then module-12.

Outcomes:

- define a deterministic baseline before evaluating adaptation;
- constrain an adaptive policy by probe allowlist, exact argv, action budget, and structured trace;
- discover randomized weakness classes without external targets or exploit generation;
- report `not_observed` without claiming security when evidence is absent;
- package hypothesis, trace, result, limits, and replay command as a Boundary Atlas candidate.

All oracles and observations are synthetic and local. No public, LAN, cloud, employer, or production target is contacted. Prerequisites are Modules 04, 10, and 11.

12.01 - Establish a scripted baseline and its limits

Goal

Run a fixed probe sequence before adaptation, deliberately miss a weakness outside that sequence, and report non-discovery without claiming safety.

```
sed -n '1,180p' local_oracle.py
sed -n '1,180p' scripted_baseline.py
python3 scripted_baseline.py local_oracle.py network-scenario.json baseline.json
python3 -m json.tool baseline.json
```

Line by line

- The oracle accepts only five named synthetic probes plus inventory; it performs no network request.
- The baseline always runs argv, credential, and filesystem probes in that order using exact argv.
- The network scenario is intentionally outside that fixed set, so status is not_observed even though a weakness exists.
- The trace preserves every action and observation. This proves what ran, not that untested boundaries are safe.

Intentional mistake: rewrite not_observed as secure. The trace contradicts that claim because network and cleanup were never probed. Restore the calibrated status.

Checkpoint and troubleshooting

```
test "$(python3 -c 'import json;print(json.load(open("baseline.json"))["status"])' )" = not_observed
rm baseline.json
```

- Run from the lesson workspace so scenario paths resolve.
- Checkpoint: name the two untested surfaces and explain why the baseline remains useful for comparison.

lesson-01/local_oracle.py

```
#!/usr/bin/env python3
"""Synthetic one-weakness oracle: JSON scenario plus one named probe."""
import json, sys
from pathlib import Path

PROBES = ("argv", "credential", "filesystem", "network", "cleanup")
if len(sys.argv) != 3 or sys.argv[2] not in ("inventory", *PROBES):
    raise SystemExit(2)
scenario = json.loads(Path(sys.argv[1]).read_text())
weakness = scenario.get("weakness")
probe = sys.argv[2]
if probe == "inventory":
    value = {"ok": True, "surface": list(PROBES), "next_probe": weakness}
else:
    value = {"ok": True, "probe": probe, "observed": probe == weakness,
            "evidence": f"synthetic-{probe}-effect" if probe == weakness else None}
print(json.dumps(value, sort_keys=True))
```

lesson-01/scripted_baseline.py

```
#!/usr/bin/env python3
"""Run the same fixed probes in the same order for every scenario."""
import json, subprocess, sys
from pathlib import Path

if len(sys.argv) != 4:
    raise SystemExit(2)
oracle, scenario, output = sys.argv[1:]
trace = []
for probe in ("argv", "credential", "filesystem"):
    run = subprocess.run([sys.executable, oracle, scenario, probe], text=True,
                        capture_output=True, check=False)
    observation = json.loads(run.stdout)
    trace.append({"probe": probe, "observation": observation})
finding = next((row for row in trace if row["observation"].get("observed")), None)
Path(output).write_text(json.dumps({"strategy": "scripted", "status": "observed" if finding else "not_observed",
                                   "trace": trace, indent=2, sort_keys=True) + "\n")
```

lesson-01/network-scenario.json

```
{"schema":1,"weakness":"network"}
```


12.02 - Adapt within an action budget

Goal

Use one inventory observation to choose one focused probe, preserve an auditable trace, and discover the same network weakness in two actions.

```
sed -n '1,260p' adaptive_runner.py
python3 adaptive_runner.py network-spec.json result.json
python3 -m json.tool result.json
```

Line by line

- The runner validates the exact spec, structured oracle command, integer budget, and run identity.
- Action one is always `inventory`. Its `next_probe` value must be in the fixed allowlist.
- If budget remains, action two invokes that probe with exact argv and a timeout; no shell or generated exploit is used.
- Finding, calibrated claim, budget use, and the complete observations are written together.

Expected status is observed, actions used is 2, and finding probe is `network`.

Intentional failure and repair:

```
python3 -c 'import json;p=json.load(open("network-spec.json"));p["budget"]=1;open("short.json","w").write(json.dumps(p))'
python3 adaptive_runner.py short.json short-result.json
grep 'not established' short-result.json
python3 adaptive_runner.py network-spec.json result.json
rm short.json short-result.json result.json
```

Budget one cannot run the focused probe; the repair restores budget two rather than overstating inventory evidence.

Checkpoint and troubleshooting

- If the oracle path fails, preserve the three argv elements in `oracle_command`; do not join them into a shell string.
- Checkpoint: show which observation caused action two and which field proves the budget was respected.

lesson-02/adaptive_runner.py

```
#!/usr/bin/env python3
"""Use one inventory observation to choose one bounded focused probe."""
import json, subprocess, sys
from pathlib import Path

ALLOWED = {"inventory", "argv", "credential", "filesystem", "network", "cleanup"}

def main():
    if len(sys.argv) != 3:
        return 2
    spec = json.loads(Path(sys.argv[1]).read_text())
    if not isinstance(spec, dict) or set(spec) != {"oracle_command", "budget", "run_id"}:
        return 1
    command, budget = spec["oracle_command"], spec["budget"]
    if not isinstance(command, list) or not command or any(not isinstance(x, str) for x in command):
        return 1
    if type(budget) is not int or not 1 <= budget <= 8 or not isinstance(spec["run_id"], str):
        return 1
    trace = []
    for probe in ("inventory",):
        run = subprocess.run([*command, probe], text=True, capture_output=True, timeout=3, check=False)
        try: observation = json.loads(run.stdout)
        except json.JSONDecodeError: return 1
        trace.append({"action": len(trace) + 1, "probe": probe, "observation": observation})
    suggested = trace[0]["observation"].get("next_probe")
    if suggested is not None and budget > 1:
        if suggested not in ALLOWED or suggested == "inventory": return 1
        run = subprocess.run([*command, suggested], text=True, capture_output=True, timeout=3, check=False)
        try: observation = json.loads(run.stdout)
        except json.JSONDecodeError: return 1
        trace.append({"action": 2, "probe": suggested, "observation": observation})
    finding = next((row for row in trace if row["observation"].get("observed") is True), None)
    result = {"schema": 1, "run_id": spec["run_id"], "budget": budget,
              "actions_used": len(trace), "status": "observed" if finding else "not_observed",
              "finding": finding, "claim": "bounded probe observed a synthetic effect" if finding else
              "no effect observed within this bounded probe budget; security is not established", "trace": trace}
    Path(sys.argv[2]).write_text(json.dumps(result, indent=2, sort_keys=True) + "\n")
    return 0

if __name__ == "__main__": raise SystemExit(main())
```

lesson-02/network-spec.json

```
{"oracle_command":["python3","../../course/module-12-adaptive-adversary/lesson-01/local_oracle.py","../../course/module-12-adaptive-adversary/lesson-02/network-spec.json"]}
```


12.03 - Package a calibrated candidate experiment

Goal

Turn a bounded result into a replayable experiment candidate without expanding its claim beyond the evidence.

```
sed -n '1,180p' package_experiment.py
python3 ../../course/module-12-adaptive-adversary/lesson-02/adaptive_runner.py \
    ../../course/module-12-adaptive-adversary/lesson-02/network-spec.json result.json
python3 package_experiment.py result.json candidate-12 package.json
python3 -m json.tool package.json
```

Line by line

- The packager accepts only a schema-1 observed/not-observed result.
- It preserves the trace and claim, adds a falsifiable bounded hypothesis, lists four interpretation limits, and supplies structured replay argv.
- `candidate_id` labels this package; it is not a vulnerability identifier or publication claim.

Intentional mistake: remove the `non-discovery is not proof` limit. The package then invites an inference its evidence cannot support. Restore all limits and verify:

```
grep -F 'non-discovery is not proof' package.json
python3 package_experiment.py result.json candidate-12 second.json
cmp package.json second.json
rm result.json package.json second.json
```

Checkpoint and troubleshooting

- Package the raw trace, not a prose-only summary.
- A promising observation remains a candidate until independently reproduced under a broader protocol.
- Checkpoint: distinguish the package hypothesis, observed result, and stated limits.

lesson-03/package_experiment.py

```
#!/usr/bin/env python3
"""Package a runner result as a replayable candidate experiment."""
import json, sys
from pathlib import Path

if len(sys.argv) != 4: raise SystemExit(2)
result = json.loads(Path(sys.argv[1]).read_text())
if result.get("schema") != 1 or result.get("status") not in {"observed", "not_observed"}:
    raise SystemExit("invalid result")
package = {"schema": 1, "candidate_id": sys.argv[2],
           "hypothesis": "a bounded focused probe can reproduce the indicated synthetic boundary effect",
           "result_status": result["status"], "claim": result["claim"], "trace": result["trace"],
           "limits": ["synthetic local oracle", "single run", "bounded probe set", "non-discovery is not proof"],
           "replay": ["python3", "adaptive_runner.py", "SPEC.json", "RESULT.json"]}
Path(sys.argv[3]).write_text(json.dumps(package, indent=2, sort_keys=True) + "\n")
```

lab/README.md

Module 12 independent lab - Bounded adaptive runner

Implement `adaptive_runner.py`; the grader calls `python3 adaptive_runner.py SPEC.json RESULT.json`. The spec contains exact `oracle_command` `argv`, integer action `budget`, and opaque `run_id`.

Validate before acting. Invoke only `inventory`, `argv`, `credential`, `filesystem`, `network`, or `cleanup`, using exact `argv` without a shell and with bounded execution. Record sequential action numbers and complete parsed observations. Never exceed budget. Preserve run identity. For an observed focused effect, return its trace row as the finding. If no effect is observed, use `not_observed` and explicitly say security is not established.

The grader rotates five hidden synthetic weakness classes plus a no-weakness case. It checks discovery within two actions, allowlisting, trace integrity, calibrated non-discovery, determinism, and identity preservation. The starter inventories only and therefore misses every weakness.

```
python3 -m py_compile adaptive_runner.py
../../lab-grade module-12
../../lab-grade module-12 --mode exam
```

Do not inspect, modify, or target anything outside the supplied local oracle command.

Integrated interpretation

The scripted and adaptive runs share an oracle and evidence schema. Adaptation changes probe selection, not the target or success definition. Budget, allowlist, exact argv, deterministic trace, and calibrated claims keep comparison meaningful. A `not_observed` result says only that this bounded run found no effect; it never establishes security.

Safety and graduation

The oracle is local, synthetic, deterministic, and non-destructive. The module creates no network connection, credential, service, or persistent resource. A Boundary Atlas candidate is a reproducible experiment package—not a vulnerability announcement. It preserves hypothesis, trace, result, replay argv, and interpretation limits for independent review.